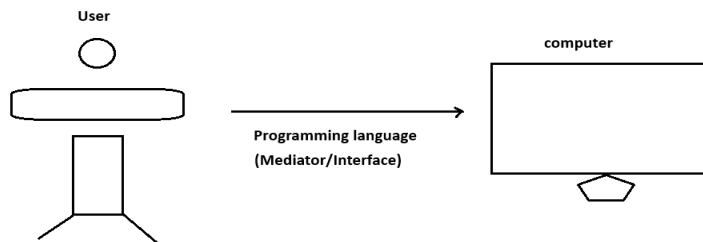


Programming language

=====

Diagram: class1.1



A language which is used to communicate between user and computer is called programming language.

Programming language acts like a mediator or interface between user and computer.

Programming language divided into two types.

1) Low Level Language

2) High Level Language

1) Low Level Language

A language which is understood by a computer easily is called low level language.

A language which is computer dependent is called low level language.

ex:

Machine Language
Assembly Language

Machine Language

It is a fundamental language of a computer which is combination of 0's and 1's.

It is also known as binary language.

A computer may understands many languages but to understand machine language it does not required any translator.

Advantages:

> A program writtens in machine language consumes very less memory.

> It does not required any translator.

> It is more efficient when compare to other languages.

Disadvantages:

> It is a burdun on a programmer to remember dozen's of binary code.

> If anywhere error raised in our program then locating and handling that error becomes very difficult.

> Modifications can't be done easily.

Assembly Language

The second generation language came into an existence is called assembly language.

Assembly language is a replacement of symbols and letters for our mathematical programing code.

Assembly language is a symbolic language.

Assembly language can't understand by a computer directly. We required translator.

We have three translators.

i) Assembler

ii) compiler

iii) Interpreter

i) Assembler

It is one of the translator which converts assemblic code to machine code.

Merits:

> If anywhere error raised in our program then locating and handling that error becomes easy.

> Modifications can be done easily.

Demerits:

> It is a mind trick to remember all symboli code.

> It requires translator.

> It is less efficient when compare to machine language.

Interview Question

=====

Q) What is debugging?

Bug is also known as error.

The process of removing the bugs from the application is called debugging.

2) High Level Language

A language which is understand by a user easily is called high level language.

A language which is user dependent is called high level language.

ex:

C++, Java, .Net, Python and etc.

High level language can't understand by a computer directly. We required translators.

compiler

It compiles and executes our program at a time.

interpreter

It is used to execute our program line by line procedure.

Advantages:

> It is easy to learn and easy to use because it is similar to english language.

> Debugging can be done easily.

> Modifications can be done easily.

Disadvantages:

> A program writtens in high level language consumes huge amount of memory.

> It requires translator.

> It is not efficient when compare to low level languages.

Flavours in Java

=====

We have three flavours in Java.

```

                        Java
    |-----|-----|
    JSE/J2SE      JEE/J2EE      JME/J2ME
  (Java Standard Edition)  (Java Enterprise Edition)  (Java Micro
Edition)
```

> Standalone app

> Distributed app

> Mobile App

> Desktop app

> Enterprise app

> Two-tier app

> ERP app

> N-tier app

> Standalone app

A normal java program which contains main method is called standalone application.

ex:

```
class Test
{
    public static void main(String[] args)
    {
        -
        - //some logic
        -
    }
}
```

> Desktop app

It is a software application or a program specially designed to perform particular task.

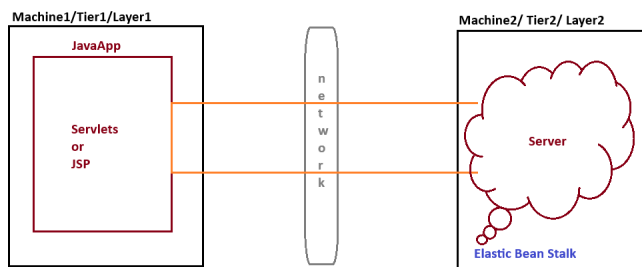
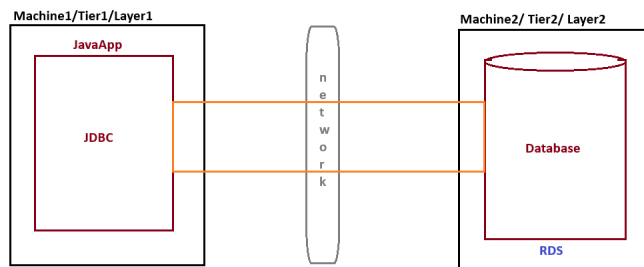
ex:

Control Panel
Recycle Bin
VLC media player
and etc.

> Two-tier app

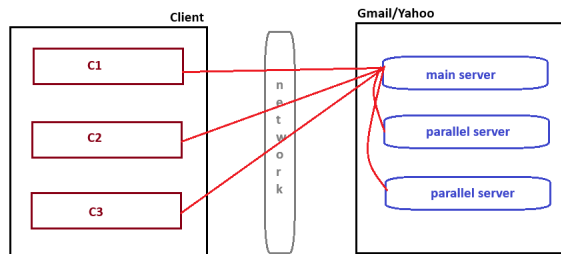
Having more than one tier is called two-tier application.

Diagram: class2.1



> Distributed app

Diagram: class2.3



In client/server architecture, if multiple requests go to main server then main server will distributed those request to it's parallel servers to reduce the burdun of main server such type of application is called distributed application.

> Enterprise app

An application which deals with large complex business logic with the help of middleware services is called enterprise application.

Middleware services means authentication, autherization, malware production, security and etc.

ex:

SAP
HRM
Zooming Meeting
Skype
online shopping website
and etc.

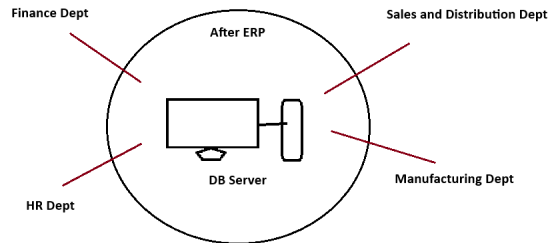
> ERP app

ERP stands for Enterprise Resource Planning.

It is used to integrate disparate functionsâ€”such as finance, HR, manufacturing, and supply chainâ€”into a single, cohesive platform.

Diagram: class2.4

Amazon :



> N-tier app

Having more than two tiers is called N-tier application.

> Mobile App

It is a software application or a program specially designed for wireless network devices like phone, cell, tab, cellular and etc rather than laptop's and pc's.

ex:

PubG
Gpay
and etc.

Note:

JME flavour is outdated from java because we got many alternate technologies to develop mobile applications like Android, Flutter, Swift Programming and etc.

Interview Question

=====

Q) What is Java?

Java is a versatile programming language which is used to develop various applications like standalone applications, desktop applications, distributed applications, enterprise applications, ERP applications, Mobile application, networking application and etc.
Naming conventions in java

=====

In java, uppercase letters consider as different and lowercase letters consider as different. Hence we consider java is a case sensitive programming language.

As java is a case sensitive, we must and should follow naming conventions for following things.

ex:

- classes
- interfaces
- variables
- methods
- keywords
- packages
- constants

classes

In java, a class name must and should starts with uppercase letter and if it contains multiple words then each inner word must starts with initcap.

ex:

Predefined classes	Userdefined classes
-----	-----
System	Test
String	TestApp
FileWriter	DemoApp
BufferedReader	MyProgram
and etc.	and etc.

interfaces

In java, an interface name must and should starts with uppercase letter and if it contains multiple words then each inner word must starts with initcap.

ex:

Predefined interfaces	Userdefined interfaces
-----	-----
Serializable	ITest
ListIterator	IDemoApp
Enumeration	IMyProgram
Iterator	IMyExample
and etc.	and etc.

variables

A variable name must and should starts with lowercase letter and if it contains multiple words then each inner word must start with initcap.

ex:

Predefined variables	Userdefined variables
-----	-----
length	a
out	empId
in	studName
err	deptNo

and etc.

and etc.

Methods

In java, a method name must and should starts with lowercase letter and if it contains multiple words then each inner word must starts with initcap.

ex:

Predefined methods

hashCode()
toString()
getPriority()
setName()
and etc.

Userdefined methods

getDetails()
setStudentInfo()
calculateBill()
getEmployeeDetails()
and etc.

keywords

In java, all keywords we need to declare under lowercase letters only.

ex:

Predefined keywords

class, public, static, void, int, for, do, while and etc.

packages

In java, all packages we need to declare under lowercase letters only.

ex:

Predefined packages

java.lang
java.io
java.util
java.util.stream
java.time
java.text
and etc.

Userdefined packages

com.ihub.www
com.qt.www
com.google.www
and etc.

constants

In java, all constants we need to declare under uppercase letters only.

ex:

Predefined constants

MAX_PRIORITY
MIN_PRIORITY
NORM_PRIORITY
MAX_VALUE

Userdefined constants

DRIVER
N
LIMIT
URL

```
MIN_VALUE          and etc.
and etc.
Java
=====
Version            :      JDK 25

Vendor             :      Oracle Corporation

Creator            :      James Gosling

Open Source        :      Open Source

Website            :      www.oracle.com/java

Download link      :

https://www.oracle.com/java/technologies/downloads/#jdk25-windows
```

Steps to setup environment variables

=====

step1:

Make sure JDK 25 installed successfully.

step2:

Copy bin directory from "JAVA-Home" folder.

ex:

C:\Program Files\Java\jdk-25\bin

step3:

Paste "bin" directory to environment variables.

ex:

Right click to This PC --> properties -->
Advanced system settings --> Environment variables
--> goto System variables --> click to new button -->
variable name : path
variable value : C:\Program Files\Java\jdk-25\bin;
---> ok --> ok --> ok.

step4:

Check environment setup done perfectly or not.

ex:

cmd> javap
cmd> java -version

Steps to develop first java application

=====

step1:

Create a "javaprogram" folder inside 'D' drive.

step2:

Open the notepad and type simple Hello world program.

ex:

latest approach

void main()

{

System.out.println("Hello World - Batch58");

}

traditional approach

class Test

{

public static void main(String[] args)

{

System.out.println("I love Java");

}

}

step3:

Save above program with any name inside "Javaprogram" folder.

ex:

Test.java

Demo.java

Example.java

Sample.java

Main.java

and etc.

step4:

Open the command prompt from "javaprogram" location.

step5:

Compile above program by using below command.

ex:

javac Test.java (This process is called compilation)

step6:

Run above program by using below command.

ex:

```
java Test      (This process is called execution)
```

Note:

From java 11, we can compile and execute our program at a time.

ex:

```
java Test.java
```

History of Java

=====

In 1990 , Sun Micro System took one project to develop a software called consumer electronic device which can be controlled by a remote like setup box. That time project was called Stealth project and later it is renamed to Green project.

James Gosling, Mike Sheradin and Patrick Naughton were there to develop the project and they have met in a place called Aspan/Colarado to start the work with Graphic System. James Gosling decided to use C and C++ languages to develop the project but the problem what they have faced is C and C++ languages are system dependent. Then James Gosling decided why don't we create our own programming language which is system independent.

In 1991, they have developed a programming language called an OAK. OAK means coffee seed name and it is a national tree for many countries like Germany, France, USA and etc.

In 1995, they have renamed OAK to Java. Java is island of an Indonasia where first coffee of seed was produced. During the development of project they were consuming lot of coffee's. Hence a symbol of java is a cup of coffee with saucer.

Interview Questions

=====

Q) Who is the creator of java?

James Gosling

Q) In which year java was developed?

In 1995

Q) Java originally known as _____ ?

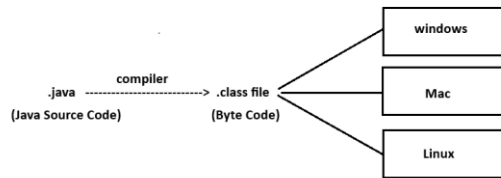
OAK

Q) Is java platform dependent or independepent?

Java is platform independent at byte code level.

Hence java is also known as WORA (Write Once Run Anywhere).

Diagram: class5.1



Q) Is JVM platform dependent or independent?

JVM is platform dependent.

Q) What is the difference between JDK, JRE and JVM ?

JDK

JDK stands for Java Development Kit.

JDK is a installable software which consist Java Runtime Environment (JRE), Java Virtual Machine (JVM), compiler (javac), interpreter (java), an archiever (.jar), document generator (javadoc) and other tools needed for java application development.

JRE

JRE stands for Java Runtime Environment.

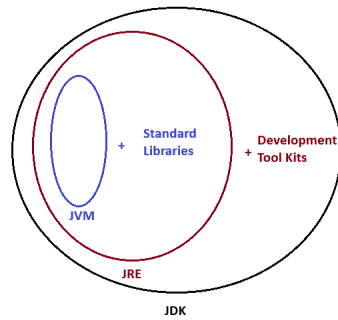
JRE is a part of a JDK which provides very good environment to run java applications only.

JVM

JVM stands for Java Virtual Machine.

JVM is a part of a JRE which is used to execute our program line by line procedure and converts byte code to machine code.

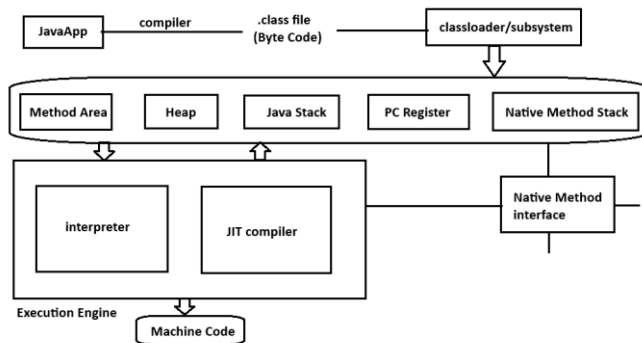
Diagram: class5.2



Internal Architecture of JVM

=====

Diagram : class6.1



Our java application contains java code instructions. Once if we compiled java code instructions convert to byte code instructions in .class file.

JVM will invoke one module called classloader or subsystem to load all the byte code instructions from .class file. The work of classloader is to check these byte code instructions are proper or not. If they are not proper then it will refuse the execution. If they are proper then it will allocates the memory.

We have five types of memories in java.

1) Method Area

It contains code of a class, code of a variable and code of a method.

2) Heap

Our object creations will store in heap area.

3) Java Stack

Java methods store in method area but to execute those methods we required some memory and that memory is allocated in java stack.

4) PC Register

It is a program counter register which is used to track the address of an instructions.

5) Native Method Stack

Java methods execute in method area.

Similarly native methods execute in native method stack.

But to execute native methods we required a program called Native method interface.

Execution Engine

Execution engine contains interpreter and JIT compiler.

Whenever JVM loads byte code instructions from .class file. It simlutenously uses interpreter and JIT compiler.

Interpreter is used to execute our program line by line procedure.

JIT compiler is used to increase the execution speed of our program.

Finally JVM converts byte code to machine code.

Interview Questions

=====

Q) How many memories are there in java?

We have five memories in java.

1) Method Area

2) Heap

3) Java Stack

4) PC Register

5) Native Method Stack

Q) What is JIT Compiler?

JIT stands for Just In Time.

It is a part of a JVM which is used to increase the execution speed of our program.

Q) What is Native Method in Java?

A method which is developed by using some other language is called native method.

But we can't execute native methods directly. We required a program called Native method interface and all native methods store in native method stack memory.

Identifier

=====

A name given to a program elements is called identifier.

Program elements are class name, interface name, variable name, method name, package name and constant name.

ex:

```
class Test
{
    public static void main(String[] args)
    {
        int x = 10;
        System.out.println(x);
    }
}
```

Here Test,main,String,args, x, System and println are identifiers

Rules to declare an identifiers

Rule1:

Identifier will accept following characters.

ex:

A-Z

a-z

0-9

_

\$

Rule2:

If we take other characters then we will get compile time error.

ex:

```
int emp_id; // valid
int emp$alary; // valid
int emp#no; // invalid
```

Rule3:

Every identifier is a case sensitive.

ex:


```
int n;
int N;
```

Rule4:

Identifier must and should starts with alphabet, underscore or dollar symbol but not with digit.

ex:

```
int a1234; //valid
int _abcd; //valid
int $; //valid
int labcd; // invalid
```

Rule5:

We can't take reserved words as an identifier name.

ex:

```
int if; //invalid
int else; //invalid
int for; //invalid
```

Rule6:

There is no length limit for an identifier but it is not recommended to take more than 15 characters.

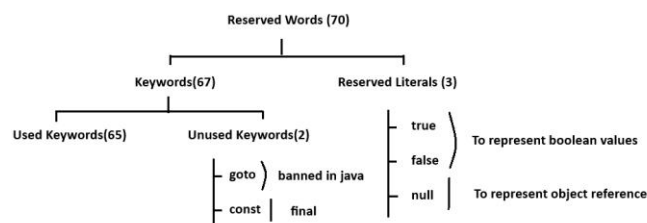
Reserved Words

=====

There some identifiers which are reserved to associate some functionality or meaning such type of identifiers are called reserved words.

In java we have 70 reserved words and it is classified into two types.

Diagram: class7.1



Used keywords with respect to Access modifiers

public
private
protected

Used keywords with respect to classes and interfaces

class
interface
enum
extends
implements
abstract
final
static
strictfp
sealed
permits

Used keywords with respect to exception handling

try
catch
finally
throw
throws
assert

Used keywords with respect to variables and methods

void
var
new
this
super
volatile
transient
synchronized

Used keywords with respect to datatypes.

int
long
short
byte
float

double
char
boolean

Used keywords with respect to package

package
import
module (Java 9+)

Used keywords with respect to object and reference handling

instanceof
typeof (reserved but unused in Java)
yield (Java 14+ switch expression)
record (Java 16+)
native

Used keywords with respect to misc reserved words

requires (Java 9 modules)
exports (Java 9 modules)
opens (Java 9 modules)
uses (Java 9 modules)
provides (Java 9 modules)
with (Java 9 modules)
to (Java 9 modules)
non-sealed (Java 17+)

Datatypes

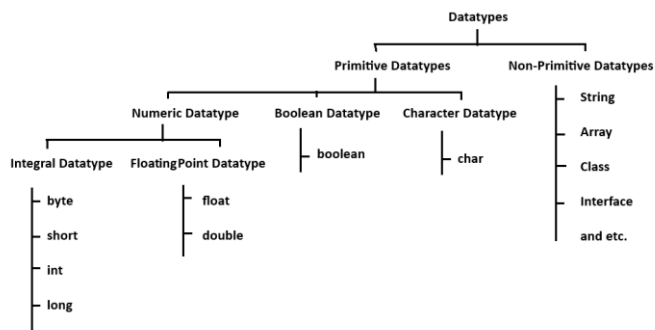
=====

Datatype describes what type of value we want to store inside a variable.

Datatype also tells how much memory has to be created for a variable.

In java, datatypes are divided into two types.

Diagram: class8.1



byte

It is a smallest datatype in java.

Size : 1 byte (8 bits)

Range : -128 to 127 (-2^7 to 2^7-1)

ex:

- 1) `byte b = 10;`
`System.out.println(b); // 10`
- 2) `byte b = 130;`
`System.out.println(b); // C.T.E`
- 3) `byte b = 10.5;`
`System.out.println(b); // C.T.E`

short

It is rarely used datatype in java.

Size : 2 bytes (16 bits)

Range : -32768 to 32767 (-2^{15} to $2^{15}-1$)

ex:

- 1) `byte b=10;`
`short s = b;`
`System.out.println(s); // 10`
- 2) `short s="Hi";`
`System.out.println(s); // C.T.E`

```
3) short s = true;
   System.out.println(s); // C.T.E
```

int

It is mostly used datatype in java.

Size : 4 bytes (32 bits)

Range : -2147483648 to 2147483647 (-2^{31} to $2^{31}-1$)

ex:

```
1) int i = "true";
   System.out.println(i); // C.T.E

2) int i = 10.56;
   System.out.println(i); // C.T.E

3) int i = 'a';
   System.out.println(i); // 97
```

Note:

In java, for every character we have universal unicode value.

ex:

```
a -- 97
A -- 65
```

long

If int datatype is not enough to hold large value then we need to use long datatype.

Size : 8 bytes (64 bits)

Range : (-2^{63} to $2^{63}-1$)

ex:

```
1) long l = 'A';
   System.out.println(l); // 65

2) long l = true;
   System.out.println(l); // C.T.E

3) long l = 10.56;
   System.out.println(l); // C.T.E

4) long l = "Hi";
   System.out.println(l); // C.T.E
```

float	double
-----	-----
If we need 4 to 6 decimal point of accuracy decimal point of then we need to use float. double.	If we need 14 to 16 accuracy then we need to use
Size : 4 bytes (32 bits)	Size : 8 bytes (64 bits)
Range : -3.4e38 to 3.4e38	Range : -1.7e308 to 1.7e308
To declare a float value we need to suffix we need to with 'f' or 'F'. ex:	To declare a double value suffix with 'd' or 'D'. ex:
10.56f;	10.56d;
ex:	
1) float f = 10; System.out.println(f); // 10.0 2) float f = 'a'; System.out.println(f); // 97.0 3) float f = "A"; System.out.println(f); // C.T.E 4) float f = true; System.out.println(f); // C.T.E 5) float f = 10.56f; System.out.println(f); // 10.56	
ex:	
1) double d = 10; System.out.println(d); // 10.0 2) double d = 'a'; System.out.println(d); // 97.0 3) double d = "A"; System.out.println(d); // C.T.E 4) double d = true; System.out.println(d); // C.T.E 5) double d = 10.56d; System.out.println(d); // 10.56	

boolean

It is used to represent boolean values either true or false.

Size : (Not Applicable)

Range : (Not Applicable)

ex:

- 1) boolean b = "true";
System.out.println(b); // C.T.E
- 2) boolean b = TRUE;
System.out.println(b); // C.T.E
- 3) boolean b = true;
System.out.println(b); // true

char

It is a single character which is enclosed in a single quotation.

Size : 2 bytes (16 bits)

Range : 0 to 65535

ex:

- 1) char ch = 'a';
System.out.println(ch); // a
- 2) char ch = 65;
System.out.println(ch); // A
- 3) char ch = 'ab';
System.out.println(ch); // C.T.E

Diagram: class8.2

Datatype	Size	Range	Wrapper class	Default Value
byte	1 byte	-128 to 127	Byte	0
short	2 bytes	-32768 to 32767	Short	0
int	4 bytes	-2147483648 to 2147483647	Integer	0
long	8 bytes	-2 ⁶³ to 2 ⁶³ -1	Long	0
float	4 bytes	-3.4e38 to 3.4e38	Float	0.0f
double	8 bytes	-1.7e308 to 1.7e308	Double	0.0d
boolean	-	-	Boolean	false
char	2 bytes	0 to 65535	Character	space(0)

Q) Write a java program to find out size of int datatype?

```
class Test
{
    public static void main(String[] args)
    {
        System.out.println(Integer.BYTES); // 4
        System.out.println(Integer.SIZE); // 32
    }
}
```

Q) Write a java program to display range of int datatype?

```
class Test
{
    public static void main(String[] args)
    {
        System.out.println(Integer.MIN_VALUE);
        System.out.println(Integer.MAX_VALUE);
    }
}
```

Types of variables

=====

A name given to a memory location is called variable.

Purpose of variable is used to store the data.

In java, variables are divided into two types.

1) Primitive variables

2) Reference variables

1) Primitive variables

It is used to represent primitive values.

2) Reference variables

It is used to represent object reference.

ex:

```
Student s = new Student();
      |
reference variable
```

Based on the position and execution above variables are divided into three types.

1) Instance variables / Non-static variables

2) Static variables / Global variables

3) Local variables / Temporary variables

1) Instance variables

A value of a variable varied(changes) from object to object is called instance variable.

Instance variables allocates memory at the time of object creation and they will destroy at the time of object destruction. Hence scope of instance variables is same as scope of an object.

Instance variables store in heap area as a part of an object.

Instance variable must and should declare immediately after the class but not inside methods, blocks and constructors.

Instance variables we can access directly from instance area but we can't access directly from static area.

To access instance variable from static area we need to create object reference.

ex:1

```
class Test
{
    //instance variable
    int i = 10;

    public static void main(String[] args)
```

```

        {
            System.out.println(i); // C.T.E
        }
    }

ex:2
----
class Test
{
    //instance variable
    int i = 10;

    public static void main(String[] args)
    {
        Test t = new Test();
        System.out.println(t.i); // 10
    }
}

```

Note:

If we won't initialize any value to instance variable then JVM will initialized default values.

```

ex:3
----
class Test
{
    //instance variable
    boolean b;

    public static void main(String[] args)
    {
        Test t = new Test();
        System.out.println(t.b); // false
    }
}

```

In java , for every object a seperate copy of instance variable will be created.

```

ex:4
----
class Test
{
    //instance variable
    int i=10;

    public static void main(String[] args)
    {
        Test t1 = new Test();
    }
}

```

```

        Test t2 = new Test();
        System.out.println(t1.i); // 10
        System.out.println(t2.i); // 10

        t1.i=100;
        System.out.println(t1.i); // 100
        System.out.println(t2.i); // 10
    }
}

ex:5
----
class Test
{
    public static void main(String[] args)
    {
        Test t = new Test();
        t.methodOne();
    }
    //non-static method
    public void methodOne()
    {
        System.out.println("instance-method");
    }
}

```

2) Static variables

A value of a variable which is not varied from object to object is called static variable.

Static variables allocate memory at the time of classloading and they will destroy at the time of class unloading. Hence scope of static variables is same as scope of .class file.

Static variables store in method area.

Static variables must and should declare immediately after the class using static keyword but not inside methods, blocks and constructors.

Static variables access directly from instance area as well as from static area.

Static variables we can access by using object reference and class name.

```

ex:1
---
class Test
{
    //static variable
    static int i = 10;
}

```

```

    public static void main(String[] args)
    {
        System.out.println(i); // 10

        Test t = new Test();
        System.out.println(t.i); // 10

        System.out.println(Test.i); // 10
    }
}

```

Note:

If we won't initialize any value to static variable then JVM will initialized default values.

ex:2

```

class Test
{
    //static variable
    static String s;

    public static void main(String[] args)
    {
        System.out.println(s); // null

        Test t = new Test();
        System.out.println(t.s); // null

        System.out.println(Test.s); // null
    }
}

```

In java, only one copy of static variable will be created and shared to multiple objects.

ex:3

```

class Test
{
    //static variable
    static int i = 10;

    public static void main(String[] args)
    {
        Test t1 = new Test();
        Test t2 = new Test();
        System.out.println(t1.i); // 10
        System.out.println(t2.i); // 10
    }
}

```

```

        t1.i=100;
        System.out.println(t1.i); // 100
        System.out.println(t2.i); // 100
    }
}

ex:4
----
class Test
{
    public static void main(String[] args)
    {
        methodOne();

        Test t = new Test();
        t.methodOne();

        Test.methodOne();
    }
    //static method
    public static void methodOne()
    {
        System.out.println("static-method");
    }
}

```

3) Local variables

To meet temporary requirements a programmer will declare some variables inside methods, blocks and constructors such type of variables are called local variables.

Local variables allocates memory at the time of execution block and they will destroy when execution block is executed. Hence scope of local variables is same as scope of execution block where it is declared.

Local variables will store in java stack.

```

ex:
--
class Test
{
    public static void main(String[] args)
    {
        //local variables
        int i = 10;
        System.out.println(i);
    }
}

```

Note:

If we won't initialize any value to local variable then JVM will not initialize any default values.

ex:

```
---  
class Test  
{  
    public static void main(String[] args)  
    {  
        //local variables  
        int i;  
        System.out.println(i); // C.T.E  
    }  
}
```

A local variable will accept only one modifier i.e final.

ex:

```
---  
class Test  
{  
    public static void main(String[] args)  
    {  
        //local variables  
        final int i=10;  
        System.out.println(i); // 10  
    }  
}
```

Var-Arg Method

=====

Untill 1.4 version , it is not possible to declare a method with variable number of arguments. But from 1.5 version onwards it is possible to declare a method with variable number of arguments.

We can declare var-arg method as follow.

ex:

```
        var-arg parameter  
        |  
        -----  
void methodOne(int... i)  
        |  
        ellipse
```

Here var-arg parameter is a replacement of single dimensional array.

ex:

```
... -----> []
```

We can invoke var-arg method with any number of integer values including zero.

ex:1

```
class Test
{
    public static void main(String[] args)
    {
        methodOne();
    }
    public static void methodOne()
    {
        System.out.println("0-arg method");
    }
}
```

ex:2

```
class Test
{
    public static void main(String[] args)
    {
        methodOne(10);
    }
    public static void methodOne(int i)
    {
        System.out.println("one-arg method");
    }
}
```

ex:3

```
class Test
{
    public static void main(String[] args)
    {
        methodOne(10,20);
    }
    public static void methodOne(int i,int j)
    {
        System.out.println("two-arg method");
    }
}
```

ex:4

```
class Test
{
    public static void main(String[] args)
    {
        methodOne();
        methodOne(10);
        methodOne(10,20);
        methodOne(10,20,30);
    }
}
```

```

    }
    public static void methodOne(int... i)
    {
        System.out.println("var-arg method");
    }
}

```

In real life we can use var-arg method when number of inputs are not fixed to a method.

ex:

```

class Restaurant
{
    public static void main(String[] args)
    {
        //bookingOrder("Biryani");
        bookingOrder("Biryani","Mandi");
    }
    public static void bookingOrder(String... orders)
    {
        //System.out.println(orders[0]);
        System.out.println(orders[0]+" "+orders[1]);
    }
}

```

case1:

We can mix var-arg parameter with general parameter.

ex:

```
methodOne(int x,int... y)
```

case2:

If we mix var-arg parameter with general parameter then var-arg parameter

must be last parameter.

ex:

```
methodOne(int x,int... y)
methodOne(int... x,int y) //invalid

```

case3:

A var-arg method can have only one var-arg parameter.

ex:

```
methodOne(int... x)
methodOne(int... x,int... y) //invalid

```

Java Source File Structure

=====

case1:

A java program can have multiple classes.

case2:

If a java program contains multiple classes then we need to check which

class contains main method and that class treated as main class.

ex:

Test.java

```
class Account
```

```
{
```

```
    double balance = 10000d;
```

```
}
```

```
class Test
```

```
{
```

```
    public static void main(String[] args)
```

```
    {
```

```
        Account acc = new Account();
```

```
        System.out.println(acc.balance);
```

```
    }
```

```
}
```

If we compile above program we will get two .class files

i.e

Account.class and Test.class.

case3:

If a java program contains multiple classes with main method then we need to

declare any one class as public and that public class treated as main class.

ex:

A.java

```
public class A
```

```
{
```

```
    public static void main(String[] args)
```

```
    {
```

```
        System.out.println("A-class");
```

```
    }
```

```
}
```

```
class B
```

```
{
```

```
    public static void main(String[] args)
```

```
    {
```

```
        System.out.println("B-class");
```

```
    }
```

```
}
```

IF we compile above program we will get two .class files

i.e

A.class and B.class.

ex:

```
javac A.java
```

We can execute above program as follow.

ex:

```
java A
java B
```

Main Method

=====

Our program contains main method or not. Either it is properly declared or not. It is not a responsibility of a compiler to check. it is a liability of a JVM to check for main method always at runtime.

If JVM won't find main method then it will throw one runtime error called main method not found.

JVM always looks for main method with following signature.

Signature:

```
public static void main(String[] args)
```

Before Java 21, If we perform any changes in above signature then JVM will throw one runtime error called main method not found.

We can perform following changes in main method.

- 1) In traditional approach, order of modifiers is not important incase of public static we can declare static public also.

ex:

```
static public void main(String[] args)
```

- 2) In traditional approach, We can replace String[] in following acceptable formats.

ex:

```
public static void main(String[] args)
public static void main(String []args)
public static void main(String args[])
```

- 3) In traditional approach, we can replace args with any java valid identifier.

ex:

```
public static void main(String[] ihub)
```

- 4) In traditional approach, we can change String[] with var-arg parameter.

ex:

```
public static void main(String... args)
```

5) In traditional approach, main method will accept following modifiers.

```
ex:
    synchronized
    strictfp
    final
```

6) Since Java 21, Main method does not need to be a public.

```
ex:
    static void main(String[] args)
```

7) Since Java 21, It allows instance main method.

```
ex:
    void main(String[] args)
```

8) Since Java 21, String[] args is optional.

```
ex:
    void main()
```

Q) Explain main method in java?

public

JVM wants to call main method from anywhere.

static

JVM wants to call main method without using object reference.

void

Main method does not return anything to JVM.

main

It is a identifier given to a main method.

String[] args

It is a command line argument

Command Line Arguments

=====

Arguments which are passing through command prompt such type of arguments are called command line arguments.

In command line arguments we need to pass our input values at runtime command.

ex:

```
javac    Test.java

java     Test  101  Alan  M  1000.0
          |      |      |      |_____args[3]
          |      |      |_____args[2]
          |      |_____args[1]
          |_____args[0]
```

ex:

```
---
class Test
{
    void main(String[] args)
    {
        System.out.println(args[0]);
        System.out.println(args[1]);
        System.out.println(args[2]);
        System.out.println(args[3]);
    }
}
```

In reallife we can use command line arguments when we want to calculate salary whenever we run the program.

ex:

```
---
class Test
{
    void main(String[] args)
    {
        int salary = Integer.parseInt(args[0]);
        int hra = salary*10/100;
        int da  = salary*20/100;
        int totalSalary = salary + hra + da;
        System.out.println(totalSalary);
    }
}
```

```
javac Test.java
java Test 10000
System.out.println()
=====
```

It is a output statement in java.

Whenever we want to display any userdefined statements or data then we need to use output stmt in java.

syntax:

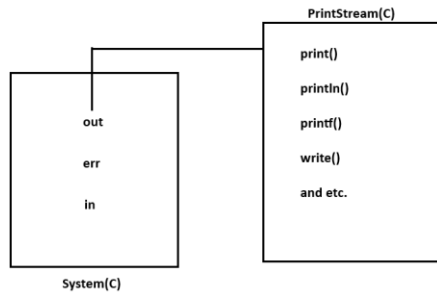
```
-----
    static variable
    |
```

```

        System.out.println()
        |                   |
predefined                predefined method
final class

```

Diagram: class13.1



ex:1

```

class Test
{
    public static void main(String[] args)
    {
        System.out.print("Quality");
        System.out.print("Thought");
    }
}

```

ex:2

```

class Test
{
    public static void main(String[] args)
    {
        System.out.println("Quality");
        System.out.println("Thought");
    }
}

```

ex:3

```

class Test
{
    public static void main(String[] args)
    {
        double d = 24.6789432891d;
        System.out.printf("%.2f",d);
    }
}

```

```
    }  
}
```

Various ways to display the data in java

1) Custom Message

```
ex:  
    System.out.println("Hello World");
```

2) int i=10;

```
ex:  
    System.out.println(i);  
    System.out.println("The value is =" + i);  
    System.out.println(i + " is the value");
```

3) int i=10, j=20;

```
ex:  
    System.out.println(i + " " + j);  
    System.out.println(i + " and " + j);
```

4) int i=1, j=2, k=3;

```
ex:  
    System.out.println(i + " " + j + " " + k);
```

Q) What is the difference between System.out.println() and System.err.println()?

System.out.println()

It is used for standard output.

It is used to display the output only on console.

ex:

```
class Test
```

```
{  
    public static void main(String[] args)  
    {  
        System.out.println("Standard Output");  
    }  
}
```

System.err.println()

It is used for error output.

It is used to display the output on console as well it will redirect to physical file.

ex:

```
class Test
```

```
{  
    public static void main(String[] args)
```

```

        {
            System.err.println("Error Output");
        }
    }
}
javac Test.java

java Test 2>abc.txt

```

Fully Qualified Name

=====

In fully qualified name, we will declare a class or interface along with package name.

ex:

```

    java.io.File (C)
    java.util.Iterator (I)

```

Fully qualified name improves readability of our code.

ex:

--

```

class Test
{
    public static void main(String[] args)
    {
        java.util.Date d = new java.util.Date();
        System.out.println(d);
    }
}

```

In real life we will use fully qualified name when our class or interface present in two different packages and we need both classes or interface in a same program.

ex:

```

    java.util.Date
    java.sql.Date

```

Import statements

=====

Whenever we use import statements we should not use fully qualified name.

Using short name also we can achieve.

The import keyword in Java is used to bring other classes or interfaces into visibility within the current class.

In java, we have three types of import statements.

1) Explicit class import

2) Implicit class import

3) Static import

1) Explicit class import

This type of import statement is highly recommended to use it improves readability of our code.

ex:

```
import java.time.LocalDate;
import java.time.LocalTime;
class Test
{
    public static void main(String[] args)
    {
        LocalDate date = LocalDate.now();
        System.out.println(date);

        LocalTime time = LocalTime.now();
        System.out.println(time);
    }
}
```

2) Implicit class import

This type of import statement is not recommended to use because it will reduce readability of our code.

ex:

```
import java.time.*;
class Test
{
    public static void main(String[] args)
    {
        LocalDate date = LocalDate.now();
        System.out.println(date);

        LocalTime time = LocalTime.now();
        System.out.println(time);
    }
}
```

3) Static import

Using static import we can call static members directly.

Often use of static import makes our program complex and unreadable.


```

ex:
---
import static java.lang.System.*;
class Test
{
    public static void main(String[] args)
    {
        out.println("stmt1");
        out.println("stmt2");
        out.println("stmt3");
    }
}

```

```

ex:
---
import static java.lang.System.*;
class Test
{
    public static void main(String[] args)
    {
        out.println("stmt1");
        exit(0);
        out.println("stmt3");
    }
}

```

EditPlus Editor

=====

Download link : <https://www.editplus.com/download.html>

Java Basic Programs

=====

Q) Write a java program to perform sum of two numbers?

```

import java.util.Scanner;
class Example1
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);

        System.out.println("Enter the first number :");
        int a = sc.nextInt();

        System.out.println("Enter the second number :");
        int b = sc.nextInt();

        int c = a + b;
    }
}

```

```

        System.out.println("Sum of two numbers is =" + c);
    }
}
Q) Write a java program to find out square of a given number?

input:
    5
output:
    25

import java.util.Scanner;
class Example2
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);

        System.out.println("Enter the number :");
        int n = sc.nextInt(); // 5

        //logic
        int square = n * n;

        System.out.println("Square of a given number is =" + square);
    }
}

```

Q) Write a java program to find out cube of a given number?

```

input:
    5
output:
    125

import java.util.Scanner;
class Example3
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);

        System.out.println("Enter the number :");
        int n = sc.nextInt(); // 5

        //logic
        int cube = (int) Math.pow(n, 3);

        System.out.println("Cube of a given number is =" + cube);
    }
}

```

Q) Write a java program to find out area of a circle?

input:

r = 5

output:

78.5

```
import java.util.Scanner;
class Example4
{
    public static void main(String[] args)
    {
        Scanner sc =new Scanner(System.in);

        System.out.println("Enter the radius :");
        int r = sc.nextInt(); // 5

        //logic
        double area = 3.14d * r * r;

        System.out.println("Area of a circle is =" +area);
    }
}
```

Q) Write a java program to find out perimeter of a circle?

input:

r = 5

output:

31.42

```
import java.util.Scanner;
class Example5
{
    public static void main(String[] args)
    {
        Scanner sc =new Scanner(System.in);

        System.out.println("Enter the radius :");
        int r = sc.nextInt(); // 5

        //logic
        double perimeter = 2 * Math.PI * r;

        System.out.printf("Perimeter of a circle is =
%.2f",perimeter);
    }
}
```

Q) Write a java program to perform swapping of two numbers?

input:
10 20

output:
Before swapping a=10 and b=20
After swapping a=20 and b=10

```
import java.util.Scanner;
class Example6
{
    public static void main(String[] args)
    {
        Scanner sc =new Scanner(System.in);

        System.out.println("Enter the two numbers :");
        int a = sc.nextInt(); // 10
        int b = sc.nextInt(); // 20

        System.out.println("Before swapping a="+a+" and b="+b);

        //logic
        int temp = a;
        a = b;
        b = temp;

        System.out.println("After swapping a="+a+" and b="+b);
    }
}
```

Q) Write a java program to perform swapping of two numbers without using third variable?

input:
10 20

output:
Before swapping a=10 and b=20
After swapping a=20 and b=10

```
import java.util.Scanner;
class Example7
{
    public static void main(String[] args)
    {
        Scanner sc =new Scanner(System.in);

        System.out.println("Enter the two numbers :");
        int a = sc.nextInt(); // 10
        int b = sc.nextInt(); // 20
```

```

        System.out.println("Before swapping a="+a+" and b="+b);

        //logic
        a = a + b;
        b = a - b;
        a = a - b;

        System.out.println("After swapping a="+a+" and b="+b);
    }
}

```

Q) Write a java program to display Hello world without using print stmt?

```

import java.io.IOException;
class Example8
{
    public static void main(String[] args)throws IOException
    {
        String str= "Hello World";
        System.out.write(str.getBytes());
    }
}

```

Q)
There is one running race between different participants. So in that race two partners need to finish the race. There is four rounds of race in that each lane length is 100m. So in that one participant will cover 30% of the lane length and another partner will cover remaining length of lane. So print each participant covers distance in that race.

ex:

```

class Example9
{
    public static void main(String[] args)
    {
        int lane = 100;
        int rounds = 4;
        int firstParticipantPercent = 30;

        int firstParticipant = lane * rounds * firstParticipantPercent /
100;
        int secondParticipant = lane * rounds - firstParticipant;

        System.out.println("First Participant covered "+firstParticipant+"
distance");
        System.out.println("Second Participant covered "+secondParticipant+"
distance");
    }
}

```

Alice is planning to organize a contest with 4 players in each team.

There are two types of players named Experienced and Freshers.
To make the contest unbiased Alice wants to have a team in such a way that each team must contain at least 1 Experienced and 1 Fresher.
You are given N the number of Experienced and M the number of Freshers.
Your task is to determine the maximum number of team formations possible.
Example:

Input:
5 5
Output:
2

Input:
10 1
Output:
1

```
class Example10
{
    public static void main(String[] args)
    {
        int N=10,M=1;

        int teams = Math.min((N+M)/4, Math.min(N,M));

        System.out.println(teams);
    }
}
```

Q) Write a java program to accept date of birth of a person and find out his age?

```
import java.time.LocalDate;
import java.time.Period;
class Example11
{
    public static void main(String[] args)
    {
        LocalDate dob = LocalDate.of(2001,06,21);

        LocalDate curr_date = LocalDate.now(); // 2026-02-05
```

```

        Period p = Period.between(dob,curr_date);

        int age = p.getYears();

        System.out.println("Person age is "+age);
    }
}

```

Q) Write a java program to convert time to seconds?

input:

```

Hours = 10
Minutes = 30
Seconds = 23

```

output:

```

37823

```

```

class Example12
{
    public static void main(String[] args)
    {
        int hours = 10;
        int minutes = 30;
        int seconds = 23;

        int totalSeconds = (hours*3600)+(minutes*60)+seconds;

        System.out.println(totalSeconds);
    }
}

```

Q) Write a java program to convert seconds to time?

input:

```

37823

```

output:

```

10:30:23

```

```

class Example13
{
    public static void main(String[] args)
    {
        int totalSeconds = 37823;

        // (hours*3600)+(minutes*60)+seconds;

        int hours = totalSeconds/3600;
    }
}

```

```

        int minutes = (totalSeconds%3600)/60;

        int seconds = totalSeconds%60;

        System.out.println(hours+":"+minutes+":"+seconds);
    }
}

```

Q) A car travels 120 km in 2 hours.
Write a Java program to calculate the average speed of the car.

formula :

$$\text{average speed} = \text{distance} / \text{time};$$

ex:

 class Example14
 {
 public static void main(String[] args)
 {
 int distance = 120;
 int time = 2;

 int averageSpeed = distance/time;

 System.out.println(averageSpeed);
 }
 }

Q) Sunita wakes up at 6:30 AM and reaches school at 7:15 AM.
Write a Java program to calculate total travel time in minutes.

ex:

 class Example15
 {
 public static void main(String[] args)
 {
 int startHour = 6;
 int startMinute = 30;

 int endHour = 7;
 int endMinute = 15;

 int startTimeInMinutes = (startHour * 60) + startMinute;
 int endTimeInMinutes = (endHour * 60) + endMinute;

 int travelTime = endTimeInMinutes - startTimeInMinutes;

 System.out.println(travelTime);
 }
 }

Q)

A company works 10 hours per day.

Employees spend 30% of the time in meetings and the rest working on projects.

Write a Java program to calculate meeting time and project work time.

```
class Example16
{
    public static void main(String[] args)
    {
        int totalHours = 10;

        int meetingTime = totalHours*30/100;

        int projectWorkTime = totalHours - meetingTime;

        System.out.println("Total Hours :"+totalHours);
        System.out.println("Meeting Time :"+meetingTime);
        System.out.println("Project Work Time :"+projectWorkTime);

    }
}
```

Typecasting in Java

=====

Process of converting from one datatype to another datatype is called typecasting.

In java, There are two ways to perform typecasting.

1) Implicit typecasting

2) Explicit typecasting

1) Implicit typecasting

If we want to store small value into a bigger variable then we need to use implicit typecasting.

A compiler is responsible to perform implicit typecasting.

There is no possibility to loss the information.

It is also known as widening or upcasting.

We can perform implicit typecasting as follow.

ex:

```

byte --> short
      -->
      int --> long --> float --> double
      -->
      char

```

ex:

```

class Test
{
    public static void main(String[] args)
    {
        byte b = 10;

        int i = b;

        System.out.println(i); //10
    }
}

```

ex:

--

```

class Test
{
    public static void main(String[] args)
    {
        char ch = 'a';

        long l = ch;

        System.out.println(l); //97
    }
}

```

ex:

--

```

class Test
{
    public static void main(String[] args)
    {
        int i = 10;

        double d = i;

        System.out.println(d); //10.0
    }
}

```

2) Explicit typecasting

If we want to store bigger value into a smaller variable then we need to use explicit typecasting.

A programmer is responsible to perform explicit typecasting.

There is a possibility to loss the information.

It is also known as narrowing or downcasting.

We can perform explicit typecasting as follow.

ex:

```
byte <-- short
      <--
      int <-- long <-- float <-- double
      <--
      char
```

ex:

```
---
class Test
{
    public static void main(String[] args)
    {
        double d = 10.56d;

        int i = (int)d;

        System.out.println(i); // 10
    }
}
```

ex:

```
---
class Test
{
    public static void main(String[] args)
    {
        int i = 65;

        char ch = (char)i;

        System.out.println(ch); // A
    }
}
```

ex:

```
---
class Test
{
    public static void main(String[] args)
    {
        int i = 130;

        byte b = (byte)i;
```

```

        System.out.println(b); // -126
    }
}

```

Types of blocks

=====

A block is a set of statements which is enclosed in a curly braces i.e {}.

We have three types of blocks in Java.

- 1) Instance block
- 2) Static block
- 3) Local block

1) Instance block

Instance block is used to initialize instance variables.

Instance block will execute when instance of a class is created.

Instance block must and should declare immediately after the class but not inside methods and constructors.

We can declare instance block as follow.

syntax:

```

//instance block
{
    -
    - //set of statements
    -
}

```

ex:

```

class Test
{
    //instance block
    {
        System.out.println("instance-block");
    }

    public static void main(String[] args)
    {
        System.out.println("main-method");
    }
}

```

```

    }
}
o/p:
    main-method

ex:
---
class Test
{
    //instance block
    {
        System.out.println("instance-block");
    }

    public static void main(String[] args)
    {
        System.out.println("main-method");
        Test t = new Test();
    }
}

```

```

o/p:
    main-method
    instance-block

```

```

ex:
---
class Test
{
    //instance block
    {
        System.out.println("instance-block");
    }

    public static void main(String[] args)
    {
        Test t1 = new Test();
        System.out.println("main-method");
        Test t2 = new Test();
    }
}

```

```

o/p:
    instance-block
    main-method
    instance-block

```

```

ex:
---
class Test
{
    //instance variable

```

```

    int i;

    //instance block
    {
        i=100;
    }

    public static void main(String[] args)
    {
        Test t = new Test();
        System.out.println(t.i); // 100
    }
}

```

2) Static block

Static block is used to initialize static variables.

Static block is executed at the time of class loading.

Static block must and should declare immediately after the class using static keyword but not inside methods and constructors.

We can declare static block as follow.

syntax:

```

    //static block
    static
    {
        -
        - //set of statements
        -
    }

```

ex:

```

class Test
{
    //static block
    static
    {
        System.out.println("static-block");
    }

    public static void main(String[] args)
    {
        System.out.println("main-method");
    }
}

```

o/p:

```

static-block
main-method

```

```

ex:
---
class Test
{
    //instance block
    {
        System.out.println("instance-block");
    }

    //static block
    static
    {
        System.out.println("static-block");
    }

    public static void main(String[] args)
    {
        System.out.println("main-method");
        Test t = new Test();
    }
}

```

```

ex:
--
class Test
{
    //static variable
    static int i;

    //static block
    static
    {
        i=200;
    }

    public static void main(String[] args)
    {
        System.out.println(i);    //200
    }
}

```

3) Local block

Local block is used to initialize the local variables.

Local block is execute just like a normal statement.

Local block must and should declare inside methods and constructors.

We can declare local block as follow.

syntax:

```
//local block
{
    -
    - //set of statements
    -
}
```

ex:

```
class Test
{
    public static void main(String[] args)
    {
        System.out.println("stmt1");

        //local block
        {
            System.out.println("stmt2");
        }

        System.out.println("stmt3");
    }
}
```

o/p:

```
stmt1
stmt2
stmt3
```

ex:

```
class Test
{
    public static void main(String[] args)
    {
        //local variable
        int i;

        //local block
        {
            i=300;
        }

        System.out.println(i); // 300
    }
}
```

Operators

=====

Operator is a symbol which is used to perform some operations on operands.

ex:

```
c = a + b;
```

Here a,b and c are operands.

Here = and + are operators.

We have following list of operators in java.

- 1) Assignment operators
- 2) Ternary operators/conditional operators
- 3) Bitwise operators
- 4) Logical operators
- 5) Arithmetic operators
- 6) Relational operators
- 7) Shift operators
- 8) Unary operators

1) Assignment operators

ex:

```
class Test
{
    public static void main(String[] args)
    {
        int i=10;

        i=20;

        i=30;

        System.out.println(i); // 30
    }
}
```

Note:

Reinitialization is possible in java.

ex:

```
class Test
{
    public static void main(String[] args)
    {
```

```

        final int i=10;

        i=20;

        i=30;

        System.out.println(i); // C.T.E
    }
}
o/p:
    C.T.E : We can't modify final variable

```

```

ex:
---
class Test
{
    public static void main(String[] args)
    {
        int i,j;

        i = j = 10;

        System.out.println(i+" "+j); // 10  10
    }
}

```

```

ex:
---
class Test
{
    public static void main(String[] args)
    {
        int i;

        i = 1,2,3,4,5;

        System.out.println(i); // C.T.E
    }
}

```

```

ex:
---
class Test
{
    public static void main(String[] args)
    {
        int i=10;

        i+=5; // i = i + 5;
    }
}

```

```

        System.out.println(i); //15
    }
}

ex:
---
class Test
{
    public static void main(String[] args)
    {
        int i=10;

        i*=5; // i = i * 5;

        System.out.println(i); //50
    }
}

```

```

ex:
---
class Test
{
    public static void main(String[] args)
    {
        int i=10;

        i-=5; // i = i - 5;

        System.out.println(i); //5
    }
}

```

```

ex:
---
class Test
{
    public static void main(String[] args)
    {
        int i=10;

        i/=5; // i = i / 5;

        System.out.println(i); //2
    }
}

```

```

ex:
---
class Test
{
    public static void main(String[] args)
    {

```

```

        int i=10;

        i/=20; // i = i / 20;

        System.out.println(i); //0
    }
}

ex:
---
class Test
{
    public static void main(String[] args)
    {
        int i=10;

        i%=5; // i = i % 5;

        System.out.println(i); //0
    }
}

```

```

ex:
---
class Test
{
    public static void main(String[] args)
    {
        int i=10;

        i%=20; // i = i % 20;

        System.out.println(i); //10
    }
}

```

2) Ternary / Conditional operators

syntax:

(condition)?value1:value2;

```

ex:
---
class Test
{
    public static void main(String[] args)
    {
        String s = (true)?"Hi":"Bye";
        System.out.println(s);
    }
}

```

```

ex:
---
class Test
{
    public static void main(String[] args)
    {
        int i = (false)?1:0;
        System.out.println(i); //0
    }
}

```

```

ex:
---
class Test
{
    public static void main(String[] args)
    {
        boolean b = (5>2)?true:false;
        System.out.println(b); //true
    }
}

```

Q) Write a java program to find out greatest of two numbers?

```

import java.util.Scanner;
class Test
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);

        System.out.println("Enter the two numbers :");
        int a = sc.nextInt(); // 10
        int b = sc.nextInt(); // 20

        int max = (a>b)?a:b;
        System.out.println(max+" is greatest");
    }
}

```

Q) Write a java program to find out greatest of three numbers?

```

import java.util.Scanner;
class Test
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);

        System.out.println("Enter the three numbers :");
    }
}

```

```

        int a = sc.nextInt(); // 10
        int b = sc.nextInt(); // 20
        int c = sc.nextInt(); // 30

        int max = (a>b)?((a>c)?a:c):((b>c)?b:c);

        System.out.println(max+" is greatest");
    }
}

```

3) Bitwise operators

How to convert decimal number to binary number

10 - decimal number
1010 - binary number

```

2|10
---- 0
2|5
---- 1
2|2
---- 0      ^
 1      |
-----
1010

```

ex:

```

class Test
{
    public static void main(String[] args)
    {
        int decimal = 10;

        String binary = Integer.toBinaryString(decimal);

        System.out.println(binary);
    }
}

```

How to convert binary number to decimal number

binary number - 1010
decimal number - 10

1010 <-----

$0 \times 1 + 1 \times 2 + 0 \times 4 + 1 \times 8$

0 + 2 + 0 + 8 = 10

ex:

```
class Test
{
    public static void main(String[] args)
    {
        String binary = "1010";

        int decimal =Integer.parseInt(binary,2);

        System.out.println(decimal);
    }
}
```

Bitwise AND operator (&)

Bitwise AND operator deals with binary numbers.

Truth table

T	T	= T
T	F	= F
F	T	= F
F	F	= F

ex:

```
class Test
{
    public static void main(String[] args)
    {
        int a=10,b=15;
        int c = a & b;
        System.out.println(c);    // 10
    }
}

/*
10 - 1010
15 - 1111
-----
&  - 1010 <----

0*1 + 1*2 + 0*4 + 1*8

0 + 2 + 0 + 8
*/
```

ex:

```

class Test
{
    public static void main(String[] args)
    {
        int a=10,b=5;
        int c = a & b;
        System.out.println(c);    //0
    }
}
/*
    10 - 1010
    5  - 0101
    -----
    &  - 0000
*/

```

Bitwise OR operator (|)

Bitwise OR operator deals with binary numbers.

Truth table

T	T	= T
T	F	= T
F	T	= T
F	F	= F

ex:

```

class Test
{
    public static void main(String[] args)
    {
        int a=10,b=15;
        int c = a | b;
        System.out.println(c);    //15
    }
}
/*
    10 - 1010
    15 - 1111
    -----
    |  - 1111 <----

    1*1 + 1*2 + 1*4 + 1*8

    1 + 2 + 4 + 8
*/

```

Bitwise XOR operator (^)

Bitwise XOR operator deals with binary numbers.

Truth table

T T = F
T F = T
F T = T
F F = F

ex:

```
class Test
{
    public static void main(String[] args)
    {
        int a=10,b=15;
        int c = a ^ b;
        System.out.println(c);  //5
    }
}
/*
    10 - 1010
    15 - 1111
    -----
    ^  - 0101  <-----

    1*1 + 0*2 + 1*4 + 0*8

    1 + 0 + 4 + 0
*/
```

Bitwise NOT operator (~)

syntax:

-(n+1)

ex:

```
class Test
{
    public static void main(String[] args)
    {
        int i = ~(10);
        System.out.println(i);  // -11
    }
}
```

ex:

```

class Test
{
    public static void main(String[] args)
    {
        int i = ~(34);
        System.out.println(i); // -35
    }
}

```

ex:

```

class Test
{
    public static void main(String[] args)
    {
        int i = ~(-12);
        System.out.println(i); // 11
    }
}

```

Q) Write a java program to find out given number is even or odd without using % modules operator ?

```

import java.util.Scanner;
class Test
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter the number :");
        int n = sc.nextInt();

        String str = ((n&1)==0)?"It is even number":"It is odd
number";
        System.out.println(str);
    }
}

```

```

/*
    10 - 1010
    1  - 0001
    -----
        & - 0000
*/

```

4) Logical operators

=====

Logical AND operator (&&)

Logical AND operator deals with boolean values either true or false.

Truth table

T	T	= T
T	F	= F
F	T	= F
F	F	= F

ex:

```
class Test
{
    public static void main(String[] args)
    {
        System.out.println(true && true); // true
        System.out.println(true && false); // false
        System.out.println(false && true); // false
        System.out.println(false && false); // false
    }
}
```

ex:

```
class Test
{
    public static void main(String[] args)
    {
        boolean b = (5>2) && (6<10);
        System.out.println(b); // true
    }
}
```

ex:

```
class Test
{
    public static void main(String[] args)
    {
        boolean b = (5>20) && (6<10);
        System.out.println(b); // false
    }
}
```

Logical OR operator (||)

Logical OR operator deals with boolean values either true or false.

Truth table

T	T	= T
T	F	= T

F T = T
F F = F

ex:

```
class Test
{
    public static void main(String[] args)
    {
        System.out.println(true || true); // true
        System.out.println(true || false); // true
        System.out.println(false || true); // true
        System.out.println(false || false); // false
    }
}
```

ex:

```
class Test
{
    public static void main(String[] args)
    {
        boolean b = (5>20) || (5<10);
        System.out.println(b); // true
    }
}
```

ex:

```
class Test
{
    public static void main(String[] args)
    {
        boolean b = (5>20) || (5<1);
        System.out.println(b); //false
    }
}
```

ex:

```
class Test
{
    public static void main(String[] args)
    {
        boolean b = (5>2) && (6<1) || (10 == 10);
        System.out.println(b); //true
    }
}
```

Logical NOT operator (!)

Logical NOT operator deals with boolean values either true or false.

ex:

```
class Test
{
    public static void main(String[] args)
    {
        boolean b = !(5>2);
        System.out.println(b); // false
    }
}
```

ex:

```
class Test
{
    public static void main(String[] args)
    {
        boolean b = !(5>20);
        System.out.println(b); // true
    }
}
```

5) Arithmetic operators

=====

We have following list of arithmetic operators according to operators preceders

ex:

- * - multiplication
- / - division
- % - modules
- + - addition
- - subtraction

ex:

```
class Test
{
    public static void main(String[] args)
    {
        int i = 5+3*2+6/2+7%2+10/20+2*8+10%30+6-20;
        System.out.println(i);
    }
}
/*
5+3*2+6/2+7%2+10/20+2*8+10%30+6-20

5+6+6/2+7%2+10/20+16+10%30+6-20

5 + 6 + 3 + 7%2 + 0 + 16 + 10%30 + 6 - 20
```

```

    5 + 6 + 3 + 1 + 0 + 16 + 10 + 6 - 20

    47 - 20

    27
*/

6) Relational operators
=====

ex:
---
class Test
{
    public static void main(String[] args)
    {
        System.out.println(10 > 20); // false
        System.out.println(10 < 20); // true
        System.out.println(10 >= 20); // false
        System.out.println(10 <= 10 ); // true
    }
}

ex:
---
class Test
{
    public static void main(String[] args)
    {
        System.out.println(10 == 10); // true
        System.out.println(10 == 20); // false
        System.out.println(10 != 20); // true
        System.out.println(10 != 10 ); // false
    }
}

7) Shift operators
=====

Right Shift operator (>>)
-----
10 >> 1 -> 10/2
10 >> 2 -> 10/4
10 >> 3 -> 10/8

ex:
---
class Test
{
    public static void main(String[] args)
    {
        int i = 10 >> 4;

```

```

        System.out.println(i); // 10/16 = 0
    }
}

ex:
---
class Test
{
    public static void main(String[] args)
    {
        int i = 100 >> 6;
        System.out.println(i); // 100 / 64 =1
    }
}

```

Left Shift operator (<<)

```

-----
10 << 1 -> 10*2
10 << 2 -> 10*4
10 << 3 -> 10*8

```

```

ex:
---
class Test
{
    public static void main(String[] args)
    {
        int i = 10 << 3;
        System.out.println(i); // 10*8 = 80
    }
}

```

```

ex:
---
class Test
{
    public static void main(String[] args)
    {
        int i = 100 << 4;
        System.out.println(i); // 100 * 16 = 1600
    }
}

```

8) Unary operators

=====

Increment/Decrement (++/--)

We have two types of increment operators.

1) Post Increment

```
ex:
    i++;
```

2) Pre Increment

```
ex:
    ++i;
```

We have two types of decrement operators.

1) Post Decrement

```
ex:
    i--;
```

2) Pre Decrement

```
ex:
    --i;
```

POST Increment/Decrement operators

Rule1 : First Take
Rule2 : Then Change

```
ex:
---
class Test
{
    public static void main(String[] args)
    {
        int i=10;

        i++;

        System.out.println(i); // 11
    }
}
```

```
ex:
---
class Test
{
    public static void main(String[] args)
    {
        int i=10;

        System.out.println(i++); // 10
    }
}
```



```

ex:
---
class Test
{
    public static void main(String[] args)
    {
        int i=10;

        int j = i++;

        System.out.println(i+" "+j); // 11 10
    }
}
ex:
---
class Test
{
    public static void main(String[] args)
    {
        int i = 10;

        int j = i++ + i++; // 10 11

        System.out.println(i+" "+j); // 12 21
    }
}

ex:
---
class Test
{
    public static void main(String[] args)
    {
        int i = 10;

        int j = i-- + i--; //10 + 9

        System.out.println(i+" "+j); // 8 19
    }
}

PRE increment/Decrement
-----
Rule1: First Change
Rule2: Then Take

ex:
---
class Test
{
    public static void main(String[] args)

```

```

        {
            int i = 10;

            ++i;

            System.out.println(i); //11
        }
    }

ex:
---
class Test
{
    public static void main(String[] args)
    {
        int i = 10;
        System.out.println(++i); // 11
    }
}

ex:
---
class Test
{
    public static void main(String[] args)
    {
        int i = 10;

        int j = ++i;

        System.out.println(i+" "+j); //11  11
    }
}

ex:
---
class Test
{
    public static void main(String[] args)
    {
        int i = 10;

        int j = ++i + ++i; // 11 + 12

        System.out.println(i+" "+j); // 12  23
    }
}

ex:
---
class Test
{

```

```

    public static void main(String[] args)
    {
        int i = 10;

        int j = --i + --i; // 9 + 8

        System.out.println(i+" "+j); // 8 17
    }
}

ex:
---
class Test
{
    public static void main(String[] args)
    {
        int i = 10;

        int j = ++i + --i - ++i; // 11 + 10 - 11

        System.out.println(i+" "+j); // 11 10
    }
}

ex:
---
class Test
{
    public static void main(String[] args)
    {
        int i = 100;

        100++;

        System.out.println(i); // C.T.E
    }
}

ex:
---
class Test
{
    public static void main(String[] args)
    {
        byte b=127;

        b++;

        System.out.println(b); // -128
    }
}

```

```

ex:
---
class Test
{
    public static void main(String[] args)
    {
        int i=10;

        System.out.println(++(i++)); // C.T.E
    }
}

```

```

ex:
---
class Test
{
    public static void main(String[] args)
    {
        int i=10;

        System.out.println(i++ + ++i); // 10 + 12 = 22
    }
}

```

Control Statements

=====

Control statement enables the programmer to control flow of a program.

Control statement allows us to make decisions, to jump from one section of code to another section and to execute the code repeatedly.

In java, we have three control statements.

- 1) Decision Making Statements
- 2) Iteration Statements
- 3) Jump Statements

1) Decision Making Statements

It is used to declare conditions in our program.

Decision making statement is possible by using following ways.

- i) if stmt
- ii) if else stmt
- iii) if else if ladder
- iv) nested if stmt

v) switch case

i) if stmt

=====

It is used to execute the source code only if our condition is true.

syntax:

```
    if(condition)
    {
        -
        - //set of statements
        -
    }
```

ex:

```
class Test
{
    public static void main(String[] args)
    {
        System.out.println("stmt1");

        if(!(5>20))
        {
            System.out.println("stmt2");
        }

        System.out.println("stmt3");
    }
}
```

o/p:

```
stmt1
stmt2
stmt3
```

ex:

```
class Test
{
    public static void main(String[] args)
    {
        System.out.println("stmt1");

        if((10==10) && (10!=10))
        {
            System.out.println("stmt2");
        }

        System.out.println("stmt3");
    }
}
```

```

}
o/p:
    stmt1
    stmt3

ex:
---
class Test
{
    public static void main(String[] args)
    {
        if(5>20)
            System.out.println("stmt1");
            System.out.println("stmt2");
            System.out.println("stmt3");
        }
    }
}

```

```

o/p:
    stmt2
    stmt3

```

Q) Write a java program to find out greatest of two numbers?

```

import java.util.Scanner;
class Test
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter the two numbers:");
        int a = sc.nextInt(); // 10
        int b = sc.nextInt(); // 20

        if(a>b)
            System.out.println(a+" is greatest");

        if(b>a)
            System.out.println(b+" is greatest");
    }
}

```

Q) Write a java program to find out greatest of three numbers?

```

import java.util.Scanner;
class Test
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter the three numbers:");
    }
}

```

```

        int a = sc.nextInt(); // 10
        int b = sc.nextInt(); // 20
        int c = sc.nextInt(); // 30

        if((a>b) && (a>c))
            System.out.println(a+" is greatest");

        if((b>a) && (b>c))
            System.out.println(b+" is greatest");

        if((c>a) && (c>b))
            System.out.println(c+" is greatest");
    }
}

```

ii) if else stmt

=====

It is used to execute the source code either our condition is true or false.

syntax:

```

    if(condition)
    {
        - //code to be execute if cond is true
    }
    else
    {
        - //code to be execute if cond is false
    }

```

ex:

```

class Test
{
    public static void main(String[] args)
    {
        System.out.println("stmt1");

        if(5>2)
        {
            System.out.println("stmt2");
        }
        else
        {
            System.out.println("stmt3");
        }

        System.out.println("stmt4");
    }
}

```

o/p:

```
stmt1
stmt2
stmt4
```

ex:

```
class Test
{
    public static void main(String[] args)
    {
        System.out.println("stmt1");

        if(5>20)
        {
            System.out.println("stmt2");
        }
        else
        {
            System.out.println("stmt3");
        }

        System.out.println("stmt4");
    }
}
```

o/p:

```
stmt1
stmt3
stmt4
```

Q) Write a java program to check given age is eligible to vote or not?

```
import java.util.Scanner;
class Test
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter the age :");
        int age = sc.nextInt();

        if(age>=18)
            System.out.println("U r eligible to vote");
        else
            System.out.println("U r not eligible to vote");
    }
}
```

Q) Write a java program to find out given number is even or odd ?


```

import java.util.Scanner;
class Test
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter the number :");
        int n = sc.nextInt();

        if(n%2==0)
            System.out.println("It is even number");
        else
            System.out.println("It is odd number");
    }
}

```

Q) Write a java program to find out given number is odd or not ?

```

import java.util.Scanner;
class Test
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter the number :");
        int n = sc.nextInt();

        if(n%2!=0)
            System.out.println("It is odd number");
        else
            System.out.println("It is not odd number");
    }
}

```

Q) Write a java program to find out given number is positive or negative?

```

import java.util.Scanner;
class Test
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter the number :");
        int n = sc.nextInt();
        if(n==0)
        {

```

```

        System.out.println("It is not a positive or negative
number");
        System.exit(0);
    }

    if(n>0)
        System.out.println("It is a positive number");
    else
        System.out.println("It is a negative number");
    }
}

```

Q) One day John went to a big showroom to buy a pair of shoe. But he has a myth if shoe price divisible by 3 and 5 then only he can buy the shoe. Write a java program to find out John can buy the shoe or not.

input:
1500

output:
John can buy the shoe

```

import java.util.Scanner;
class Test
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter the shoe price :");
        int price = sc.nextInt(); // 1500

        if((price%3==0) && (price%5==0))
            System.out.println("John can buy the shoe");
        else
            System.out.println("John can't buy the shoe");
    }
}

```

Q) Write a java program to find out given alphabet is a vowel or not?

```

import java.util.Scanner;
class Test
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter the alphabet :");
        char ch = sc.next().charAt(0);

        if(ch=='a' || ch=='e' || ch=='i' || ch=='o' || ch=='u')

```

```

        System.out.println("It is a vowel");
    else
        System.out.println("It is not a vowel");
}
}

```

Q) Write a java program to find out given year is a leap year or not?

input:

2024

output:

It is a leap year

```

import java.util.Scanner;
class Test
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter the year :");
        int year = sc.nextInt();

        if(year%4==0 && year%100!=0 || year%400==0)
            System.out.println("It is a leap year");
        else
            System.out.println("It is not a leap year");
    }
}

```

iii) if else if ladder

=====

It executes our source code based on multiple conditions.

syntax:

```

if(cond1)
{
    - //code to be execute if cond1 is true
}
else if(cond2)
{
    - //code to be execute if cond2 is true
}
else if(cond3)
{
    - //code to be execute if cond3 is true
}
else
{
    - //code to be execute if all conditions are false
}

```

ex:

```
import java.util.Scanner;
class Test
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter the option :");
        int option = sc.nextInt(); // 108

        if(option==100)
            System.out.println("It is a police number");
        else if(option==103)
            System.out.println("It is a enquiry number");
        else if(option==108)
            System.out.println("It is a emergency number");
        else
            System.out.println("Invalid option");
    }
}
```

Q) Write a java program to find out given alphabet is a uppercase letter, lowercase letter, digit or a special symbol?

ex:

```
import java.util.Scanner;
class Test
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter the Alphabet :");
        char ch = sc.next().charAt(0);

        if(Character.isUpperCase(ch))
            System.out.println("It is a uppercase letter");
        else if(Character.isLowerCase(ch))
            System.out.println("It is a lowercase letter");
        else if(Character.isDigit(ch))
            System.out.println("It is a digit");
        else
            System.out.println("It is a special symbol");
    }
}
```

Q) Write a java program to display given alphabet is a vowel or not?

```

import java.util.Scanner;
class Test
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter the Alphabet :");
        char ch = sc.next().charAt(0);

        ch = Character.toLowerCase(ch);

        if(ch=='a')
            System.out.println("It is a vowel");
        else if(ch=='e')
            System.out.println("It is a vowel");
        else if(ch=='i')
            System.out.println("It is a vowel");
        else if(ch=='o')
            System.out.println("It is a vowel");
        else if(ch=='u')
            System.out.println("It is a vowel");
        else
            System.out.println("It is a not a vowel");
    }
}

```

Q) Write a java program to accept six marks of a student then find out total, average and grade?

- i) if average is greater than equals to 70 then A grade.
- ii) if average is greater than equals to 50 then B grade.
- iii) if average is greater than equals to 35 then C grade.
- iv) if average is less than 35 then failed.

```

import java.util.Scanner;
class Test
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter the six marks of a student :");
        int m1 = sc.nextInt();
        int m2 = sc.nextInt();
        int m3 = sc.nextInt();
        int m4 = sc.nextInt();
        int m5 = sc.nextInt();
        int m6 = sc.nextInt();

        int total = m1+m2+m3+m4+m5+m6;
        double average = (double)total/6;
    }
}

```

```

        System.out.println("Total :"+total);
        System.out.printf("Average : %.2f",average);

        if(average>=70)
            System.out.println("Grade : A grade");
        else if(average>=50)
            System.out.println("Grade : B grade");
        else if(average>=35)
            System.out.println("Grade : C grade");
        else
            System.out.println("Grade : Failed");

    }
}

iv) Nested if stmt
=====
If stmt contains another if stmt is called nested if stmt.

syntax:
-----
    if(condition)
    {
        if(condition)
        {
            - //code to be execute
        }
    }

ex:
---
class Test
{
    public static void main(String[] args)
    {
        System.out.println("stmt1");
        if(5>2)
        {
            System.out.println("stmt2");
            if(true)
            {
                System.out.println("stmt3");
            }
            System.out.println("stmt4");
        }
        System.out.println("stmt5");
    }
}

o/p:
    stmt1
    stmt2
    stmt3

```

```

        stmt4
        stmt5

ex:
---
class Test
{
    public static void main(String[] args)
    {
        System.out.println("stmt1");
        if(5>20)
        {
            System.out.println("stmt2");
            if(true)
            {
                System.out.println("stmt3");
            }
            System.out.println("stmt4");
        }
        System.out.println("stmt5");
    }
}
o/p:
    stmt1
    stmt5

```

```

ex:
---
class Test
{
    public static void main(String[] args)
    {
        System.out.println("stmt1");
        if(5>2)
        {
            System.out.println("stmt2");
            if(false)
            {
                System.out.println("stmt3");
            }
            System.out.println("stmt4");
        }
        System.out.println("stmt5");
    }
}
o/p:
    stmt1
    stmt2
    stmt4
    stmt5

```

Q) Write a java program to find out given number is positive or negative using nested if stmt?

```
import java.util.Scanner;
class Test
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter the number :");
        int n = sc.nextInt(); // 5

        if(n!=0)
        {
            if(n>0)
            {
                System.out.println("It is a positive number");
                System.exit(0);
            }

            System.out.println("It is a negative number");
        }
    }
}
```

Q) Write a java program to find out the cost based on user inputs. A program must prompt users to enter total weight of items (in kilograms) and shipping destination (domestic or international). For domestic orders, a program must charge Rs.500 upto 5kg and Rs.100 per additional kg. For international orders a program must charge Rs.1000 upto 5kg and Rs.200 per additional kg, and Rs.500 surcharge if weight exceeded to 10 kg. Find out or calculated shipping cost.

Input:

```
Enter total weight of items : 11
Enter shipping destination : domestic
```

output:

```
1100
500(5kg) + 600(6kg)
```

```
import java.util.Scanner;
class Test
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter the total weight of items :");
        int weight = sc.nextInt(); // 11
```



```

        System.out.println("Enter the shipping destination :");
        String destination = sc.next().toLowerCase(); // domestic

        int cost =0;

        if(destination.equals("domestic"))
        {
            if(weight<=5)
            {
                cost = 500;
            }
            else
            {
                cost = 500 + (weight-5)*100;
            }
        }
        if(destination.equals("international"))
        {
            if(weight<=5)
            {
                cost = 1000;
            }
            else if(weight<=10)
            {
                cost = 1000 + (weight-5)*200;
            }
            else
            {
                cost = 1000 + (weight-5)*200 + 500;
            }
        }
        System.out.println("Shipping Cost :"+cost);
    }
}

```

v) switch case

=====

It is used to execute the source code based on multiple conditions.
It is similar to if else if ladder.

syntax:

```

switch(condition/expression)
{
    case value1 : //code to be execute
                break;
    case value2 : //code to be execute
                break;
    -
    -
    default : //code to be execute when all cases are false

}

```

```

ex:
---
import java.util.Scanner;
class Test
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);

        System.out.println("Enter the option :");
        int option = sc.nextInt();

        switch(option)
        {
            case 100 : System.out.println("It is a police
number");
                        break;
            case 103 : System.out.println("It is a enquiry
number");
                        break;
            case 108 : System.out.println("It is a emergency
number");
                        break;
            default:  System.out.println("Invalid option");
        }
    }
}

```

Declaration of break statement is optional. If we won't declare then from where our condition is satisfied from there all cases will be executed that state is called "Fall Throught State" of switch case.

```

ex:
---
import java.util.Scanner;
class Test
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);

        System.out.println("Enter the option :");
        int option = sc.nextInt();

        switch(option)
        {
            case 100 : System.out.println("It is a police
number");
                        //break;
            case 103 : System.out.println("It is a enquiry
number");

```

```

                                //break;
        case 108 : System.out.println("It is a emergency
number");
                                //break;
        default:  System.out.println("Invalid option");
    }
}

```

Note:

We need to use switch case when we want to read options from the enduser.

The allowed datatype for switch case are byte,short,int,char and string.

Q) Write a java program to find out given alphabet is a vowel or consonent?

```

import java.util.Scanner;
class Test
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);

        System.out.println("Enter the alphabet :");
        char ch = sc.next().charAt(0);

        ch = Character.toLowerCase(ch);

        switch(ch)
        {
            case 'a','e','i','o','u' -> System.out.println("It is
a vowel");
            default -> System.out.println("It is a consonent");
        }
    }
}

```

Q) Write a java program to accept day of a week then display it a weekday or weekend?

ex:

```

import java.util.Scanner;
class Test
{
    public static void main(String[] args)

```

```

{
    Scanner sc = new Scanner(System.in);

    System.out.println("Enter the day of a week :");
    int day = sc.nextInt();

    switch(day)
    {
        case 1,2,3,4,5 -> System.out.println("Weekday");
        case 6,7 -> System.out.println("weekend");
        default -> System.out.println("Invalid Day");
    }
}

```

yield
=====

A yield keyword introduced in Java 14.

It is used to return a value from a case block within a switch expression.

ex:

```

import java.util.Scanner;
class Test
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);

        System.out.println("Enter the day of a week :");
        int day = sc.nextInt();

        String result = switch(day)
        {
            case 1,2,3,4,5 -> { yield "Weekday"; }
            case 6,7 -> { yield "Weekend"; }
            default -> { yield "invalid day"; }
        };

        System.out.println(result);
    }
}

```

2) Iteration statements

=====

Iteration statement is used to execute the code repeatedly.

Iteration statement is possible by using loops.

We have four types of loops.

i) do while loop

ii) while loop

iii) for loop

iv) for each loop

i) do while loop

It executes the source code how long our condition is true.

syntax:

```
do
{
    -
    - //code to be execute
    -
}while(condition);
```

ex:

```
class Test
{
    public static void main(String[] args)
    {
        int i=1;
        do
        {
            System.out.print(i+" "); // infinite 1
        }
        while (i<=10);
    }
}
```

In do while loop, our code will execute atleast for time either our condition is true or false.

ex:

```
class Test
{
    public static void main(String[] args)
    {
        int i=11;
        do
```

```

        {
            System.out.print(i+" "); // 11
        }
        while (i<=10);
    }
}

```

Q) Write a java program to display 10 natural numbers?

```

class Test
{
    public static void main(String[] args)
    {
        int i=1;
        do
        {
            System.out.print(i+" "); //1 2 3 4 5 6 7 8 9 10
            i++;
        }
        while (i<=10);
    }
}

```

Q) Write a java program to perform sum of 10 natural numbers?

```

class Test
{
    public static void main(String[] args)
    {
        int i=1,sum=0;
        do
        {
            sum=sum+i;
            i++;
        }
        while (i<=10);

        System.out.println(sum);
    }
}

```

Q) Write a java program to find out factorial of a given number?

input:
n=5;

output:
120 (5*4*3*2*1)

```

import java.util.Scanner;
class Test

```

```

{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter the number :");
        int n = sc.nextInt(); // 5

        int i=n, fact=1;
        do
        {
            fact = fact * i;
            i--;
        }
        while (i>=1);

        System.out.println(fact);
    }
}

```

Q) Write a java program to display multiplication table of a given number?

input:

n=5

output:

5 * 1 = 5

-

-

5 * 10 = 50

```
import java.util.Scanner;
```

```
class Test
```

```

{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter the number :");
        int n = sc.nextInt(); // 5

        int i=1;
        do
        {
            System.out.println(n+" * "+i+" = "+n*i);
            i++;
        }
        while (i<=10);
    }
}

```

ii) while loop

=====

It is used to execute the source code how long our condition is true.

syntax:

```
while(condition)
{
    -
    - //set of statements
    -
}
```

ex:

```
class Test
{
    public static void main(String[] args)
    {
        int i=1;
        while(i<=10)
        {
            System.out.print(i+" "); //infinite 1
        }
    }
}
```

ex:

```
class Test
{
    public static void main(String[] args)
    {
        int i=11;
        while(i<=10)
        {
            System.out.print(i+" "); // no output
        }
    }
}
```

Q) Write a java program to display 10 natural numbers?

```
class Test
{
    public static void main(String[] args)
    {
        int i=1;
        while(i<=10)
        {
            System.out.print(i+" "); //1 2 3 4 5 6 7 8 9 10
            i++;
        }
    }
}
```



```
    }  
}
```

Q) Write a java program to perform sum of 10 natural numbers?

```
class Test  
{  
    public static void main(String[] args)  
    {  
        int i=1,sum=0;  
        while(i<=10)  
        {  
            sum = sum + i;  
            i++;  
        }  
        System.out.println(sum);  
    }  
}
```

Q) Write a java program to find out factorial of a given number ?

input:

n=5

output:

120 (5*4*3*2*1)

```
import java.util.Scanner;  
class Test  
{  
    public static void main(String[] args)  
    {  
        Scanner sc = new Scanner(System.in);  
        System.out.println("Enter the number :");  
        int n = sc.nextInt(); // 5  
  
        int i=n,fact=1;  
        while(i>=1)  
        {  
            fact = fact * i;  
            i--;  
        }  
        System.out.println(fact);  
    }  
}
```

Q) Write a java program to display multiplication table of a given number?

input:

n=5

output:

5 * 1 = 5

-
-
5 * 10 = 50

```
import java.util.Scanner;
class Test
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter the number :");
        int n = sc.nextInt(); // 5

        int i=1;
        while(i<=10)
        {
            System.out.println(n+" * "+i+" = "+n*i);
            i++;
        }
    }
}
```

Q) Write a java program to perform sum of digits of a given number?

input:
123
output:
6

```
import java.util.Scanner;
class Test
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter the number :");
        int n = sc.nextInt(); // 123

        int rem,sum=0;
        while(n>0)
        {
            rem = n%10;
            sum = sum + rem;
            n = n/10;
        }
        System.out.println(sum);
    }
}
```

Q) Write a java program to display special number?

input:
978
output:
6

```
import java.util.Scanner;
class Test
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter the number :");
        int n = sc.nextInt(); // 978

        while(n>9)
        {
            //sum of digits
            int rem,sum=0;
            while(n>0)
            {
                rem = n%10;
                sum = sum + rem;
                n = n/10;
            }
            n = sum;
        }
        System.out.println(n);
    }
}
```

Q) Write a java program to find out given number is armstrong or not?

input:
153
output:
It is a armstrong number (1*1*1+5*5*5+3*3*3) (1+125+27) (153)

```
import java.util.Scanner;
class Test
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter the number :");
        int n = sc.nextInt(); // 153

        int temp = n;

        int rem,sum=0;
        while(n>0)
        {
```

```

        rem = n%10;
        sum = sum + rem * rem * rem;
        n = n /10;
    }
    if(temp==sum)
        System.out.println("It is a armstrong number");
    else
        System.out.println("It is not a armstrong number");
    }
}

```

Q) Write a java program to find out reverse of a given number?

input:
123
output
321

```

import java.util.Scanner;
class Test
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter the number :");
        int n = sc.nextInt(); // 123

        int rem,rev=0;
        while(n>0)
        {
            rem = n%10;
            rev = rev * 10 + rem;
            n = n/10;
        }
        System.out.println(rev);
    }
}

```

Q) Write a java program to find out given number is palindrome or not?

input:
121
output:
It is a palindrome number

```

import java.util.Scanner;
class Test
{
    public static void main(String[] args)
    {

```

```

Scanner sc = new Scanner(System.in);
System.out.println("Enter the number :");
int n = sc.nextInt(); // 123

int temp = n;

int rem, rev=0;
while(n>0)
{
    rem = n%10;
    rev = rev * 10 + rem;
    n = n/10;
}

if(temp==rev)
    System.out.println("It is a palindrome number");
else
    System.out.println("It is not a palindrome number");
}
}

```

Q) Write a java program to display reverse of a given number?

input:
123
output:
ThreeTwoOne

```

import java.util.Scanner;
class Test
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter the number :");
        int n = sc.nextInt();//123

        while(n>0)
        {
            switch(n%10)
            {
                case 0 : System.out.print("Zero");break;
                case 1 : System.out.print("One");break;
                case 2 : System.out.print("Two");break;
                case 3 : System.out.print("Three");break;
                case 4 : System.out.print("Four");break;
                case 5 : System.out.print("Five");break;
                case 6 : System.out.print("Six");break;
                case 7 : System.out.print("Seven");break;
                case 8 : System.out.print("Eight");break;
                case 9 : System.out.print("Nine");break;
            }
            n = n/10;
        }
    }
}

```

```

    }
}

```

Q) A frog fell down in a well of 10 meters. Every day frog tries to come out from the well. He jumps 2 meters and rollback 1 meter. So write a java program to find out in how many days a frog will come out from the well.

```

class Test
{
    public static void main(String[] args)
    {
        int wellLength = 10;
        int jump = 2;
        int rollback =1;

        int days=0;
        int currentPosition=0;

        while(currentPosition<wellLength)
        {
            currentPosition+=jump;
            if(currentPosition>wellLength)
            {
                break;
            }
            currentPosition-=rollback;
            days++;
        }
        System.out.println("In "+days+" days frog will come out
from the well");
    }
}

```

iii) for loop
=====

It is used to execute the source code how long our condition is true.

syntax:

```

-----
for(initialization;condition;increment/decrement)
{
    -
    - //code to be execute
    -
}

```

If number of iterations are known by the user then we need to use for loop.

If number of iterations are not known by the user then we need to use while loop.

If number of iterations are not known by the user but code must execute atleast for one time then we need to use do while loop.

ex:

```
class Test
{
    public static void main(String[] args)
    {
        for(int i=1;i<=10;i++)
        {
            System.out.print(i+" "); //1 2 3 4 5 6 7 8 9 10
        }
    }
}
```

ex:

```
class Test
{
    public static void main(String[] args)
    {
        for(int i=1;i<=10;i++)
        {
            if(i%2==0)
            {
                System.out.print(i+" "); // 2 4 6 8 10
            }
        }
    }
}
```

ex:

```
class Test
{
    public static void main(String[] args)
    {
        int cnt=0;
        for(int i=1;i<=10;i++)
        {
            if(i%2==0) // 2 4 6 8 10
            {
                cnt++;
            }
        }
    }
}
```

```

        }
        System.out.println(cnt);//5
    }
}

ex:
----
class Test
{
    public static void main(String[] args)
    {
        int cnt=0;
        for(int i=1;i<=10;i++)
        {
            if(i%2==0)
            {
                System.out.print(i+" "); //2 6 10
                i+=2;
            }
        }
    }
}

```

```

ex:
---
class Test
{
    public static void main(String[] args)
    {
        for(;;)
        {
            System.out.print("Hello "); // infinite Hello
        }
    }
}

```

Q) Write a java program to find out given number is prime or not?

input :
5
output:
It is a prime number

```

import java.util.Scanner;
class Test
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter the number :");
        int n =sc.nextInt(); //5
    }
}

```



```

        boolean flag=true;
        for(int i=2;i<=n/2;i++)
        {
            if(n%i==0)
            {
                flag=false;
                break;
            }
        }
        if(flag==true)
            System.out.println("It is a prime number");
        else
            System.out.println("It is not a prime number");
    }
}

```

Q) Write a java program to display prime numbers between 1 to 100?

output:

```

2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41,
43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97.

```

ex:

```

class Test
{
    public static void main(String[] args)
    {
        for(int n=2;n<=100;n++)
        {
            //prime logic
            boolean flag=true;
            for(int i=2;i<=n/2;i++)
            {
                if(n%i==0)
                {
                    flag=false;
                    break;
                }
            }
            if(flag==true)
                System.out.print(n+" ");
        }
    }
}

```

Q) Write a java program to find out fibonacci series of a given number?

input:

6
output:
0 1 1 2 3 5 8

```
import java.util.Scanner;
class Test
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter the number :");
        int n = sc.nextInt();//5

        int a=0,b=1,c;
        System.out.print(a+" "+b+" ");

        for(int i=2;i<=n;i++)
        {
            c = a + b;
            System.out.print(c+" ");
            a=b;
            b=c;
        }

    }
}
```

Q) Write a java program to find out given number is perfect or not?

input:
6

output:
It is a perfect number

```
import java.util.Scanner;
class Test
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter the number :");
        int n = sc.nextInt();//6

        int sum=0;
        for(int i=1;i<n;i++)
        {
            if(n%i==0) // 6%1==0 , 6%2==0 , 6%3==0
            {
                sum+=i;
            }
        }
    }
}
```

```

        }
    }
    if(sum==n)
        System.out.println("It is a perfect number");
    else
        System.out.println("It is not a perfect number");
}
}

```

Q) Write a java program to find out GCD (Greatest Common Divisor) of two numbers?

input:
 12 18
 output:
 6

```

class Test
{
    public static void main(String[] args)
    {
        int a=12,b=18,gcd=0;

        for(int i=1;i<=a && i<=b;i++)
        {
            if((a%i==0) && (b%i==0))
            {
                gcd = i;
            }
        }
        System.out.println(gcd);
    }
}

```

Assignment
 =====

Q) Write a java program to display given number in words?

input:
 123
 output:
 One hundreded twenty three

iv) for each loop
 =====

It is used to iterate the elements from arrays/Collections.

syntax:

```

-----
    for(variable-declaration : array/Collection)
    {
        -
        - //code to be execute
        -
    }

ex:
---

class Test
{
    public static void main(String[] args)
    {
        int[] arr={10,20,30};

        for(int i : arr)
        {
            System.out.print(i+" ");
        }
    }
}

ex:
---
import java.util.List;
class Test
{
    public static void main(String[] args)
    {
        List<Integer> list = List.of(10,20,30,40);

        for(int i : list)
        {
            System.out.print(i+" ");
        }
    }
}

```

Loop Patterns =====

1)

```

1 1 1 1
2 2 2 2
3 3 3 3
4 4 4 4

```

```

class Test
{
    public static void main(String[] args)

```

```

        {
            //rows
            for(int i=1;i<=4;i++)
            {
                //cols
                for(int j=1;j<=4;j++)
                {
                    System.out.print(i+" ");
                }
                //new line
                System.out.println();
            }
        }
    }
}

```

2)

```

1 2 3 4
1 2 3 4
1 2 3 4
1 2 3 4

```

```

class Test
{
    public static void main(String[] args)
    {
        //rows
        for(int i=1;i<=4;i++)
        {
            //cols
            for(int j=1;j<=4;j++)
            {
                System.out.print(j+" ");
            }
            //new line
            System.out.println();
        }
    }
}

```

3)

```

* * * *
* * * *
* * * *
* * * *

```

```

class Test
{
    public static void main(String[] args)
    {
        //rows
    }
}

```

```

        for(int i=1;i<=4;i++)
        {
            //cols
            for(int j=1;j<=4;j++)
            {
                System.out.print("* ");
            }
            //new line
            System.out.println();
        }
    }
}

```

4)

```

4 4 4 4
3 3 3 3
2 2 2 2
1 1 1 1

```

```

class Test
{
    public static void main(String[] args)
    {
        //rows
        for(int i=4;i>=1;i--)
        {
            //cols
            for(int j=1;j<=4;j++)
            {
                System.out.print(i+" ");
            }
            //new line
            System.out.println();
        }
    }
}

```

5)

```

* * * *
*   *
*   *
*   *
* * * *

```

```

class Test
{
    public static void main(String[] args)
    {
        //rows
        for(int i=1;i<=4;i++)

```

```

        {
            //cols
            for(int j=1;j<=4;j++)
            {
                if(i==1 || i==4 || j==1 || j==4)
                    System.out.print("* ");
                else
                    System.out.print("  ");
            }
            //new line
            System.out.println();
        }
    }
}

```

6)

```

* - - -
- * - -
- - * -
- - - *

```

```

class Test
{
    public static void main(String[] args)
    {
        //rows
        for(int i=1;i<=4;i++)
        {
            //cols
            for(int j=1;j<=4;j++)
            {
                if(i==j)
                    System.out.print("* ");
                else
                    System.out.print("- ");
            }
            //new line
            System.out.println();
        }
    }
}

```

7)

```

* - - - *
- * - * -
- - * - -
- * - * -

```

* - - - *

```
class Test
{
    public static void main(String[] args)
    {
        //rows
        for(int i=1;i<=5;i++)
        {
            //cols
            for(int j=1;j<=5;j++)
            {
                if(i==j || i+j==6)
                    System.out.print("* ");
                else
                    System.out.print("- ");
            }
            //new line
            System.out.println();
        }
    }
}
```

8)

```
      *
      *
* * * * *
      *
      *
```

```
class Test
{
    public static void main(String[] args)
    {
        //rows
        for(int i=1;i<=5;i++)
        {
            //cols
            for(int j=1;j<=5;j++)
            {
                if(i==3 || j==3)
                    System.out.print("* ");
                else
                    System.out.print("  ");
            }
            //new line
            System.out.println();
        }
    }
}
```

9)


```

4 4 4 4 4 4 4
4 3 3 3 3 3 4
4 3 2 2 2 3 4
4 3 2 1 2 3 4
4 3 2 2 2 3 4
4 3 3 3 3 3 4
4 4 4 4 4 4 4

```

```

class Test
{
    public static void main(String[] args)
    {
        int size=7,n=4;

        for(int i=0;i<size;i++)
        {
            for(int j=0;j<size;j++)
            {
                int result = n - Math.min(Math.min(i,j),Math.min(size-i-1,size-j-1));
                System.out.print(result+" ");
            }
            //new line
            System.out.println();
        }
    }
}

```

Left Side Patterns
=====

```

1)

1
2 2
3 3 3
4 4 4 4

```

```

class Test
{
    public static void main(String[] args)
    {
        //rows
        for(int i=1;i<=4;i++)
        {
            //cols
            for(int j=1;j<=i;j++)
            {

```

```

        System.out.print(i+" ");
    }
    //new line
    System.out.println();
}

}

```

2)

```

1
1 2
1 2 3
1 2 3 4

```

```

class Test
{
    public static void main(String[] args)
    {
        //rows
        for(int i=1;i<=4;i++)
        {
            //cols
            for(int j=1;j<=i;j++)
            {
                System.out.print(i+" ");
            }
            //new line
            System.out.println();
        }
    }
}

```

3)

```

*
* *
* * *
* * * *

```

```

class Test
{
    public static void main(String[] args)
    {
        //rows
        for(int i=1;i<=4;i++)
        {
            //cols
            for(int j=1;j<=i;j++)

```

```

        {
            System.out.print("* ");
        }
        //new line
        System.out.println();
    }

}

4)

4 4 4 4
3 3 3
2 2
1

class Test
{
    public static void main(String[] args)
    {
        //rows
        for(int i=4;i>=1;i--)
        {
            //cols
            for(int j=1;j<=i;j++)
            {
                System.out.print(i+" ");
            }
            //new line
            System.out.println();
        }
    }
}

```

```

5)

1
2 3
4 5 6
7 8 9 0

class Test
{
    public static void main(String[] args)
    {
        int k=1;
        //rows
        for(int i=1;i<=4;i++)
        {

```

```

        //cols
        for(int j=1;j<=i;j++)
        {
            if(k<=9)
                System.out.print(k++ +" ");
            else
                System.out.print("0 ");
        }
        //new line
        System.out.println();
    }

}

6)

2
4 6
8 10 12
14 16 18 20

class Test
{
    public static void main(String[] args)
    {
        int k=2;

        //rows
        for(int i=1;i<=4;i++)
        {
            //cols
            for(int j=1;j<=i;j++)
            {
                System.out.print(k+" ");
                k+=2;
            }
            //new line
            System.out.println();
        }
    }
}

7)

2
3 5
7 11 13
17 19 23 29

8)

```

```

1
1 # 2
1 # 2 # 3
1 # 2 # 3 # 4

```

```

class Test
{
    public static void main(String[] args)
    {
        //rows
        for(int i=1;i<=4;i++)
        {
            //cols
            for(int j=1;j<=i;j++)
            {
                if(j==1)
                    System.out.print(j);
                else
                    System.out.print(" # "+j);
            }
            //new line
            System.out.println();
        }
    }
}

```

9)

```

1
2 1
1 2 3
4 3 2 1

```

```

class Test
{
    public static void main(String[] args)
    {
        //rows
        for(int i=1;i<=4;i++)
        {
            if(i%2!=0)
            {
                for(int j=1;j<=i;j++)
                {
                    System.out.print(j+" ");
                }
                //new line
                System.out.println();
            }
        }
    }
}

```

```

        else
        {
            for(int j=i;j>=1;j--)
            {
                System.out.print(j+" ");
            }
            //new line
            System.out.println();
        }
    }
}

```

Right Side Patterns

=====

1)

```

    1
  2 2
 3 3 3
4 4 4 4

```

```

class Test

```

```

{
    public static void main(String[] args)
    {
        //rows
        for(int i=1;i<=4;i++)
        {
            //space
            for(int j=4;j>i;j--)
            {
                System.out.print(" ");
            }

            //right side
            for(int j=1;j<=i;j++)
            {
                System.out.print(i+" ");
            }
            //new line
            System.out.println();
        }
    }
}

```

2)

```

    *
  * *
 * * *
* * * *

```

```

class Test
{
    public static void main(String[] args)
    {
        //rows
        for(int i=1;i<=4;i++)
        {
            //space
            for(int j=4;j>i;j--)
            {
                System.out.print("  ");
            }

            //right side
            for(int j=1;j<=i;j++)
            {
                System.out.print("* ");
            }
            //new line
            System.out.println();
        }
    }
}

```

Pyramid loop patterns
=====

1)

```

      1
    1 2 1
  1 2 3 2 1
1 2 3 4 3 2 1

```

```

class Test
{
    public static void main(String[] args)
    {
        //rows
        for(int i=1;i<=4;i++)
        {
            //space
            for(int j=4;j>i;j--)
            {
                System.out.print("  ");
            }

            //left side
            for(int j=1;j<=i;j++)
            {
                System.out.print(j+" ");
            }
        }
    }
}

```

```

        }

        //right side
        for(int j=i-1;j>=1;j--)
        {
            System.out.print(j+" ");
        }
        //new line
        System.out.println();
    }
}

```

2)

```

1 2 3 4 3 2 1
 1 2 3 2 1
   1 2 1
    1

```

```

class Test
{
    public static void main(String[] args)
    {
        //rows
        for(int i=4;i>=1;i--)
        {
            //space
            for(int j=4;j>i;j--)
            {
                System.out.print(" ");
            }

            //left side
            for(int j=1;j<=i;j++)
            {
                System.out.print(j+" ");
            }

            //right side
            for(int j=i-1;j>=1;j--)
            {
                System.out.print(j+" ");
            }
            //new line
            System.out.println();
        }
    }
}

```

3)


```

      *
    * * *
  * * * * *
* * * * * * *

```

```

class Test
{
    public static void main(String[] args)
    {
        //rows
        for(int i=1;i<=4;i++)
        {
            //space
            for(int j=4;j>i;j--)
            {
                System.out.print("  ");
            }

            //left side
            for(int j=1;j<=i;j++)
            {
                System.out.print("* ");
            }

            //right side
            for(int j=i-1;j>=1;j--)
            {
                System.out.print("* ");
            }

            //new line
            System.out.println();
        }
    }
}

```

Q) Write a java program to display below loop pattern?

```

1           1
1 2         2 1
1 2 3       3 2 1
1 2 3 4 4 3 2 1

```

```

class Test
{
    public static void main(String[] args)
    {
        int rows=4;

        for(int i=1;i<=rows;i++)

```

```

        {
            //left side pattern
            for(int j=1;j<=i;j++)
            {
                System.out.print(j+" ");
            }

            //space
            for(int j=1;j<=(rows-i)*2;j++)
            {
                System.out.print(" ");
            }

            //right side pattern
            for(int j=i;j>=1;j--)
            {
                System.out.print(j+" ");
            }
            //new line
            System.out.println();
        }
    }
}

```

Q) Write a java program to display below loop pattern?

```

6 5 4 3 2 1
 6 5 4 3 2
  6 5 4 3
   6 5 4
    6 5
     6

```

```

class Test
{
    public static void main(String[] args)
    {
        for(int i=1;i<=6;i++)
        {
            //space
            for(int j=1;j<i;j++)
            {
                System.out.print(" ");
            }

            //elements
            for(int j=6;j>=i;j--)
            {
                System.out.print(j+" ");
            }
            //new line

```

```

        System.out.println();
    }
}

```

Q) Write a java program to display pascal triangle?

```

1
1 1
1 2 1
1 3 3 1
1 4 6 4 1

```

```

class Test
{
    public static void main(String[] args)
    {
        for(int i=0;i<5;i++)
        {
            //space
            for(int j=4;j>i;j--)
            {
                System.out.print(" ");
            }

            int result = 1;
            for(int k=0;k<=i;k++)
            {
                System.out.print(result+" ");
                result = result * (i-k)/(k+1);
            }
            //new line
            System.out.println();
        }
    }
}

```

Q) Write a java program to display below loop pattern?

```

*
 * *
* * *
* * * *
* * * * *

```

```

class Test
{
    public static void main(String[] args)
    {

```

```

        for(int i=0;i<5;i++)
        {
            //space
            for(int j=4;j>i;j--)
            {
                System.out.print(" ");
            }

            for(int k=0;k<=i;k++)
            {
                System.out.print("* ");
            }
            //new line
            System.out.println();
        }
    }
}

```

Screening Test

=====

Q) What will be the output of below snippet?

```

/*
    Operators precedence
    -----
    -> *, / , %, + , - , << and >>.
*/
class Test
{
    public static void main(String[] args)
    {
        int i = 10>>2/7*4+7<<5*2+10/20-2;
        System.out.println(i);
    }
}
/*
    10>>0+7<<10+0-2
    10>>7<<10-2
    10>>7<<8
    10 >> 256
    0
*/

```

3) Jump Statements

=====

It is used to jump from one section of code another section.

We have three jump statements in java.

1) break stmt

2) continue stmt

3) return stmt

1) break stmt

It is used to break the execution of loops and switch case.

For conditional statements we can use if condition.

syntax:

```
break;
```

ex:

```
class Test
{
    public static void main(String[] args)
    {
        System.out.println("stmt1");
        break;
        System.out.println("stmt2");
    }
}
```

o/p:

C.T.E : break outside switch or loop

ex:

```
class Test
{
    public static void main(String[] args)
    {
        System.out.println("stmt1");
        if(false)
        {
            break;
        }
        System.out.println("stmt2");
    }
}
```

o/p:

C.T.E : break outside switch or loop

ex:

```
class Test
{
    public static void main(String[] args)
    {
        for(int i=1;i<=10;i++)
        {
```

```

        if(i==5)
        {
            break;
        }

        System.out.print(i+" "); // 1 2 3 4
    }
}

```

2) continue stmt

It is used to continue the execution of loops.

For conditional statements we can use if condition.

syntax:

```
continue;
```

ex:

```
class Test
```

```
{
    public static void main(String[] args)
    {
        System.out.println("stmt1");
        continue;
        System.out.println("stmt2");
    }
}

```

o/p:

C.T.E : continue outside of loop

ex:

```
class Test
```

```
{
    public static void main(String[] args)
    {
        System.out.println("stmt1");
        if(false)
        {
            continue;
        }
        System.out.println("stmt2");
    }
}

```

o/p:

C.T.E : continue outside of loop

ex:

```

class Test
{
    public static void main(String[] args)
    {
        for(int i=1;i<=10;i++)
        {
            if(i==5)
            {
                continue;
            }
            System.out.print(i+" "); // 1 2 3 4 6 7 8 9 10
        }
    }
}

```

3) return stmt

It is used to exit from methods.

It contains optional value.

syntax:

```
return;
```

ex:

```
import java.util.Scanner;
```

```
class Test
```

```

{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);

        System.out.println("Enter the option :");
        int option = sc.nextInt();

        if(option==100)
            System.out.println("Police number");
        else if(option==103)
            System.out.println("Enquiry number");
        else if(option==108)
            System.out.println("Emergency number");
        else
            return;
    }
}

```

Various ways to declare the methods in java

=====

There are four ways to declare the methods in java.

1) No returntype with No argument method

- 2) No returntype with Argument method
- 3) With returntype with No argument method
- 4) With returntype with Argument method

1) No returntype with No argument method

If there is no argument then we need to ask our input values inside callie method.

Q) Write a java program to perform sum of two numbers using No returntype with No argument method?

```
import java.util.Scanner;
class Test
{
    public static void main(String[] args)
    {
        //caller method
        sum();
    }
    //callie method
    public static void sum()
    {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter the two numbers :");
        int a = sc.nextInt();//10
        int b = sc.nextInt();//20
        int c = a + b;
        System.out.println("sum of two numbers is =" + c);
    }
}
```

Q) Write a java program to find out factorial of a given number using No returntype with No argument method?

```
import java.util.Scanner;
class Test
{
    public static void main(String[] args)
    {
        //caller method
        factorial();
    }
    //callie method
    public static void factorial()
    {
```



```

        Scanner sc = new Scanner(System.in);
        System.out.println("Enter the number :");
        int n = sc.nextInt();//5

        int fact=1;
        for(int i=n;i>=1;i--)
        {
            fact*=i;
        }
        System.out.println("Factorial of a given number is
="+fact);
    }
}

```

2) No returntype with Argument method

If we have arguments then we need to ask input values inside main method.

Number of arguments depends upon number of inputs.

Q) Write a java program to perform sum of two numbers using no returntype with argument method?

```

import java.util.Scanner;
class Test
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter the two numbers :");
        int a = sc.nextInt();//10
        int b = sc.nextInt();//20

        //caller method
        sum(a,b);
    }
    //callie method
    public static void sum(int a,int b)
    {
        int c = a + b;
        System.out.println("Sum of two numbers is =" +c);
    }
}

```

Q) Write a java program to find out factorial of a given number using with no returntype with argument method?

```

import java.util.Scanner;
class Test

```

```

{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter the number :");
        int n = sc.nextInt();

        //caller method
        factorial(n);
    }
    //callie method
    public static void factorial(int n)
    {
        int fact=1;
        for(int i=n;i>=1;i--)
        {
            fact*=i;
        }
        System.out.println("Factorial of a given number is
="+fact);
    }
}

```

3) With returntype with No argument method

A returntype is completely depend upon output datatype.

A method can return only one value at a time.

Q) Write a java program to perform sum of two numbers using with returntype with no argument method?

```

import java.util.Scanner;
class Test
{
    public static void main(String[] args)
    {
        //caller method
        int k = sum();
        System.out.println("sum of two numbers is =" +k);
    }
    //callie method
    public static int sum()
    {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter the two numbers :");
        int a = sc.nextInt();
        int b = sc.nextInt();
        int c = a + b;
    }
}

```

```

        return c;
    }
}

```

Q) Write a java program to find out area of a circle using with returntype with no argument method?

```

import java.util.Scanner;
class Test
{
    public static void main(String[] args)
    {
        //caller method
        double k = circle();
        System.out.printf("Area of a circle is =%.2f",k);
    }
    //callie method
    public static double circle()
    {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter the radius :");
        int r = sc.nextInt();

        double area = Math.PI*Math.pow(r,2);
        return area;
    }
}

```

4) With returntype with Argument method

We can declare multiple return statements but at a time we can return only one value.

Q) Write a java program to perform sum of two numbers using with returntype with argument method?

ex:

```

import java.util.Scanner;
class Test
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter the two numbers :");
        int a = sc.nextInt();
        int b = sc.nextInt();

        //caller method
        System.out.println("sum of two numbers is =" +sum(a,b));
    }
}

```

```

    }
    //callie method
    public static int sum(int a,int b)
    {
        int c = a + b;
        return c;
    }
}

```

Q) Write a java program to find out given number is even or odd using with returntype with argument method?

approach1

```

-----
import java.util.Scanner;
class Test
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter the number :");
        int n = sc.nextInt();//5

        //caller method
        System.out.println(find(n));
    }
    //callie method
    public static String find(int n)
    {
        if(n%2==0)
            return "It is even number";
        else
            return "It is odd number";
    }
}

```

approach2

```

-----
import java.util.Scanner;
class Test
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter the number :");
        int n = sc.nextInt();//5

        //caller method
        int k = find(n);
        if(k==1)
            System.out.println("It is even number");
    }
}

```

```

        else
            System.out.println("It is odd number");
    }
    //callie method
    public static int find(int n)
    {
        if(n%2==0)
            return 1;
        else
            return 0;
    }
}

```

approach3

```

import java.util.Scanner;
class Test
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter the number :");
        int n = sc.nextInt();//5

        //caller method
        if(find(n))
            System.out.println("It is even number");
        else
            System.out.println("It is odd number");
    }
    //callie method
    public static boolean find(int n)
    {
        if(n%2==0)
            return true;
        else
            return false;
    }
}

```

Recursion

=====

It is a programming technique which breaks complex program into sub-programs.

A method which call itself for many number of times is called recursion.

Recursion is similar to loopings. If we use recursion then we should not use loops.

Q) Write a java program to display 10 natural numbers without using loops?

```
class Test
{
    public static void main(String[] args)
    {
        //caller method
        display(1);
    }
    //callie method
    public static void display(int i)
    {
        if(i<=10)
        {
            System.out.print(i+" "); // 1 2 3 4 5 6 7 8 9 10

            display(i+1);
        }
    }
}
```

Q) Write a java program to perform sum of two numbers without using arithmetic operator (+) ?

```
import java.util.Scanner;
class Test
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter the two numbers :");
        int a = sc.nextInt();//5
        int b = sc.nextInt();//10

        //caller method
        System.out.println(sum(a,b));
    }
    //callie method
    public static int sum(int a,int b)
    {
        if(a==0)
            return b;

        return sum(--a,++b);
    }
}
```

Q) Write a java program to find out factorial of a given number using recursion?

```

import java.util.Scanner;
class Test
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter the number :");
        int n =sc.nextInt();//5

        //caller method
        System.out.println(factorial(n));
    }
    //callie method
    public static int factorial(int n)
    {
        if(n<0)
            return -1;
        if(n==0)
            return 1;

        return n*factorial(n-1);
    }
}

```

Q) Write a java program to find out N-th element of fibonacci series?

fibonacci sequence : 0 1 1 2 3 5 8

input:
4

output:
2

```

import java.util.Scanner;
class Test
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter the number :");
        int n =sc.nextInt();//4

        //caller method
        System.out.println(fib(n));
    }
    //callie method
    public static int fib(int n)
    {
        if(n==0 || n==1)

```

```

        return 0;
    if(n==2)
        return 1;

    return fib(n-1)+fib(n-2);
}
}

```

Arrays

=====

In a normal variable we can store only one value at a time.

To store more than one value we need to use array concept.

Array is a collection of homogeneous data elements.

The main advantages of arrays are

1) We can represent multiple elements using single variable name.

ex:

```
int[] arr={10,20,30};
```

2) Performance point of view arrays are recommended to use.

The main disadvantages of arrays are

1) Arrays are fixed in size. Once if we create an array there is no chance of increasing

or decreasing the size of an array.

2) To use array concept in advanced we should know what is the size of an array which

is always not possible.

In java, Arrays are classified into three types.

1) Single Dimensional Array

2) Double Dimensional Array

3) Multi Dimensional Array

Array Declaration

At the time of array declaration we should not specify array size.

ex:

```

                                Arrays
        |-----|-----|
        |
Single Dimensional Array   Double Dimensional Array   Multi
Dimensional Array

```



```

int[] arr;           int[][] arr;           int[][][] arr;
int []arr;           int [][]arr;           int [][][]arr;
int arr[];           int arr[][];           int arr[][][];
                    int[] []arr;           int[][] []arr;
                    int[] arr[];           int[][] arr[];
                    int []arr[];           int[] [][]arr;
                                int[] arr[][];
                                int[] []arr[];
                                int [][]arr[];
                                int []arr[][];

```

Array Creation

In java , every array consider as an object. Hence we will use new operator to create an array.

ex:

```
int[] arr=new int[3];
```

Rules to constructor an array

Rule1:

At the time of array creation compulsory we need to specify array size.

ex:

```
int[] arr = new int[3];
int[] arr = new int[]; // C.T.E Array dimension missing
```

Rule2:

It is legal to have an array size with zero.

ex:

```
int[] arr = new int[0];
```

Rule3:

We can't take negative number as an array size otherwise we will get

runtime exception called NegativeArraySizeException.

ex:

```
int[] arr = new int[-2]; // R.E NegativeArraySizeException
```

Rule4:

The allowed datatype for an array size with byte,short,int and char.

If we take other datatypes then we will get compile time error.

ex:

```
byte b=10;
```

```

int[] arr = new int[b];

int[] arr = new int['a'];

int[] arr = new int[10.5d]; // C.T.E

```

Rule5:

The maximum length we can take for an array size is maximum length of int.

ex:

```
int[] arr = new int[2147483647];
```

Array initialization

Once if we create an array , every array element is initialized with default values.

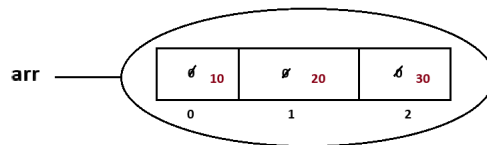
If we are not happy with default values then we can change with customized values.

Diagram: class25.1

```

int[] arr = new int[3];
arr[0]=10;
arr[1]=20;
arr[2]=30;

```



Array Declaration, Creation and Initialization using Single Line

```

int[] arr;
arr = new int[3];
arr[0]=10;
arr[1]=20;
arr[2]=30; ==> int[] arr = {10,20,30};
               ==> char[] arr = {'a','b','c'};
               ==> String[] arr = {"Hi","Hello","Bye"};

```

Q) What is the difference between length and length()?

length

It is applicable only for arrays.

It returns size of an array.

ex:

```
int[] arr = new int[5];
System.out.println(arr.length);//5
```

length()

It is applicable for string objects.

It returns number of characters present in a string.

ex:

```
String str="qualitythought";
System.out.println(str.length()); // 14
```

Single Dimensional Array Programs

=====

Q) Write a java program to accept array elements and display them?

```
import java.util.Scanner;
class Test
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter the array size :");
        int size = sc.nextInt(); // 4

        int[] arr = new int[size];

        //insert elements
        for(int i=0;i<arr.length;i++)
        {
            System.out.println("Enter the element :");
            arr[i] = sc.nextInt();
        }

        //display elements
        for(int i=0;i<arr.length;i++)
        {
            System.out.print(arr[i]+" ");
        }
    }
}
```

Q) Write a java program to display the elements one by one from array?

input:

7 9 1 4 6 2

output:

7
9
1
4
6
2

```
class Test
{
    public static void main(String[] args)
    {
        int[] arr = {7,9,1,4,6,2};

        for(int i : arr)
        {
            System.out.println(i);
        }
    }
}
```

Q) Write a java program to display array elements in reverse order?

input:

7 9 1 4 6 2

output:

2 6 4 1 9 7

```
class Test
{
    public static void main(String[] args)
    {
        int[] arr = {7,9,1,4,6,2};

        //reading reverse
        for(int i=arr.length-1;i>=0;i--)
        {
            System.out.print(arr[i]+" ");
        }
    }
}
```

Q) Write a java program to copy an array?

input:

7 9 1 4 6 2

```

class Test
{
    public static void main(String[] args)
    {
        int[] arr = {7,9,1,4,6,2};

        int[] copyArr = new int[arr.length];

        copyArr = arr;

        for(int i : copyArr)
        {
            System.out.print(i+" ");
        }
    }
}

```

Q) Write a java program to display even elements from given array?

input:

7 2 9 4 6 3

output:

2 4 6

```

class Test
{
    public static void main(String[] args)
    {
        int[] arr = {7,2,9,4,6,3};

        for(int n : arr)
        {
            if(n%2==0)
            {
                System.out.print(n+" ");
            }
        }
    }
}

```

Q) Write a java program to display prime elements from given array?

input:

9 7 4 3 10 13 18 2

output:

7 3 13 2

```

class Test
{

```

```

public static void main(String[] args)
{
    int[] arr = {9,7,4,3,10,13,18,2};

    for(int n : arr)
    {
        boolean flag=true;
        for(int i=2;i<=n/2;i++)
        {
            if(n%i==0)
            {
                flag=false;
                break;
            }
        }
        if(flag==true)
            System.out.print(n+" ");
    }
}

```

Q) Write a java program to display the array elements in sorting order?

input:
6 1 9 2 5 8
output:
1 2 5 6 8 9

```

ex:
---
import java.util.Arrays;
class Test
{
    public static void main(String[] args)
    {
        int[] arr = {6,1,9,2,5,8};

        Arrays.sort(arr);

        for(int i : arr)
        {
            System.out.print(i+" ");
        }
    }
}

```

Q) Write a java program to display the array elements in sorting order without using sort() method?

input:

```
6 1 9 2 5 8
output:
1 2 5 6 8 9
```

```
class Test
{
    public static void main(String[] args)
    {
        int[] arr = {6,1,9,2,5,8};

        for(int i=0;i<arr.length-1;i++)
        {
            if(arr[i] > arr[i+1])
            {
                int temp = arr[i];
                arr[i] = arr[i+1];
                arr[i+1] = temp;

                i=-1;
            }
        }

        //display
        for(int i : arr)
        {
            System.out.print(i+" ");
        }
    }
}
```

Q) Write a java program to display the array elements in descending order?

```
input:
6 1 9 2 5 8
output:
9 8 6 5 2 1
```

```
import java.util.Arrays;
class Test
{
    public static void main(String[] args)
    {
        int[] arr = {6,1,9,2,5,8};

        Arrays.sort(arr);

        //reading reverse
        for(int i=arr.length-1;i>=0;i--)
        {
            System.out.print(arr[i]+" ");
        }
    }
}
```

```

    }
}

```

Q) Write a java program to display the array elements in descending order without using sort() method?

input:
 6 1 9 2 5 8
 output:
 9 8 6 5 2 1

```

class Test
{
    public static void main(String[] args)
    {
        int[] arr = {6,1,9,2,5,8};

        for(int i=0;i<arr.length-1;i++)
        {
            if(arr[i] < arr[i+1])
            {
                int temp = arr[i];
                arr[i] = arr[i+1];
                arr[i+1] = temp;

                i=-1;
            }
        }
        //display elements
        for(int i : arr)
        {
            System.out.print(i+" ");
        }
    }
}

```

Q) Write a java program to find out highest element from given array?

input:
 6 1 9 2 5 8
 output:
 9

Approach1

 import java.util.Arrays;
 class Test


```

{
    public static void main(String[] args)
    {
        int[] arr = {6,1,9,2,5,8};

        Arrays.sort(arr); // 1 2 5 6 8 9

        System.out.println(arr[arr.length-1]);
    }
}

```

Approach2

```

class Test
{
    public static void main(String[] args)
    {
        int[] arr = {6,1,9,2,5,8};

        int big = arr[0];

        for(int ele : arr)
        {
            if(ele > big)
            {
                big = ele;
            }
        }
        System.out.println(big);
    }
}

```

Q) Write a java program to find out least element from given array?

input:

6 1 9 2 5 8

output:

1

Approach1

```

import java.util.Arrays;
class Test
{
    public static void main(String[] args)
    {
        int[] arr = {6,1,9,2,5,8};

        Arrays.sort(arr);

        System.out.println(arr[0]);
    }
}

```

```
}
```

Approach2

```
import java.util.Arrays;
class Test
{
    public static void main(String[] args)
    {
        int[] arr = {6,1,9,2,5,8};

        int small = arr[0];

        for(int ele : arr)
        {
            if(ele < small)
            {
                small = ele;
            }
        }
        System.out.println(small);
    }
}
```

Q) Write a java program to display three highest elements from given array?

input:

1 7 2 8 5 4 9

output:

9 8 7

```
class Test
{
    public static void main(String[] args)
    {
        int[] arr = {1,7,2,8,5,4,9};

        int firstElement = Integer.MIN_VALUE;
        int secondElement = Integer.MIN_VALUE;
        int thirdElement = Integer.MIN_VALUE;

        for(int i : arr)
        {
            if(i>firstElement)
            {
                thirdElement = secondElement;
                secondElement = firstElement;
                firstElement = i;
            }
            else if(i>secondElement)
```

```

        {
            thirdElement = secondElement;
            secondElement = i;
        }
        else if(i>thirdElement)
        {
            thirdElement = i;
        }
    } //for loop
    System.out.println(firstElement+" "+secondElement+"
    "+thirdElement);
}
}

```

Q) Write a java program to display unique elements from given array?

input:

4 8 1 3 9 6 3 8 7 4

output:

1 9 6 7

```

class Test
{
    public static void main(String[] args)
    {
        int[] arr = {4,8,1,3,9,6,3,8,7,4};

        for(int i=0;i<arr.length;i++)
        {
            int cnt=0;

            for(int j=0;j<arr.length;j++)
            {
                if(arr[i] == arr[j])
                {
                    cnt++;
                }
            }

            if(cnt==1)
                System.out.print(arr[i]+" ");
        }
    }
}

```

Q) Write a java program to find out most repeating element from given array?

input:
6 2 7 2 8 9 2 8 5 2 1
output:
2 repeating for 4 times

```
class Test
{
    public static void main(String[] args)
    {
        int[] arr = {6,2,7,2,8,9,2,8,5,2,1};

        int maxCount = 0;
        int element = 0;

        for(int i=0;i<arr.length;i++)
        {
            int cnt=0;

            for(int j=0;j<arr.length;j++)
            {
                if(arr[i] == arr[j])
                {
                    cnt++;
                }
            }
            if(cnt>maxCount)
            {
                maxCount = cnt;
                element = arr[i];
            }
        }
        System.out.println(element +" repeating for "+maxCount+"
times");
    }
}
```

Q) Write a java program to segregate array elements?

input:
1 0 1 1 0 0 1 0 1 0
output:
0 0 0 0 0 1 1 1 1 1

```
class Test
{
    public static void main(String[] args)
    {
        int[] arr = {1,0,1,1,0,0,1,0,1,0};
```

```

        int index=0;
        for(int i=0;i<arr.length;i++)
        {
            if(arr[i] == 0)
            {
                int temp = arr[i];
                arr[i]=arr[index];
                arr[index]=temp;

                index++;
            }
        }

        for(int i : arr)
        {
            System.out.print(i+" ");
        }
    }
}
Q) Write a java program to display pair of elements equals to given
sum=10 ?

```

input:
6 9 1 4 2 3 7 5

output:
6 4
9 1
3 7

```

class Test
{
    public static void main(String[] args)
    {
        int[] arr = {6,9,1,4,2,3,7,5};
        int sum = 10;

        for(int i=0;i<arr.length;i++)
        {
            for(int j=i+1;j<arr.length;j++)
            {
                if(arr[i]+arr[j]==sum)
                {
                    System.out.println(arr[i]+" "+arr[j]);
                }
            }
        }
    }
}

```

Q) Write a java program to display triplet of elements equals to given sum=10 ?

input:

6 9 1 4 2 3 7 5

output:

6 1 3

1 4 5

1 2 7

2 3 5

```
class Test
```

```
{
    public static void main(String[] args)
    {
        int[] arr = {6,9,1,4,2,3,7,5};
        int sum = 10;

        for(int i=0;i<arr.length;i++)
        {
            for(int j=i+1;j<arr.length;j++)
            {
                for(int k=j+1;k<arr.length;k++)
                {
                    if(arr[i]+arr[j]+arr[k]==sum)
                    {
                        System.out.println(arr[i]+" "+arr[j]+"
"+arr[k]);
                    }
                }
            }
        }
    }
}
```

Q) Write a java program to find out leader elements from given array?

input:

2 9 60 23 7 12

output:

60 23 12

```
class Test
```

```
{
    public static void main(String[] args)
    {
```

```

int[] arr = {2,9,60,23,7,12};

for(int i=0;i<arr.length;i++)
{
    boolean flag=true;
    for(int j=i+1;j<arr.length;j++)
    {
        if(arr[i]>arr[j])
        {
            continue;
        }
        else
        {
            flag=false;
            break;
        }
    }
    if(flag==true)
        System.out.print(arr[i]+" ");
}
}

```

Q) Write a java program to find out missing element from given array?

input:

6 7 3 5 1 2

output:

4

```

class Test
{
    public static void main(String[] args)
    {
        int[] arr = {6,7,3,5,1,2};

        int sum_of_ele = arr.length+1;

        int sum = (sum_of_ele * (sum_of_ele + 1))/2;

        for(int i : arr)
        {
            sum-=i;
        }
        System.out.println(sum);
    }
}

```

Q) Write a java program to display array elements which are greater to its immediate predecessor and successor and consider first and last element having only one neighbour?

input:

7 2 5 32 4 12 90

output:

7 32 90

```
class Test
{
    public static void main(String[] args)
    {
        int[] arr = {7,2,5,32,4,12,90};

        //first element
        if(arr[0]>arr[1])
        {
            System.out.print(arr[0]+" ");
        }

        //middle elements
        for(int i=1;i<arr.length-2;i++)
        {
            if(arr[i]>arr[i-1] && arr[i]>arr[i+1])
            {
                System.out.print(arr[i]+" ");
            }
        }

        //last element
        if(arr[arr.length-1]>arr[arr.length-2])
        {
            System.out.print(arr[arr.length-1]+" ");
        }
    }
}
```

Q) Write a java program to display all missing elements from given array?

input:

1 4 2 9

output:

3 5 6 7 8


```

import java.util.Arrays;
class Test
{
    public static void main(String[] args)
    {
        int[] arr = {1,4,2,9};

        Arrays.sort(arr); // {1,2,4,9}

        for(int i=0;i<arr.length-1;i++)
        {
            int current = arr[i];
            int next = arr[i+1];

            while((current+1)<next)
            {
                System.out.print((current+1)+" ");
                current++;
            }
        }
    }
}

```

Q) Write a java program to determine the smallest number of coins needed to total 86 rupees. Use the denominations provided in the array {1,2,5,10}?

Output:

```

1 coin(s) of 1 rupee(s)
1 coin(s) of 5 rupee(s)
8 coin(s) of 10 rupee(s)

```

ex:

```

import java.util.Arrays;
class Test
{
    public static void main(String[] args)
    {
        int[] denomination={1,2,5,10};
        int amount = 86;

        //caller method
        int[] result = minimumCoins(denomination,amount);

        for(int i=0;i<result.length;i++)
        {
            if(result[i]>0)
            {

```

```

        System.out.println(result[i]+ " coin(s) of
"+denomination[i]+" rupee(s)");
    }
}
//callie method
public static int[] minimumCoins(int[] denomination,int amt)
{
    int[] coinsCount = new int[denomination.length];

    for(int i=denomination.length-1;i>=0;i--)
    {
        coinsCount[i] = amt/denomination[i];
        amt = amt%denomination[i];
    }
    return coinsCount;
}
}

```

Q) Write a java program to display distinct elements from given array?

input:

5 9 1 3 7 6 9 3 4 2 2

output:

5 9 1 3 7 6 4 2

```

class Test
{
    public static void main(String[] args)
    {
        int[] arr = {5,9,1,3,7,6,9,3,4,2,2};

        for(int i=0;i<arr.length;i++)
        {
            int cnt=0;

            for(int j=0;j<i;j++)
            {
                if(arr[i]==arr[j])
                {
                    cnt++;
                    break;
                }
            }
            if(cnt==0)
                System.out.print(arr[i]+" ");
        }
    }
}

```

Q) Write a java program to merge two arrays and display them in sorting order?

input:

5 1 3 2 4
9 6 8 7 10

output:

1 2 3 4 5 6 7 8 9 10

```
import java.util.Arrays;
class Test
{
    public static void main(String[] args)
    {
        int[] arr1 = {5,1,3,2,4};
        int[] arr2 = {9,6,8,7,10};

        int size1 = arr1.length; // 5
        int size2 = arr2.length; // 5

        arr1 = Arrays.copyOf(arr1,size1+size2);

        int index=0;
        for(int i=size1;i<arr1.length;i++)
        {
            arr1[i] = arr2[index++];
        }

        Arrays.sort(arr1);

        //display
        for(int i : arr1)
        {
            System.out.print(i+" ");
        }
    }
}
```

Q) Write a java program to insert given element on a given index of an array?

input:

arr = 7 9 6 1 2 8
ele = 10
index = 2

output:

7 9 10 6 1 2 8

```
import java.util.Arrays;
class Test
```

```

{
    public static void main(String[] args)
    {
        int[] arr = {7,9,6,1,2,8};
        int ele = 10;
        int index = 2;

        arr = Arrays.copyOf(arr,arr.length+1);

        for(int i=arr.length-1;i>=index;i--)
        {
            arr[i]=arr[i-1];
        }
        arr[index]=ele;

        //display
        for(int i : arr)
        {
            System.out.print(i+" ");
        }
    }
}

```

Q) Write a java program to delete first occurrence of a given element in positive integer array?

input:
 arr = 7 2 9 2 4 8 2 6
 ele = 2
 output:
 7 9 2 4 8 2 6

```

import java.util.Arrays;
class Test
{
    public static void main(String[] args)
    {
        int[] arr = {7,2,9,2,4,8,2,6};
        int ele = 2;

        int index = findIndex(arr,ele);

        if(index!=-1)
        {
            for(int i=index;i<arr.length-1;i++)
            {
                arr[i] = arr[i+1];
            }
        }
    }
}

```

```

        arr = Arrays.copyOf(arr,arr.length-1);

        //print elements
        for(int i : arr)
        {
            System.out.print(i+" ");
        }
    }
    else
    {
        System.out.println("Element Not found");
    }
}

public static int findIndex(int[] arr,int ele)
{
    for(int i=0;i<arr.length;i++)
    {
        if(arr[i]==ele)
        {
            return i;
        }
    }
    return -1;
}
}

```

Q) Prime Numbers in an Array

You are given an array of integers, `arr`, of size `array length`. If a prime number which is contained in `arr` can be written as `a+b`, where `a` and `b` are also both contained somewhere in `arr` then it is called a special prime number. It is given that `a` and `b` may be the same element.

Find the index of the first occurrence of a special prime number in `arr`. If a special prime number does not exist in `arr`, return `-1`.

Input:

8 2 4 3 7

Output:

4

```

class Test
{
    public static void main(String[] args)
    {
        int[] arr= {8,2,4,3,7};
        System.out.println(specialPrime(arr));
    }
}

```

```

    }
    public static int specialPrime(int[] arr)
    {
        for(int i=0;i<arr.length;i++)
        {
            int value = arr[i];

            if(isPrime(value))
            {
                for(int j=0;j<arr.length;j++)
                {
                    for(int k=0;k<arr.length;k++)
                    {
                        if(arr[j]+arr[k]==value)
                        {
                            return i;
                        }
                    }
                }
            }
        }
        return -1;
    }
    public static boolean isPrime(int n)
    {
        boolean flag=true;
        for(int i=2;i<=n/2;i++)
        {
            if(n%i==0)
            {
                flag=false;
                break;
            }
        }
        return flag;
    }
}

```

Assignments =====

Q) Write a java program to shift all zero's to right side?

input:

5 0 1 0 9 0 6

output:

5 1 9 6 0 0 0

Q) Write a java program to reverse the array without using another array?

input:

6 9 1 4 2 7

output:

7 2 4 1 9 6

Double Dimensional Array

=====

Double dimensional array is a combination of rows and columns.

Double dimensional array is implemented based on array of arrays approach but not in matrix form.

Double dimensional array is used to develop business oriented applications, gaming applications, matrix type of applications and etc.

We can declare and create double dimensional array as follow.

syntax:

```
datatype[][] variable_name = new datatype[rows][cols];
```

ex:

```
int[][] arr = new int[3][3];
```

Here we can store 9 elements.

We can initialize double dimensional array as follow.

ex:

```
int[][] arr = {{1,2,3},{4,5,6},{7,8,9}};
```

Q) Write a java program to accept array elements and display them in matrix form?

```
import java.util.Scanner;
```

```
class Test
```

```
{
```

```
    public static void main(String[] args)
```

```
    {
```

```
        Scanner sc = new Scanner(System.in);
```

```
        System.out.println("Enter the rows :");
```

```
        int rows = sc.nextInt();//3
```

```
        System.out.println("Enter the cols :");
```

```
        int cols = sc.nextInt();//3
```

```
        int[][] arr = new int[rows][cols];
```

```
        //insert elements
```

```

        for(int i=0;i<rows;i++)
        {
            for(int j=0;j<cols;j++)
            {
                System.out.println("Enter the element :");
                arr[i][j] = sc.nextInt();
            }
        }

        //display elements
        for(int i=0;i<rows;i++)
        {
            for(int j=0;j<cols;j++)
            {
                System.out.print(arr[i][j]+" ");
            }
            //new line
            System.out.println();
        }
    }
}

```

Q) Write a java program to display double dimensional array elements?

input:

```

1 2 3
4 5 6
7 8 9

```

Approach1

```

class Test
{
    public static void main(String[] args)
    {
        int[][] arr = {{1,2,3},{4,5,6},{7,8,9}};

        int rows = arr.length;
        int cols = arr[0].length;

        for(int i=0;i<rows;i++)
        {
            for(int j=0;j<cols;j++)
            {
                System.out.print(arr[i][j]+" ");
            }
            //new line
            System.out.println();
        }
    }
}

```


Approach2

```
class Test
{
    public static void main(String[] args)
    {
        int[][] arr = {{1,2,3},{4,5,6},{7,8,9}};

        for(int[] result : arr)
        {
            for(int i : result)
            {
                System.out.print(i+" ");
            }
            //new line
            System.out.println();
        }
    }
}
```

Q) Write a java program to find out square of a matrix?

input:

```
1 2 3
4 5 6
7 8 9
```

output:

```
1 4 9
16 25 36
49 64 81
```

```
class Test
{
    public static void main(String[] args)
    {
        int[][] arr = {{1,2,3},{4,5,6},{7,8,9}};

        for(int[] result : arr)
        {
            for(int i : result)
            {
                System.out.print(i*i+" ");
            }
            //new line
            System.out.println();
        }
    }
}
```

Q) Write a java program to find out sum of diagonal elements?

```
input:
  1 2 3
  4 5 6
  7 8 9
```

```
output:
  15
```

```
class Test
{
    public static void main(String[] args)
    {
        int[][] arr = {{1,2,3},{4,5,6},{7,8,9}};

        int rows = arr.length;
        int cols = arr[0].length;

        int sum=0;
        for(int i=0;i<rows;i++)
        {
            for(int j=0;j<cols;j++)
            {
                if(i==j)
                {
                    sum += arr[i][j];
                }
            }
        }
        System.out.println(sum);
    }
}
```

Q) Write a java program to find out sum of upper triangle elements?

```
input:
  1 2 3
  4 5 6
  7 8 9
```

```
output:
  11
```

```
class Test
{
    public static void main(String[] args)
    {
        int[][] arr = {{1,2,3},{4,5,6},{7,8,9}};

        int rows = arr.length;
        int cols = arr[0].length;
```

```

        int sum=0;
        for(int i=0;i<rows;i++)
        {
            for(int j=0;j<cols;j++)
            {
                if(i<j)
                {
                    sum += arr[i][j];
                }
            }
        }
        System.out.println(sum);
    }
}

```

Q) Write a java program to find out sum of lower triangle elements?

input:

```

1 2 3
4 5 6
7 8 9

```

output:

```

19

```

```

class Test
{
    public static void main(String[] args)
    {
        int[][] arr = {{1,2,3},{4,5,6},{7,8,9}};

        int rows = arr.length;
        int cols = arr[0].length;

        int sum=0;
        for(int i=0;i<rows;i++)
        {
            for(int j=0;j<cols;j++)
            {
                if(i>j)
                {
                    sum += arr[i][j];
                }
            }
        }
        System.out.println(sum);
    }
}

```

Q) Write a java sort the array elements?

```
input:
    9 2 6
    1 8 5
    4 7 3
```

```
output:
    1 2 3
    4 5 6
    7 8 9
```

```
import java.util.Arrays;
class Test
{
    public static void main(String[] args)
    {
        int[][] arr = {{9,2,6},{1,8,5},{4,7,3}};

        int rows = arr.length;
        int cols = arr[0].length;

        int[] flatArr = new int[rows*cols];

        //convert double to single dimensional array
        int index=0;
        for(int i=0;i<rows;i++)
        {
            for(int j=0;j<cols;j++)
            {
                flatArr[index++] = arr[i][j];
            }
        }

        //sorting
        Arrays.sort(flatArr);

        //convert single to double dimensional array
        int idx=0;
        for(int i=0;i<rows;i++)
        {
            for(int j=0;j<cols;j++)
            {
                arr[i][j] = flatArr[idx++];
                System.out.print(arr[i][j]+" ");
            }
            //new line
            System.out.println();
        }
    }
}
```

Q) Write a java program to find out transpose of a matrix?

input:

```
1 2 3
4 5 6
7 8 9
```

output:

```
1 4 7
2 5 8
3 6 9
```

```
class Test
```

```
{
    public static void main(String[] args)
    {
        int[][] arr = {{1,2,3},{4,5,6},{7,8,9}};

        int rows = arr.length;
        int cols = arr[0].length;

        int[][] transpose =new int[rows][cols];

        for(int i=0;i<rows;i++)
        {
            for(int j=0;j<cols;j++)
            {
                transpose[j][i] = arr[i][j];
            }
        }

        //display
        for(int[] result : transpose)
        {
            for(int i : result)
            {
                System.out.print(i+" ");
            }
            //new line
            System.out.println();
        }
    }
}
```

Q) Write a java program to display array elements in spiral form?

input:

```
1 2 3
4 5 6
7 8 9
```

output:

```
1 2 3 6 9 8 7 4 5
```

```

class Test
{
    public static void main(String[] args)
    {
        int[][] matrix = {{1,2,3},{4,5,6},{7,8,9}};

        int left = 0;
        int right = matrix.length-1;
        int top = 0;
        int bottom = matrix[0].length-1;

        while(true)
        {
            if(left>right)
            {
                break;
            }
            for(int i=left;i<=right;i++)
            {
                System.out.print(matrix[top][i]+" ");
            }
            top++;

            if(top>bottom)
            {
                break;
            }
            for(int i=top;i<=bottom;i++)
            {
                System.out.print(matrix[i][right]+" ");
            }
            right--;

            if(left>right)
            {
                break;
            }
            for(int i=right;i>=left;i--)
            {
                System.out.print(matrix[bottom][i]+" ");
            }
            bottom--;

            if(top>bottom)
            {
                break;
            }
            for(int i=bottom;i>=top;i--)
            {
                System.out.print(matrix[i][left]+" ");
            }
        }
    }
}

```

```

        left++;
    }
}

```

Jagged Array

=====

Jagged array is also known as array of arrays.

It is a multiple dimensional array where each row having different column size.

ex:

```

class Test
{
    public static void main(String[] args)
    {
        int[][] arr = {
                                {1,2,3,4},
                                {5,6},
                                {7,8,9}
                            };

        for(int i=0;i<arr.length;i++)
        {
            for(int j=0;j<arr[i].length;j++)
            {
                System.out.print(arr[i][j]+" ");
            }
            //new line
            System.out.println();
        }
    }
}

```

Anonymous Array

=====

Sometimes we will declare an array without name such type of nameless array is called anonymous array.

The main objective of anonymous array is just for instance use.

We can declare an anonymous array as follow.

ex:

```

new int[]{10,20,30};
new int[][]{{1,2,3},{4,5,6}};

```

ex:

```

class Test
{
    public static void main(String[] args)
    {
        System.out.println(sumOfElements(new int[]{10,20,30}));
    }
    public static int sumOfElements(int[] arr)
    {
        int sum=0;
        for(int i : arr)
        {
            sum+=i;
        }
        return sum;
    }
}

```

Multi Dimensional Array

=====

If array contains more than two dimensions is called multi dimensional array.

Multi dimensional array is used to store the data in structured, layered and tabular format to establish the relationship between arrays.

We can declare and create multiple dimensional array as follow.

ex:

```

        base index
        |
int[][][] arr = int[2][3][3];
        | |
        rows cols

```

Here we can store 18 elements.

We can initialize multi dimensional array as follow.

ex:

```

int[][][] arr = {
    {
        {1,2,3},
        {4,5,6},
        {7,8,9}
    },
    {
        {1,2,3},
        {4,5,6},
        {7,8,9}
    }
};

```



```

ex:
---
class Test
{
    public static void main(String[] args)
    {
        int[][][] arr = {
                                {{1,2,3},{4,5,6},{7,8,9}},
                                {{1,2,3},{4,5,6},{7,8,9}}
                            };

        for(int i=0;i<arr.length;i++)
        {
            for(int j=0;j<arr[i].length;j++)
            {
                for(int k=0;k<arr[i][j].length;k++)
                {
                    System.out.print(arr[i][j][k]+" ");
                }
                //new line
                System.out.println();
            }
            //new line
            System.out.println();
        }
    }
}

```

OOPS

=====

OOPS stands for Object Oriented Programming System/Structure.

OOPS concept introduced to deal with real world entities using programming language.

A language is said to be object oriented if it supports following features.

ex:

```

class
object
Abstraction
Encapsulation
Inheritance &
Polymorphism

```

```

class
=====

```

A class is a blue print of an object.

ex:

Design or Template

It is a logical entity.

It is a collection of objects.

A class will accept following modifiers.

ex:

default
public
final
abstract
sealed
non-sealed

Q) What is the difference between default class and public class?

default class

public class

To declare default class we should not
should use public
use any keyword.

To declare public class we

keyword.

ex:

ex:

class A
{

}

public class A
{

}

If we declare any class as default
public then

If we declare any class as

then we can access that class within
the package
the package.

we can access that class within
and outside the package.

object

=====

It is a outcome of a blue print.

It is a instance of a class. Here instance means allocating memory for
our data members.

It is a physical entity.

It is a collection of properties and behaviours.

ex:

Dog
|
|-----|

Properties	Behaviours
> Name	> Eating
> Color	> Barking
> Breed	> Sleeping
> Height	and etc.
> Age	
and etc.	

It is possible to create more than one object is a single class.

We can declare object as follow.

syntax:

```
class_name reference_variable = new constructor();
```

ex:

```
Test t = new Test();
```

ex:

```
class Test
```

```
{
```

```
    public static void main(String[] args)
```

```
    {
```

```
        Test t1 = new Test();
```

```
        Test t2 = new Test();
```

```
        Test t3 = new Test();
```

```
        System.out.println(t1.hashCode());
```

```
        System.out.println(t2.hashCode());
```

```
        System.out.println(t3.hashCode());
```

```
        System.out.println(t1); // Test@hexadecimalvalue
```

```
        System.out.println(t2.toString());
```

```
        System.out.println(t3.toString());
```

```
    }
```

```
}
```

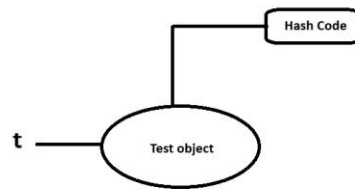
Q) What is hashcode in java?

For every object , JVM will create a unique identification number i.e hash code.

In order to read hash code of an object we need to use hashCode() method of Object class.

Diagram: class31.1

Test t = new Test();



Q) What is toString() method in java?

Whenever we are trying to display any object reference directly or indirectly toString() method will be executed.

A toString() method present in Object class.

ex:

```
class Test
{
    public static void main(String[] args)
    {
        int[] arr = new int[3];

        System.out.println(arr);
        System.out.println(arr.toString()); // [I@hexadecimalvalue
    }
}
```

ex:

```
class Test
{
    public static void main(String[] args)
    {
        Test t = new Test();

        System.out.println(t); // Test@Hexadecimalvalue
        System.out.println(t.toString()); // Test@Hexadecimalvalue
    }
}
```

ex:

--

```
class Test
{
    public static void main(String[] args)
```

```

    {
        Test t = new Test();
        System.out.println(t);
    }
    @Override
    public String toString()
    {
        return "Hello World";
    }
}

```

Q) What is the difference between class and object?

class	object
-----	-----
To declare a class we will use class keyword.	To declare an object we will use new keyword.
It is a blue print of an object.	It is instance of a class.
It is a logical entity.	It is a physical entity.
It is a collection of objects. properties and	It is a collection of behaviours.
It can't be modified.	It can be modified.
It can't be redeclared.	It can be redeclared.

Data Hiding
=====

The process of hiding object data from unauthorized access is called data hiding.

Using private modifier we can implements data hiding concept.

The main objective of data hiding is to provide security.

ex:

```

class Account
{
    private double balance = 10000d;
}
class Student
{
    public static void main(String[] args)
    {

```

```

        Account acc = new Account();
        System.out.println(acc.balance);
    }
}

```

We can access private data in other classes by using setter and getter methods.

Abstraction

=====

Hiding internal implementation and highlighting the set of services is called abstraction.

Using abstract classes and interfaces we can implements abstraction.

The best example of abstraction is GUI ATM machine where bank people hides internal implementation and highlights the set of services like banking, withdrawl, mini statement and etc.

The main advantages of abstraction are

- 1) It gives security because it hides internal implementation.
- 2) Enhacement becomes more easy without effecting enduser they can perform any changes in our internal system.
- 3) It provides flexibility to the enduser to use the system.
- 4) It improves maintain ability of an application.

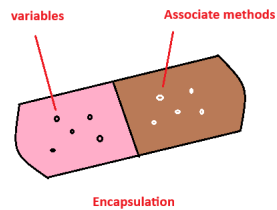
Encapsulation

=====

The process of encapsulating or grouping or wrapping variables and it's associate methods in a single entity is called encapsulation.

The best example of encapsulation is capsule.

Diagram: class31.2



Abstraction is used to hide the data and encapsulation is used to protect the data.

A class is said to be encapsulated class if it supports data hiding and abstraction.

In encapsulation for every variable we need to declare setter and getter methods.

The main advantages of encapsulation are

- 1) It gives security.
- 2) Enhancement becomes more easy.
- 3) It provides flexibility to the enduser.
- 4) It improves maintainability of an application.

The main disadvantage of encapsulation is , it will increase the length of our code and slow down the execution process.

ex:

```
class Student
{
    //current class variables
    private int studId;
    private String studName;
    private double studFee;

    //setter methods
    public void setStudId(int studId)
    {
        this.studId=studId;
    }
    public void setStudName(String studName)
    {
        this.studName=studName;
    }
}
```

```

        public void setStudFee(double studFee)
        {
            this.studFee=studFee;
        }

        //getter methods
        public int getStudId()
        {
            return studId;
        }
        public String getStudName()
        {
            return studName;
        }
        public double getStudFee()
        {
            return studFee;
        }
    }
    class Test
    {
        public static void main(String[] args)
        {
            Student s = new Student();
            s.setStudId(101);
            s.setStudName("Alan");
            s.setStudFee(1000d);

            System.out.println("Student Id : "+s.getStudId());
            System.out.println("Student Name : "+s.getStudName());
            System.out.println("Student Fee : "+s.getStudFee());
        }
    }

```

Q) What is the difference between POJO and JAVA Bean class?

POJO	Java Bean
-----	-----
It can't be serialized.	It can be serialized.
Fields can have any visibility.	Fields can have private visibility.
It may or may not have constructor.	It must have atleast zero argument constructor.
It does not extend outside classes.	It extends outside classes.
It does not implement outside interfaces.	It implements outside interfaces.

It does not allow outside annotations. It allows outside annotations.

Q) What is tightly encapsulated classes in java?

A class is said to be tightly encapsulated class if and only if all the variables of that class must be declared as private.

In this case, we don't need to check these variables contain setter and getter methods.

ex:

```
class A
{
    int i=10;
}
// It is not a tightly encapsulated class
```

ex:

```
class A
{
    private int i;
    int j;
}
// It is not a tightly encapsulated class
```

ex:

```
class A
{
    int i=10;
}
class B extends A
{
    private j=20;
}
// It is not a tightly encapsulated class
```

Q) What is difference between Abstraction and Encapsulation?

Abstraction

1. Hiding internal implementation and highlighting the set of services is called abstraction.

2. Best example is ATM machine.

Encapsulation

1. The process of encapsulating or wrapping variables and its associated methods in a single entity is called encapsulation.

2. Best example is capsule.

3.Using abstract classes and interfaces
we can implement abstraction.

It is used to hide the data.

It is a process of gaining the
the
information.

It solves the issue at design level.
implementation level.

3.Using access modifiers we
can implement
encapsulation.

It is used to protect the data.

It is a process of containing
information.

It solves the issue at

Is-A relationship

=====

Is-A relationship is also known as inheritance.

Using extends keyword we can implements Is-A relationship.

The main objective of Is-A relationship is to provide reusability.

ex:

```
class Parent
{
    public void methodOne()
    {
        System.out.println("Method-One");
    }
}
class Child extends Parent
{
    public void methodTwo()
    {
        System.out.println("Method-Two");
    }
}
class Test
{
    public static void main(String[] args)
    {
        Parent p = new Parent();
        p.methodOne();

        Child c = new Child();
        c.methodOne();
        c.methodTwo();

        Parent p1 = new Child();
```

```

        p1.methodOne();

        Child c1 = new Parent(); // C.T.E
    }
}

```

conclusion

Whatever properties are there with parent by default it comes to child. But whatever properties are there with child it not comes back to parent.

A parent reference can hold child object but child reference can't hold parent object.

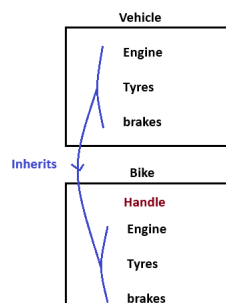
Inheritance
=====

It is a core concept of object oriented programming language which allows us to derived a class in the presence of existing class.

It is a mechanism in which one class inherits the properties of another class.

The main objective of inheritance is to promote code reusability.

Diagram: class32.1



In java, we have five types of inheritance.

- 1) Single Level Inheritance
- 2) Multi Level Inheritance
- 3) Multiple Inheritance
- 4) Hierarchical Inheritance
- 5) Hybrid Inheritance

1) Single Level Inheritance

If we derived a class in the presence of one base class is called single level inheritance.

diagram:

```
      A (parent / base / super class)
      |
      |
      B (child / derived / sub class)
```

ex:

```
---
class A
{
    public void m1()
    {
        System.out.println("M1-Method");
    }
}
class B extends A
{
    public void m2()
    {
        System.out.println("M2-Method");
    }
}
class Test
{
    public static void main(String[] args)
    {
        A a = new A();
        a.m1();

        B b = new B();
        b.m1();
        b.m2();
    }
}
```

2) Multi Level Inheritance

If we derived a class in the presence of one base class and that class is derived from another base class is called multi level inheritance.

Diagram:

```
      A
      |
      |
      B
      |
```

|
C

```
class A
{
    public void m1()
    {
        System.out.println("M1-Method");
    }
}
class B extends A
{
    public void m2()
    {
        System.out.println("M2-Method");
    }
}
class C extends B
{
    public void m3()
    {
        System.out.println("M3-Method");
    }
}
class Test
{
    public static void main(String[] args)
    {
        A a = new A();
        a.m1();

        B b = new B();
        b.m1();
        b.m2();

        C c = new C();
        c.m1();
        c.m2();
        c.m3();
    }
}
```

3) Multiple Inheritance

In java, we can't extends more then one class simultenously because java does not support multiple inheritance.

ex:

```
class A
{
}
class B
```

```

{
}
class C extends A,B --> invalid
{
}

```

But one interface can extends more then one interface so we can achieve multiple inheritance concept through interfaces.

ex:

```

interface A
{
}
interface B
{
}
interface C extends A,B
{
}

```

If our class does not extends any other class then it is a direct child class of Object class.

ex:

```

diag:
class A      Object
{            |
}            |
            A

```

If our class extends some other class then it is a indirect child class of Object class.

ex:

```

diag:
class A      Object
{            |
}            A
class B      |
{            |
}            B

```

Java does not support cyclic inheritance.

ex:

```

class A extends B
{
}
class B extends A
{
}

```

Q) Why java does not support multiple inheritance?

There is a chance of raising ambiguity problem that's why java does not support multiple inheritance.

ex:

```

P1.m1()          P2.m1()
|-----|

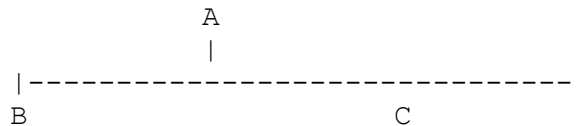
```

```
|  
C.m1()
```

4) Hierarchical Inheritance

If we derived multiple classes using one base class is called hierarchical inheritance.

diag:



ex:

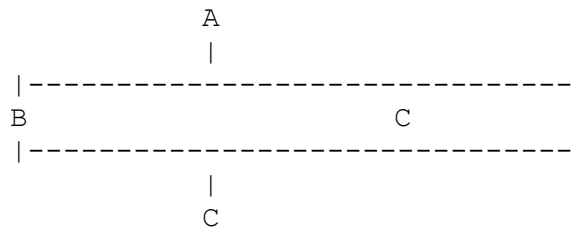
```
---  
class A  
{  
    public void m1()  
    {  
        System.out.println("M1-Method");  
    }  
}  
class B extends A  
{  
    public void m2()  
    {  
        System.out.println("M2-Method");  
    }  
}  
class C extends A  
{  
    public void m3()  
    {  
        System.out.println("M3-Method");  
    }  
}  
class Test  
{  
    public static void main(String[] args)  
    {  
        A a = new A();  
        a.m1();  
  
        B b = new B();  
        b.m1();  
        b.m2();  
  
        C c = new C();  
        c.m1();  
        c.m3();  
    }  
}
```

```
    }  
}
```

5) Hybrid Inheritance

It is a collection of more than two inheritance.
Java does not support hybrid inheritance.

diag:



Constructors

=====

A constructor is a special method which used to initialize an object.

Having same name as class name is called constructor.

A constructor is called when instance of a class is created.

A constructor accepts following modifiers.

ex:

```
    default  
    public  
    private  
    protected
```

A constructor does not allow return type.

In java, constructors are divided into two types.

1) User defined constructor

2) Default constructor

1) User defined constructor

A constructor which is created by the user based on the application requirements is called user defined constructor.

It is classified into two types.

i) Zero Argument constructor

ii) Parameterized constructor

i) Zero Argument constructor

Suppose if we are not passing any argument to user defined constructor then that constructor is called zero argument constructor.

ex:

```
class Test
{
    Test()
    {
        System.out.println("0-arg const");
    }
    public static void main(String[] args)
    {
        System.out.println("main-method");
    }
}
```

o/p:

main-method

ex:

```
class Test
{
    public Test()
    {
        System.out.println("0-arg const");
    }
    public static void main(String[] args)
    {
        System.out.println("main-method");
        Test t = new Test();
    }
}
```

o/p:

main-method

0-arg const

ex:

```
class Test
{
    private Test()
    {
        System.out.println("0-arg const");
    }
    public static void main(String[] args)
    {
        Test t1 = new Test();
        System.out.println("main-method");
    }
}
```

```

        Test t2 = new Test();
    }
}
o/p:
    0-arg const
    main-method
    0-arg const

ex:
---
class Test
{
    protected Test()
    {
        System.out.println("0-arg const");
    }
    public static void main(String[] args)
    {
        Test t1 = new Test();
        System.out.println("main-method");
        Test t2 = new Test();
    }
}

o/p:
    0-arg const
    main-method
    0-arg const

```

ii) Parameterized constructor

Suppose if we are passing any argument to userdefined constructor then that constructor is called parameterized constructor.

```

ex:
---
class Employee
{
    private int empId;
    private String empName;
    private double empSal;

    public Employee(int empId,String empName,double empSal)
    {
        this.empId=empId;
        this.empName=empName;
        this.empSal=empSal;
    }
}

```

```

    }

    public void getEmployeeDetails()
    {
        System.out.println("Employee Id : "+empId);
        System.out.println("Employee Name : "+empName);
        System.out.println("Employee Salary : "+empSal);
    }
}
class Test
{
    public static void main(String[] args)
    {
        Employee e = new Employee(101,"Alan Morries",1000d);
        e.getEmployeeDetails();
    }
}

```

2) Default constructor

It is a compiler generated constructor for every java program where we are not defining atleast zero argument constructor.

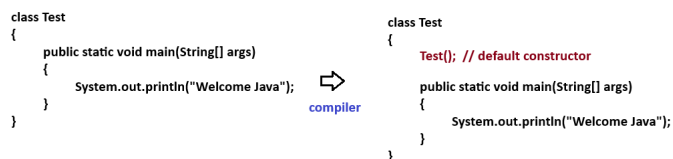
To see the default constructor we need to use below command.

ex:

```
javac Test.java
```

```
javap -c Test
```

Diagram: class33.1



Q) What is constructor overloading ?

Having same constructor name with different arguments or signatures in a single class is called constructor overloading.

ex:

```

class A
{
    A()
    {
        System.out.println("0-arg const");
    }
    A(int i)
    {
        System.out.println("int-arg const");
    }
    A(double d)
    {
        System.out.println("double-arg const");
    }
}
class Test
{
    public static void main(String[] args)
    {
        A a1 = new A();
        A a2 = new A(10);
        A a3 = new A(10.5d);
    }
}

```

Has-A relationship

=====

Has-A is also known as composition and aggregation.

There is no specific keyword to implements Has-A relationship but mostly we will use new operator.

The main objective of Has-A relationship is to provide reusability.

Has-A relationship will increase dependency between two components.

ex:

```

class QualityThought
{
    public String courseName()
    {
        return "Java with Gen AI";
    }
    public double courseFee()
    {
        return 27000d;
    }
    public String trainerName()
    {
        return "Niyaz Sir";
    }
}

```

```

    }
}
class Chandrika
{
    public void courseDetails()
    {
        QualityThought qt = new QualityThought();
        System.out.println("Course Name : "+qt.courseName());
        System.out.println("Course Fee : "+qt.courseFee());
        System.out.println("Trainer Name : "+qt.trainerName());
    }
}
class Aspirant
{
    public static void main(String[] args)
    {
        Chandrika c = new Chandrika();
        c.courseDetails();
    }
}

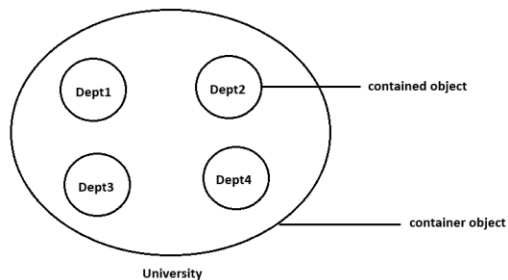
```

composition

=====

Without existing container object there is no chance of having contained object then the relationship between container and contained object is called composition which is strongly association.

Diagram: class33.2



```

class Engine
{
    public void engineStart()
    {
        System.out.println("Engine Started");
    }
}
class Car
{
    Engine e;
}

```

```

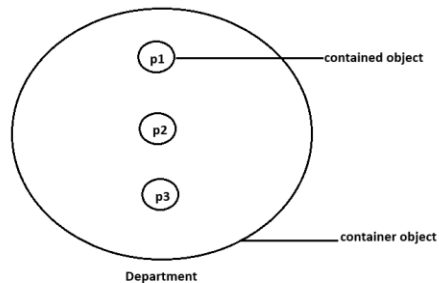
    public Car()
    {
        e = new Engine();
    }
    public void carMove()
    {
        e.engineStart();
        System.out.println("Car Moved");
    }
}
class Test
{
    public static void main(String[] args)
    {
        Car c = new Car();
        c.carMove();
    }
}

```

Aggregation =====

Without existing container object there is a chance of having contained object then the relationship between container and contained object is called aggregation which is loosely association.

Diagram: class33.3



ex:

```

class Engine
{
    public void engineStart()
    {
        System.out.println("Engine Started");
    }
}
class Car
{
    Engine e;
    public Car(Engine e)

```

```

        {
            this.e=e;
        }
        public void carMove()
        {
            e.engineStart();
            System.out.println("Car Moved");
        }
    }
}
class Test
{
    public static void main(String[] args)
    {
        Engine e = new Engine();
        Car c = new Car(e);
        c.carMove();
    }
}

```

Method Overloading

=====

Having same method name with different parameters or signatures in a single class is called method overloading.

Methods present in a class are called overloaded methods.

Method resolution will taken care by a compiler based on reference type.

Method overloading will reduce complexity of the programming.

ex:

```

class MeeSeva
{
    //overloaded methods
    public void search(int voterId)
    {
        System.out.println("Details Found via voterId");
    }
    public void search(String houseNo)
    {
        System.out.println("Details Found via houseNo");
    }
    public void search(long aadharNo)
    {
        System.out.println("Details Found via aadharNo");
    }
}
class Test
{
    public static void main(String[] args)
    {

```

```

        MeeSeva ms = new MeeSeva();
        ms.search(742);
        ms.search("1-7-65/2/1");
        ms.search(1234L);
    }
}

```

Q) Can we overload main method in java?

Yes, we can overload main method in java but JVM always execute main method with String[] parameter only.

ex:

```

class Test
{
    public static void main(String[] args)
    {
        System.out.println("String[] parameter");
    }
    public static void main(int[] iargs)
    {
        System.out.println("int[] parameter");
    }
}

```

Method Overriding

=====

Having same method name with same parameters in two different classes is called method overriding.

Methods which are present in parent class are called overridden methods.

Methods which are present in child class are called overriding methods.

Method resolution will taken care by a JVM based on runtime object.

ex:

```

class Parent
{
    //overridden methods
    public void property()
    {
        System.out.println("cash+gold+land");
    }
    public void marry()
    {
        System.out.println("Trisha");
    }
}

```



```

    }
}
class Child extends Parent
{
    //overriding methods
    public void marry()
    {
        System.out.println("Rashmika");
    }
}
class Test
{
    public static void main(String[] args)
    {
        Parent p = new Parent();
        p.property(); // cash+gold+land
        p.marry(); // Trisha

        Child c = new Child();
        c.property(); // cash+gold+land
        c.marry(); // Rashmika

        Parent p1 = new Child();
        p1.property(); // cash+gold+land
        p1.marry(); // Rashmika
    }
}

```

If we declare any method as final then overriding of that method is not possible.

ex:

```

class Parent
{
    public final void property()
    {
        System.out.println("House - Not For Sale");
    }
}
class Child extends Parent
{
    @Override
    public void property()
    {
        System.out.println("House - For Sale");
    }
}
class Test
{
    public static void main(String[] args)
    {

```

```

        Parent p = new Child();
        p.property();
    }
}
o/p:
    C.T.E : overridden method is final

```

Method Hiding

=====

Method hiding exactly same as method overriding with following differences.

Method overriding

Methods should be non-static.

Method resolution will taken care by
taken care by
JVM based on runtime object.
type.

It is also known as runtime polymorphism,
time
dynamic polymorphism or late binding.
polymorphism or

Method hiding

Methods should be static.

Method resolution will
Method resolution will
compiler based on reference
type.

It is also known as compile
time
polymorphism, static
polymorphism, static
early binding.

ex:

```

class Parent
{
    public static void property()
    {
        System.out.println("House - Not For Sale");
    }
}
class Child extends Parent
{
    public static void property()
    {
        System.out.println("House - For Sale");
    }
}
class Test
{
    public static void main(String[] args)
    {
        Parent p = new Child();
        p.property(); // House - Not For Sale
    }
}

```

Q) Can we override main method in java?

No, We can override main method in java because main method is static and static methods we can't override in java.

Polymorphism

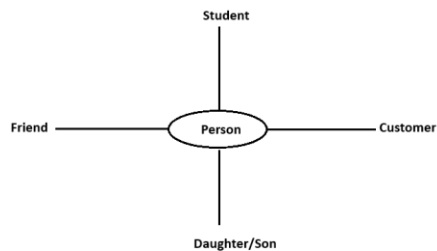
=====

Polymorphism has take from Greek word.

Poly means many and morphism means forms.

The ability to represent in a different forms is called polymorphism.

Diagram: class34.1



The main objective of polymorphism is to provide flexibility.

In java, polymorphism is divided into two types.

1) Compile time polymorphism / Static polymorphism / Early binding

2) Runtime polymorphism / Dynamic polymorphism / Late binding

1) Compile time polymorphism / Static polymorphism / Early binding

A polymorphism which exhibits at compile time is called compile time polymorphism.

Method resolution will taken care by a compiler based on reference type.

ex:

Method Overloading
Method Hiding

2) Runtime polymorphism / Dynamic polymorphism / Late binding

A polymorphism which exhibits at runtime is called runtime polymorphism.

Method resolution will taken care by a JVM based on runtime object.

ex:

Method Overriding

Note:

OOPS talks about security, reusability and flexibility.

this keyword

=====

A this keyword is a java keyword which is used to refer current class object reference.

We can utilize this keyword in following ways.

- 1) To refer current class variables
- 2) To refer current class methods
- 3) To refer current class constructors

- 1) To refer current class variables

```
class A
{
    int i=10;
    int j=20;
    A(int i,int j)
    {
        System.out.println(i+" "+j); // 100 200
        System.out.println(this.i+" "+this.j); // 10 20
    }
}
class Test
{
    public static void main(String[] args)
    {
        A a = new A(100,200);
    }
}
```

- 2) To refer current class methods

```
class A
```

```

{
    public void m1()
    {
        System.out.println("M1-Method");
        this.m2();
    }
    public void m2()
    {
        System.out.println("M2-Method");
    }
}
class Test
{
    public static void main(String[] args)
    {
        A a = new A();
        a.m1();
    }
}

```

3) To refer current class constructors

```

-----
class A
{
    A()
    {
        System.out.println("0-arg const");
    }
    A(int i)
    {
        this();
        System.out.println("int-arg const");
    }
    A(double d)
    {
        this(10);
        System.out.println("double-arg const");
    }
}
class Test
{
    public static void main(String[] args)
    {
        A a = new A(10.5d);
    }
}

```

super keyword

=====

A super keyword is a java keyword which is used to refer super class object reference.

We can utilize super keyword in following ways.

- 1) To refer super class variables
- 2) To refer super class methods
- 3) To refer super class constructors

1) To refer super class variables

```
-----  
class A  
{  
    int i=1;  
    int j=2;  
}  
class B extends A  
{  
    int i=10;  
    int j=20;  
    B(int i,int j)  
    {  
        System.out.println(this.i+" "+this.j); // 10  20  
        System.out.println(super.i+" "+super.j); // 1  2  
        System.out.println(i+" "+j); //  100  200  
    }  
}  
class Test  
{  
    public static void main(String[] args)  
    {  
        B b = new B(100,200);  
    }  
}
```

2) To refer super class methods

```
-----  
class A  
{  
    public void m1()  
    {  
        System.out.println("M1-Method");  
    }  
}  
class B extends A  
{  
    public void m2()  
    {  
        super.m1();  
        System.out.println("M2-Method");  
    }  
}
```

```

class Test
{
    public static void main(String[] args)
    {
        B b = new B();
        b.m2();
    }
}

```

3) To refer super class constructors

```

class A
{
    A()
    {
        System.out.println("A-const");
    }
}
class B extends A
{
    B()
    {
        super();
        System.out.println("B-const");
    }
}
class Test
{
    public static void main(String[] args)
    {
        B b = new B();
    }
}

```

Interview Question

=====

Q) Write a java program to shift all zero's to right side?

input:

2 0 5 0 1 0 4

output:

2 5 1 4 0 0 0

```

class Test
{
    public static void main(String[] args)
    {
        int[] arr = {2,0,5,0,1,0,4};
    }
}

```

```

        int pos=0;
        for(int i=0;i<arr.length;i++)
        {
            if(arr[i]!=0)
            {
                int temp = arr[i];
                arr[i]=arr[pos];
                arr[pos]=temp;

                pos++;
            }
        }
        for(int i: arr)
        {
            System.out.print(i+" ");
        }
    }
}

```

Sealed class
=====

A sealed class introduced in Java 17.

It provides a mechanism to restrict which other classes can't extend or implement a given class or interface.

sealed modifier:

The superclass or interface is declared with the sealed keyword.

permits clause:

The declaration must include a permits clause to explicitly list the direct subclasses or subinterfaces that are allowed to extend or implement it.

Every permitted subclass must be declared with one of three modifiers to define how the hierarchy continues:

final:

The class cannot be extended any further (record classes are implicitly final).

sealed:

The class can be extended further, but only by its own specific permitted subclasses.

non-sealed:

The class breaks the sealing hierarchy and can be extended by any unknown class (reverts to traditional open inheritance).

ex:

```
sealed class A permits B
{
    public void m1()
    {
        System.out.println("M1-Method");
    }
}
// hierarchical ends
final class B extends A
{
    public void m2()
    {
        System.out.println("M2-Method");
    }
}
class Test
{
    public static void main(String[] args)
    {
        A a = new A();
        a.m1();

        B b = new B();
        b.m1();
        b.m2();
    }
}
```

ex:

```
sealed class A permits B
{
    public void m1()
    {
        System.out.println("M1-Method");
    }
}
// class can extends by any class
non-sealed class B extends A
{
    public void m2()
    {
        System.out.println("M2-Method");
    }
}
class C extends B
```

```

{
    public void m3()
    {
        System.out.println("M3-Method");
    }
}
class Test
{
    public static void main(String[] args)
    {
        A a = new A();
        a.m1();

        B b = new B();
        b.m1();
        b.m2();

        C c = new C();
        c.m1();
        c.m2();
        c.m3();
    }
}

ex:
---
sealed class A permits B
{
    public void m1()
    {
        System.out.println("M1-Method");
    }
}
// class can extends by any class
sealed class B extends A
{
    public void m2()
    {
        System.out.println("M2-Method");
    }
}
final class C extends B
{
    public void m3()
    {
        System.out.println("M3-Method");
    }
}
class Test
{
    public static void main(String[] args)
    {

```

```

        A a = new A();
        a.m1();

        B b = new B();
        b.m1();
        b.m2();

        C c = new C();
        c.m1();
        c.m2();
        c.m3();
    }
}

```

Interfaces

=====

It is a blue print of a class and it is a collection of abstract methods, default methods, static methods and private methods.

Abstract methods are incomplete methods because they ends with semicolon and do not have any body.

ex:

```
void methodOne();
```

It is not possible to create object for interfaces.

To write the implementation of abstract methods we will use implementation class.

It is possible to create object for implementation class because it contains method with body.

By default every abstract method is a public and abstract.

ex:

```
public abstract void methodOne();
```

Interface contains only constants i.e public static final.

syntax:

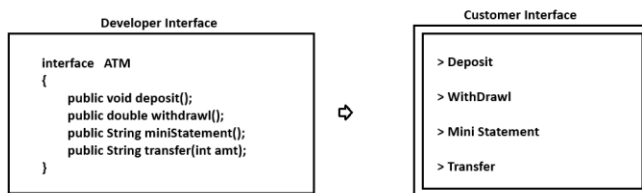
```

interface <interface_name>
{
    -
    - // abstract methods
    - // default methods
    - // static methods
    - // private methods
    - // constants
    -
}

```

If we know service requirement specification then we need to use interface.

Diagram: class35.1



```
ex:
----
interface A
{
    public abstract void m1();
}
class B implements A
{
    @Override
    public void m1()
    {
        System.out.println("M1-Method");
    }
}
class Test
{
    public static void main(String[] args)
    {
        A a = new B();
        a.m1();
    }
}
```

```
ex:
---
interface A
{
    public abstract void m1();
}

class Test
{
    public static void main(String[] args)
```

```

    {
        //Anonymous inner class
        A a = new A()
        {
            public void m1()
            {
                System.out.println("From M1-Method");
            }
        };
        a.m1();
    }
}

```

If interface contains four methods then we need to override all methods.

```

ex:
---
interface A
{
    public abstract void see();
    public void show();
    abstract void display();
    void view();
}
class B implements A
{
    @Override
    public void see()
    {
        System.out.println("See-Method");
    }
    @Override
    public void show()
    {
        System.out.println("Show-Method");
    }
    @Override
    public void display()
    {
        System.out.println("Display-Method");
    }
    @Override
    public void view()
    {
        System.out.println("View-Method");
    }
}
class Test
{
    public static void main(String[] args)
    {

```

```

        A a = new B();
        a.see();
        a.show();
        a.display();
        a.view();
    }
}

```

A class can't extend more than one class simultaneously.
But interface can extend more than one interface simultaneously.

```

ex:
---
interface A
{
    void m1();
}
interface B
{
    void m2();
}
interface C extends A,B
{
    void m3();
}
class D implements C
{
    @Override
    public void m1()
    {
        System.out.println("M1-Method");
    }
    @Override
    public void m2()
    {
        System.out.println("M2-Method");
    }
    @Override
    public void m3()
    {
        System.out.println("M3-Method");
    }
}
class Test
{
    public static void main(String[] args)
    {
        C c = new D();
        c.m1();
        c.m2();
        c.m3();
    }
}

```

```
    }  
}
```

A class can implements more then one interface.

ex:

```
interface Father  
{  
    float HT=6.2f;  
    void height();  
}  
interface Mother  
{  
    float HT=5.8f;  
    void height();  
}  
class Child implements Father,Mother  
{  
    @Override  
    public void height()  
    {  
        float height = (Father.HT+Mother.HT)/2;  
        System.out.println("Child Height :"+height);  
    }  
}  
class Test  
{  
    public static void main(String[] args)  
    {  
        Child c = new Child();  
        c.height();  
    }  
}
```

Q) How to achieve multiple inheritance in java?

We can achieve multiple inheritance via interfaces but not with classes.

```
interface Dog  
{  
    void eat();  
}  
interface Cat  
{  
    void sleep();  
}  
class Animal implements Dog,Cat  
{  
    @Override
```

```

        public void eat()
        {
            System.out.println("Eat Method");
        }
        @Override
        public void sleep()
        {
            System.out.println("Sleep Method");
        }
    }
}
class Test
{
    public static void main(String[] args)
    {
        Animal a = new Animal();
        a.eat();
        a.sleep();
    }
}

```

Marker Interface

=====

Interface which does not contain any methods or constants is called marker interface.

In general, empty interface is called marker interface.

We have following list of marker interfaces.

ex:

```

    Serializable
    Cloneable
    Remote
    and etc.

```

Using marker interface we will get some ability to do by JVM.

Abstract classes

=====

It is a collection of zero or more abstract methods and concrete methods.

A abstract keyword is applicable for methods and classes but not for variables.

It is not possible to create object for abstract class.

To write the implementation of abstract methods we will use sub classes.

Abstract class contains only instance variables.

syntax:

```
abstract class <class_name>
{
    -
    - //abstract methods
    - //concrete methods
    - //instance variables
    -
}
```

Note:

If we know partial implementation then we need to use abstract class.

ex:

```
abstract class Plan
{
    //instance variable
    protected double rate;

    //abstract method
    public abstract void getRate();

    //concrete method
    public void calculateBillAmt(int units)
    {
        System.out.println("Total Units :"+units);
        System.out.println("Total Bill :"+(rate*units));
    }
}

class DomesticPlan extends Plan
{
    @Override
    public void getRate()
    {
        rate = 2.5d;
    }
}

class CommercialPlan extends Plan
{
    @Override
    public void getRate()
    {
        rate = 5.0d;
    }
}

class Test
{
    public static void main(String[] args)
    {
        DomesticPlan dp = new DomesticPlan();
    }
}
```

```

        dp.getRate();
        dp.calculateBillAmt(250);

        CommercialPlan cp = new CommercialPlan();
        cp.getRate();
        cp.calculateBillAmt(250);
    }
}

```

ex:

```

---
abstract class Vehicle
{
    //abstract method
    public abstract void sound();
}
class Bike extends Vehicle
{
    @Override
    public void sound()
    {
        System.out.println("Boor Boor");
    }
}
class Car extends Vehicle
{
    @Override
    public void sound()
    {
        System.out.println("Foor Foor");
    }
}
class Test
{
    public static void main(String[] args)
    {
        Bike b = new Bike();
        b.sound();

        Car c = new Car();
        c.sound();
    }
}

```

Abstraction Example
 =====

Approach1

```

abstract class Shapes
{
    //abstract method
    public abstract void draw();
}
class Circle extends Shapes
{
    @Override
    public void draw()
    {
        System.out.println("Circle");
    }
}
class Rectangle extends Shapes
{
    @Override
    public void draw()
    {
        System.out.println("Rectangle");
    }
}
class Test
{
    public static void main(String[] args)
    {
        Circle c = new Circle();
        c.draw();

        Rectangle r = new Rectangle();
        r.draw();
    }
}

```

Approach2

```

interface Payment
{
    //abstract method
    public abstract void pay();
}
class CardPayment implements Payment
{
    @Override
    public void pay()
    {
        System.out.println("Card Payment");
    }
}
class Test
{
    public static void main(String[] args)
    {

```

```

        Payment p =new CardPayment();
        p.pay();
    }
}

```

Q) What are the difference between interface and abstract class?

interface	abstract class
-----	-----
To declare interface we will use interface keyword.	To declare abstract class we will use abstract keyword.
It is a blue print of a class and more abstract	It is a collection of zero or more methods and concrete methods.
it is a collection of abstract methods, default methods, static methods and private methods.	

We can achieve multiple inheritance. We can't achieve multiple inheritance.

To write the implementation for abstract methods we will use implementation class.	To write the implementation for methods we will use sub classes.
--	--

If we know specification then we implementation then need to use interface.	If we know partial we need to use abstract class.
---	---

It does not allow constructor.	It allows constructors.
--------------------------------	-------------------------

It does not allow blocks.	It allows blocks.
---------------------------	-------------------

Packages
=====

A package is a collection of classes, interfaces, enums and annotations.

Here enum is a special class and annotation is a special interface.

A package is also known as a folder or a directory.

In java, packages are divided into two types.

- 1) Predefined packages
- 2) Userdefined packages

1) Predefined packages

Built-In packages are called predefined packages.

ex:

```
java.lang (default)
java.time
java.util
java.text
java.util.stream
java.io
java.sql
javax.servlet
and etc.
```

2) Userdefined packages

Packages which are created by the user based on the application requirements are called userdefined packages.

syntax:

```
package <package_name>;
```

It is also recommended to declare package name in reverse order of url.

ex:

```
package com.ihub.www;
```

ex:

```
package com.ihub.www;
import java.util.Calendar;
class Test
{
    public static void main(String[] args)
    {
        Calendar c = Calendar.getInstance();
        int h = c.get(Calendar.HOUR_OF_DAY);
        if(h<12)
            System.out.println("Good Morning");
        else if(h<16)
            System.out.println("Good Afternoon");
        else if(h<20)
            System.out.println("Good Evening");
        else
            System.out.println("Good Night");
    }
}
```

We can compile above program by using below command.

ex:

```
current directory
```

```

    |
javac -d . Test.java
    |
destination
folder

```

We can execute above program by using below command.

ex:

```

java    com.ihub.www.Test
    |        |
package-name    class-name

```

Enum

=====

Enum concept introduced in Java 1.5.

Enum is a group of named constants.

Using Enum we can create our own datatype called enumerated datatype.

When compare to old language enum, java enum is more powerful.

We can declare enum as follow

syntax:

```

enum    type_name
{
    value1,value2,...,valueN
}

```

ex:

```

enum    Months
{
    JAN,FEB,MAR
}

```

Internal implementation of enum

Enum internally consider as class concept.

Every enum constant is a reference variable of enum type.

ex:

```

enum Months                                public final class Months extends
java.lang.Enum                             java.lang.Enum
{
    JAN,FEB,MAR                            =>    public static final Months JAN=new
Months();                                Months();

```

```

    }
    Months();
    public static final Months FEB=new
    Months();
    public static final Months MAR=new
    Months();
    }

```

Declaration and Usage of enum

```

enum Months
{
    JAN, FEB, MAR
}
class Test
{
    public static void main(String[] args)
    {
        Months m = Months.JAN;
        System.out.println(m);
    }
}

```

ex:

```

enum Months
{
    JAN, FEB, MAR
}
class Test
{
    public static void main(String[] args)
    {
        Months m = Months.FEB;
        switch(m)
        {
            case JAN : System.out.println("January"); break;
            case FEB : System.out.println("February"); break;
            case MAR : System.out.println("March"); break;
        }
    }
}

```

Note:

Every enum is a final so we can't create child enum.

ex:

```

enum A
{
}
enum B extends A
{
}

```

Every enum extends java.lang.Enum class so we can't extends some other class or enum.

ex:

```
class A
{
}
enum B extends A
{
}
```

We can conclude that inheritance concept is not applicable for enum.

java.lang.Enum

The power to enum will be inherited from java.lang.Enum class.

It contains following two methods.

1) values()

It is a static method which returns group of constants.

2) ordinal()

It returns ordinal numbers.

ex:

```
enum Drinks
{
    COLA, PEPSI, CAMPA
}
class Test
{
    public static void main(String[] args)
    {
        Drinks[] d = Drinks.values();

        for(Drinks d1 : d)
        {
            System.out.println(d1+" ----- "+d1.ordinal());
        }
    }
}
```

When compare to old language enum , java enum is more powerful because in addition to constants we can declare variables, methods and constructors.


```

ex:
---
enum Week
{
    MON, TUE, WED, THU, FRI, SAT, SUN;

    Week()
    {
        System.out.println("constructor");
    }
}
class Test
{
    public static void main(String[] args)
    {
        Week w = Week.MON;
    }
}

```

```

ex:
---
enum Week
{
    MON, TUE, WED, THU, FRI, SAT, SUN;

    static int i =10;

    public static void main(String[] args)
    {
        System.out.println(i);
    }
}

```

Wrapper classes

=====

The main objective of wrapper classes are

- 1) To wrap primitive to wrapper object and vice versa.
- 2) To defined several utility methods.

```

ex:

```

Primitive type	Wrapper object
-----	-----
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double
boolean	Boolean

char Character

constructor

Every wrapper class contains two constructors. One will take corresponding primitive as an argument and another will take corresponding string as an argument.

ex:

Wrapper class	Constructor
-----	-----
Byte	byte or String
Short	short or String
Integer	int or String
Long	long or String
Float	float or String
Double	double or String
Boolean	boolean or String
Character	char

ex:

```
class Test
{
    public static void main(String[] args)
    {
        Integer i1 = Integer.valueOf(10);
        System.out.println(i1);

        Integer i2 = Integer.valueOf("20");
        System.out.println(i2);
    }
}
```

ex:

```
class Test
{
    public static void main(String[] args)
    {
        Boolean b1 = Boolean.valueOf(true);
        System.out.println(b1); // true

        Boolean b2 = Boolean.valueOf("false");
        System.out.println(b2); // false
    }
}
```

ex:

```
class Test
{
```

```

        public static void main(String[] args)
        {
            Character c = Character.valueOf('a');
            System.out.println(c);
        }
    }

```

Utility Methods
=====

1) valueOf()

It is used to convert primitive type or String type to Wrapper object.

ex:

```

class Test
{
    public static void main(String[] args)
    {
        //primitive type
        int a=10;

        //wrapper object
        Integer i = Integer.valueOf(a);

        System.out.println(i);
    }
}

```

ex:

--

```

class Test
{
    public static void main(String[] args)
    {
        //String type
        String s="20";

        //wrapper object
        Integer i = Integer.valueOf(s);

        System.out.println(i);
    }
}

```

2) xxxValue()

It is used to convert wrapper object to primitive type.

ex:

```

class Test

```

```

{
    public static void main(String[] args)
    {
        //wrapper object
        Integer i = Integer.valueOf(10);

        //primitive type
        byte b = i.byteValue();
        System.out.println(b);

        short s = i.shortValue();
        System.out.println(s);

        int a = i.intValue();
        System.out.println(a);
    }
}

```

3) parseXxx()

It is used to convert string type to primitive type.

ex:

```

class Test
{
    public static void main(String[] args)
    {
        //string type
        String s="100";

        //primitive type
        int i = Integer.parseInt(s);
        System.out.println(i);//100

        long l = Long.parseLong(s);
        System.out.println(l);//100

        float f = Float.parseFloat(s);
        System.out.println(f);//100.0
    }
}

```

4) toString()

It is used to convert wrapper object or primitive type to string type.

ex:

```

class Test
{
    public static void main(String[] args)

```

```

    {
        //wrapper object
        Integer i = Integer.valueOf(10);

        //string type
        String s = i.toString();
        System.out.println(s);
    }
}

ex:
---
class Test
{
    public static void main(String[] args)
    {
        //primitive type
        int a=100;

        //string type
        String s = Integer.toString(a);

        System.out.println(s);
    }
}

```

Q) Write a java program to perform sum of two binary numbers?

input:

1010

0101

output:

1111

```

import java.util.Scanner;
class Test
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);

        System.out.println("Enter the first binary number :");
        String binary1 = sc.next();//1010

        System.out.println("Enter the second binary number :");
        String binary2 = sc.next();//0101

        //convert binary to decimal
        int a = Integer.parseInt(binary1,2);
        int b = Integer.parseInt(binary2,2);
    }
}

```

```

        int c = a + b;

        //convert decimal to binary
        String result = Integer.toBinaryString(c);
        System.out.println(result);
    }
}

```

Q) What is autoboxing and autounboxing in java?

Autoboxing

=====

Automatic conversion from primitive type to wrapper object performed by the compiler at the time of compilation is called autoboxing.

Autoboxing internally uses `valueOf()` method.

ex:

```

class Test
{
    public static void main(String[] args)
    {
        //primitive type
        int a = 10;

        //wrapper object
        Integer i = a;

        System.out.println(i);
    }
}

```

Autounboxing

Automatic conversion from wrapper object to primitive type performed by the compiler at the time of compilation is called autounboxing.

Autounboxing internally uses `xxxValue()` method.

ex:

```

class Test
{
    public static void main(String[] args)
    {
        //wrapper object
        Integer i = Integer.valueOf(10);

        //primitive
    }
}

```

```

        int a = i;

        System.out.println(a);
    }
}

```

Inner classes

=====

Sometimes we will declare a class inside another class such concept is called inner class.

Inner classes introduced in 1.1 version as a part of event handling to remove GUI bugs.

But due to powerful features and benefits of inner classes. Programmers started to use inner classes in regular programming.

In inner classes we can't declare static members.

We have four types of inner classes.

- 1) Normal/Regular inner class
- 2) Static inner class
- 3) Method local inner class
- 4) Anonymous inner class

1) Normal/Regular inner class

Sometimes we will declare a class inside another class such concept is called normal/regular inner class.

Here inner class must be within the scope of enclosing class.

ex:

```

class Outer
{
    class Inner
    {
        public void m1()
        {
            System.out.println("M1-Method");
        }
    }

    public static void main(String[] args)
    {
        Outer.Inner i = new Outer().new Inner();
    }
}

```

```

        i.m1();

        //or
        new Outer().new Inner().m1();
    }
}

```

If we compile above program we will get two .class files
i.e Outer.class and Outer\$Inner.class.

2) Static inner class

Sometimes we will declare a class inside another class using static keyword is called static inner class.

In regular or normal inner class, without container object there is no chance of having contained object.

ex:

--

```

class Outer
{
    static class Inner
    {
        public void m1()
        {
            System.out.println("M1-Method");
        }
    }

    public static void main(String[] args)
    {
        Inner i = new Inner();
        i.m1();
    }
}

```

3) Method local inner class

It is used to execute method specific repeated logic.

ex:

```

class Outer
{
    public void m1()
    {
        class Inner
        {
            public void sum(int a,int b)
            {
                System.out.println(a+b);
            }
        }
    }
}

```



```

        }
    }
    Inner i = new Inner();
    i.sum(1,2);
    i.sum(10,20);
    i.sum(100,200);
}

public static void main(String[] args)
{
    Outer o = new Outer();
    o.m1();
}
}

```

4) Anonymous inner class

Sometimes we will declare a class without name such type of nameless class is called anonymous class.

ex:

```

interface ATM
{
    public abstract void deposit();
}
class Test
{
    public static void main(String[] args)
    {
        //Anonymous inner class
        ATM atm = new ATM()
        {
            public void deposit()
            {
                System.out.println("Deposit Method");
            }
        };
        atm.deposit();
    }
}

```

ex:

```

abstract class Payment
{
    public abstract void paymentMethod();
}
class Test
{
    public static void main(String[] args)

```

```

    {
        //Anonymous inner class
        Payment p = new Payment()
        {
            public void paymentMethod()
            {
                System.out.println("UPI");
            }
        };
        p.paymentMethod();
    }
}

```

Cloning

=====

The process of creating exact duplicate object is called cloning.

We can perform cloning only for cloneable objects.

To create cloneable object our class must implements Cloneable interface.

A Cloneable is a marker interface which does not contain methods and constants.

By using Cloneable interface we will get some ability by JVM.

To perform cloning we will use clone() method of Object class.

ex:

```
protected native Object clone()throws CloneNotSupportedException
```

ex:

```
class Test implements Cloneable
```

```
{
```

```
    //instance variables
```

```
    int i=10;
```

```
    public static void main(String[] args)throws
CloneNotSupportedException
```

```
    {
```

```
        Test t1 = new Test();
```

```
        System.out.println(t1.i); // 10
```

```
        System.out.println(t1.hashCode()); //2060468723
```

```
        Test t2 = (Test)t1.clone();
```

```
        System.out.println(t2.i); // 10
```

```
        System.out.println(t2.hashCode()); //622488023
```

```
    }
```

```
}
```

Q) What is the difference between shallow cloning and deep cloning?

Shallow cloning

=====

The process of creating exact duplicate object reference is known as shallow cloning.

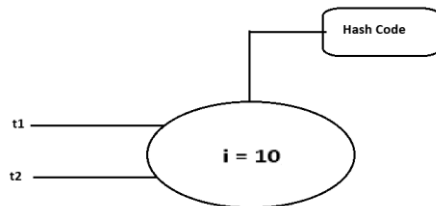
ex:

```
class Test
{
    //instance variables
    int i=10;

    public static void main(String[] args)
    {
        Test t1 = new Test();
        System.out.println(t1.i); // 10
        System.out.println(t1.hashCode()); // 2060468723

        Test t2 = t1;
        System.out.println(t2.i); // 10
        System.out.println(t2.hashCode()); // 2060468723
    }
}
```

Diagram: class37.1



Deep cloning

=====

The process of creating exact duplicate object is known as deep cloning.

ex:

```
class Test implements Cloneable
{
    //instance variables
    int i=10;

    public static void main(String[] args) throws
CloneNotSupportedException
```

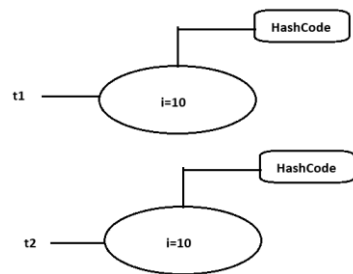
```

{
    Test t1 = new Test();
    System.out.println(t1.i); // 10
    System.out.println(t1.hashCode()); // 2060468723

    Test t2 = (Test)t1.clone();
    System.out.println(t2.i); // 10
    System.out.println(t2.hashCode()); // 622488023
}
}

```

Diagram: class37.2



Types of objects in Java

=====

We have two types of objects in java.

1) Immutable object

2) Mutable object

1) Immutable object

After object creation if we perform any changes then for every change a new object will be created such type of object is called immutable object.

ex:

String and Wrapper classes

2) Mutable object

After object creation if we perform any changes then all the required changes will be done in a same object such type of object is called mutable object.

ex:

StringBuffer and StringBuilder
String

=====

It is a collection of characters which is enclosed in a double quotation.

ex:

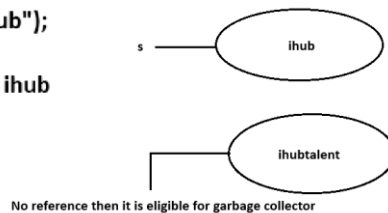
"hello"

case1:

Once if we create a String object we can't perform any changes. If we perform any changes then for every change a new object will be created such behaviour is called immutability of an object.

Diagram: class38.1

```
String s = new String("ihub");  
s.concat("talent");  
System.out.println(s); // ihub
```



case2:

Q) What is the difference between == and .equals() method?

==

It is used to reference comparison or address comparison. It is used to compare primitive types and object types.

ex:

```
class Test  
{  
    public static void main(String[] args)  
    {  
        String s1 = new String("ihub");  
        String s2 = new String("ihub");  
        System.out.println(s1==s2); // false  
    }  
}
```

.equals()

It is used for content comparison which is case sensitive. It is used to compare only object types.

ex:

--

```
class Test
```

```

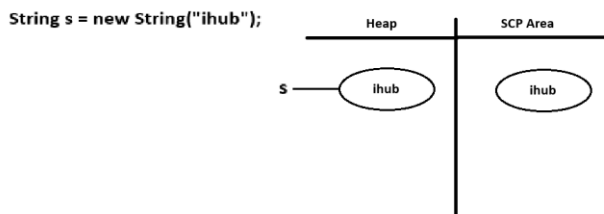
{
    public static void main(String[] args)
    {
        String s1 = new String("ihub");
        String s2 = new String("ihub");
        System.out.println(s1.equals(s2)); // true
    }
}

```

case3:

Once if we create a String object. Two objects will be created one is on heap and another is on SCP (String Constant Pool) area.

Diagram: class38.2

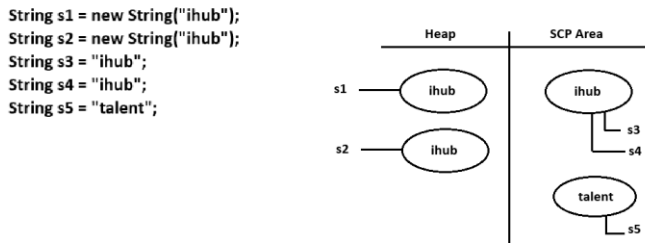


Object creations in SCP are is always optional. First JVM will check is there object is created with same content or not. If it is created then it refers to that object. If it is not created then it will create a new object. Hence there is no chance of having duplicate objects in SCP Area.

SCP objects do not have any reference , even though garbage collector can't access them.

SCP objects will destroy automatically whenever JVM shutdowns or terminated.

Diagram: class38.3



String Programs

=====

Q) Write a java program to find out length of the string?

input:
hello

output:
5

```
class Test
{
    public static void main(String[] args)
    {
        String str = "hello";
        int len = str.length();
        System.out.println(len);
    }
}
```

Q) Write a java program to convert given string to uppercase?

input:
ihubtalent
output:
IHUBTALENT

```
class Test
{
    public static void main(String[] args)
    {
        String str = "ihubtalent";
        str = str.toUpperCase();
        System.out.println(str);
    }
}
```

Q) Write a java program to convert given string to lowercase?

input:
IHUBTALENT
output:
ihubtalent

```

class Test
{
    public static void main(String[] args)
    {
        String str = "IHUBTALENT";
        str = str.toLowerCase();
        System.out.println(str);
    }
}

```

Q) Write a java program to concatenate two strings?

```

input:
    ihub
    talent
output:
    ihubtalent

```

```

class Test
{
    public static void main(String[] args)
    {
        String s1 = "ihub";
        String s2 = "talent";
        String result = s1.concat(s2);
        System.out.println(result);
    }
}

```

Q) Write a java program to check given strings are equal or not?

```

input:
    ihub
    ihub
output:
    Both are equals

```

```

class Test
{
    public static void main(String[] args)
    {
        String s1 = "ihub";
        String s2 = "ihub";
        if(s1.equals(s2))
            System.out.println("Both are equals");
        else
            System.out.println("Both are not equals");
    }
}

```


Q) Write a java program to check given strings are equal or not?

input:

ihub

IHUB

output:

Both are equals

```
class Test
{
    public static void main(String[] args)
    {
        String s1 = "ihub";
        String s2 = "IHUB";
        if(s1.equalsIgnoreCase(s2))
            System.out.println("Both are equals");
        else
            System.out.println("Both are not equals");
    }
}
```

Q) Write a java program to remove the spaces from before and after of the string?

input:

" ihub talent "

output:

ihub talent

ex:

```
class Test
{
    public static void main(String[] args)
    {
        String str=" ihub talent ";

        str = str.trim();

        System.out.println(str);
    }
}
```

Q) Write a java program to replace the string?

input:

abababa

output:
aba

```
class Test
{
    public static void main(String[] args)
    {
        String str="abababa";

        str = str.replace("aba","a");

        System.out.println(str);
    }
}
```

Q) Write a java program to check string contains alphabets or not?

input:
hello
output:
TRUE

```
class Test
{
    public static void main(String[] args)
    {
        String str="hello2";

        if(str.matches("[A-Za-z]+"))
            System.out.println("TRUE");
        else
            System.out.println("FALSE");
    }
}
```

Q) Write a java program to check string contains only digits?

input:
1234
output:
TRUE

```
class Test
{
    public static void main(String[] args)
    {
        String str="1234";

        if(str.matches("\\d+"))
```

```

        System.out.println("TRUE");
    else
        System.out.println("FALSE");
    }
}

```

Q) Write a java program to explain join() method?

ex:

--

```

class Test
{
    public static void main(String[] args)
    {
        String str="this is java class";
        String result = str.join(",",str.split(" "));
        System.out.println(result);
    }
}

```

ex:

```

class Test
{
    public static void main(String[] args)
    {
        String str="this is java class";
        String result = str.join("|",str.split(" "));
        System.out.println(result);
    }
}

```

ex:

--

```

class Test
{
    public static void main(String[] args)
    {
        String result =
String.join(",", "apple", "ball", "cat", "dog");
        System.out.println(result);
    }
}

```

Q) Write a java program to delete a word from given string?

input:

str = This is java class

word = is

output:

Th java class

```
class Test
{
    public static void main(String[] args)
    {
        String str = "This is java class";
        String word ="is";
        str = str.replace(word,"");
        System.out.println(str);
    }
}
```

Q) Write a java program to remove special characters from given string?

input:
I_h@ub%Ta#len*t
output:
IhubTalent

```
class Test
{
    public static void main(String[] args)
    {
        String str="I^h@ub%Ta#len*t";
        str = str.replaceAll("[^A-Za-z0-9]", "");
        System.out.println(str);
    }
}
```

Q) Write a java program to remove spaces from given string?

input:
"i hub tal ent"
output:
ihubtalent

```
ex:
---
class Test
{
    public static void main(String[] args)
    {
        String str="i hub tal ent";
        str = str.replaceAll("\\s", "");
        System.out.println(str);
    }
}
```

Q) Write a java program to concatenate two string?

input:

ihub12
talent28

output:

ihubtalent40

```
class Test
{
    public static void main(String[] args)
    {
        String s1 = "ihub12";
        String s2 = "talent28";

        String word1 = s1.replaceAll("[^A-Za-z]", "");
        int num1 = Integer.parseInt(s1.replaceAll("[^0-9]", ""));

        String word2 = s2.replaceAll("[^A-Za-z]", "");
        int num2 = Integer.parseInt(s2.replaceAll("[^0-9]", ""));

        String word = word1+word2;
        int num = num1+num2;

        System.out.println(word+num);
    }
}
```

Q) Write a java program to find out number of digits, sum of digits and check that number is even or odd?

input:

NEX123

output:

Number of digits : 3
Sum of digits : 6
It is even number

```
class Test
{
    public static void main(String[] args)
    {
        String str="NEX123";
        String newStr = str.replaceAll("[^0-9]", "");

        System.out.println("Number of Digits :"+newStr.length());

        int sum=0;
        for(int i=0;i<newStr.length();i++)
```

```

        {
            int n = Character.getNumericValue(newStr.charAt(i));
            sum+=n;
        }
        System.out.println("Sum of Digits :"+sum);
        if(sum%2==0)
            System.out.println("It is even number");
        else
            System.out.println("It is odd number");
    }
}

```

Q) Write a java program to perform right rotation of a given string?

input:

```

    str = ihubtalent
    cnt = 2

```

output:

```

    ubtalentih

```

ex:

```

class Test

```

```

{
    public static void main(String[] args)
    {
        String str ="ihubtalent";
        int cnt = 2;

        String word1 = str.substring(cnt);
        String word2 = str.substring(0,cnt);

        str = word1+word2;

        System.out.println(str);
    }
}

```

Q) Write a java program to display extension of a file?

input:

```

    abc.txt

```

output:

```

    txt

```

```

class Test

```

```

{
    public static void main(String[] args)
    {
        String str ="abc.txt";
    }
}

```

```

        int index = str.indexOf('.');

        String result = str.substring(index+1);

        System.out.println(result);
    }
}

```

Q) Write a java program to display reverse of a string?

input:
hello

output:
olleh

```

class Test
{
    public static void main(String[] args)
    {
        String str ="hello";

        String rev="";

        //reading reverse
        for(int i=str.length()-1;i>=0;i--)
        {
            rev+=str.charAt(i);
        }

        System.out.println(rev);
    }
}

```

Q) Write a java program to find out given string is palindrome or not?

input:
racar

output:
It is a palindrome string

```

class Test
{
    public static void main(String[] args)
    {
        String str ="racar";
    }
}

```

```

        String rev="";

        //reading reverse
        for(int i=str.length()-1;i>=0;i--)
        {
            rev+=str.charAt(i);
        }

        if(str.equals(rev))
            System.out.println("It is a palindrome string");
        else
            System.out.println("It is not a palindrome string");
    }
}

```

Q) Write a java program to display reverse of a sentence?

input:

This is java class

output:

class java is This

```

class Test
{
    public static void main(String[] args)
    {
        String str ="This is java class";
        String[] sarr = str.split(" ");

        //reading reverse
        for(int i=sarr.length-1;i>=0;i--)
        {
            System.out.print(sarr[i]+" ");
        }
    }
}

```

Q) Write a java program to display reverse of a word in a sentence?

input:

This is java class

output:

sihT si avaj ssalc

```

class Test
{

```



```

public static void main(String[] args)
{
    String str ="This is java class";
    String[] sarr = str.split(" ");

    for(String s : sarr)    // This
    {
        //reading reverse
        for(int i=s.length()-1;i>=0;i--)
        {
            System.out.print(s.charAt(i));
        }
        //space
        System.out.print(" ");
    }
}

```

Q) Write a java program to sort the string?

input:
 dacbe
output:
 abcde

```

import java.util.Arrays;
class Test
{
    public static void main(String[] args)
    {
        String str ="dacbe";

        char[] carr = str.toCharArray();

        Arrays.sort(carr);

        str = new String(carr);

        System.out.println(str);
    }
}

```

Q) Write a java program to find out maximum integer value from given string?

input:
 1kg of mango is 100 rupees and 2kg of apple is 200 rupees
output:

200

```
class Test
{
    public static void main(String[] args)
    {
        String str ="1kg of mango is 100 rupees and 2kg of apple is
200 rupees";

        String[] sarr = str.split("\\D+");

        int max = Integer.MIN_VALUE;

        for(String s : sarr)
        {
            int n = Integer.parseInt(s);
            if(n>max)
            {
                max=n;
            }
        }
        System.out.println(max);
    }
}
```

Q) Write a java program to find out the string starting with uppercase letter?

input:

This is Java class For students

output:

This Java For

```
class Test
{
    public static void main(String[] args)
    {
        String str ="This is Java class For students";

        String[] sarr = str.split(" ");

        for(String s : sarr)
        {
            if(s.charAt(0)>='A' && s.charAt(0)<='Z')
            {
                System.out.print(s+" ");
            }
        }
    }
}
```

```
}
```

Q) Write a java program to display palindrome string?

input:

madam is racar for students

output:

madam racar

```
class Test
{
    public static void main(String[] args)
    {
        String str ="madam is racar for students";

        String[] sarr = str.split(" ");

        for(String s : sarr)
        {
            String rev="";

            //reading rerverse
            for(int i=s.length()-1;i>=0;i--)
            {
                rev+=s.charAt(i);
            }
            if(s.equals(rev))
                System.out.print(s+" ");
        }
    }
}
```

Q) Jack purchased a new laptop for his office work. His son started learning how to frame the words using alphabets. Jack wants to write a program to find out longest word created by his son.

input:

apple cat banana elephant dark

output:

elephant

```
class Test
{
    public static void main(String[] args)
    {
```

```

String str ="apple cat banana elephant dark";

String[] sarr = str.split(" ");

int max = sarr[0].length();
String word =sarr[0];

for(String s : sarr)
{
    if(s.length()>max)
    {
        max = s.length();
        word = s;
    }
}
System.out.println(word);
}
}

```

Q) Write a java program to find convert uppercase to lowercase and vice versa?

input:
IhUbTAlenT
output:
iHuBtaLEnT

```

class Test
{
    public static void main(String[] args)
    {
        String str ="IhUbTAlenT";

        char[] carr = str.toCharArray();

        for(int i=0;i<carr.length;i++)
        {
            if(carr[i]>='A' && carr[i]<='Z')
            {
                carr[i] = (char) (carr[i]+32);
            }
            else if(carr[i]>='a' && carr[i]<='z')
            {
                carr[i] = (char) (carr[i]-32);
            }
        }
        str = new String(carr);
        System.out.println(str);
    }
}

```

```
}
```

Q) Write a java program to display the string in a given format?

input:

XYZ

output:

XY

XZ

YX

YZ

ZX

ZY

```
class Test
```

```
{
```

```
    public static void main(String[] args)
```

```
    {
```

```
        String str ="XYZ";
```

```
        for(int i=0;i<str.length();i++)
```

```
        {
```

```
            for(int j=0;j<str.length();j++)
```

```
            {
```

```
                if(i!=j)
```

```
                {
```

```
                    System.out.println(str.charAt(i)+" "+str.charAt(j));
```

```
                }
```

```
            }
```

```
        }
```

```
    }
```

```
}
```

Interview Question

=====

Q) Write a java program to display the string in a given format?

input:

Ho2w A3re You4 lHello

output:

Hello How Are You

Q) Write a java program to display the string in a given format?

Input:

Ho2w You4 Hlello Ar3e

Output:

Hello How Are You

```
class Test
{
    public static void main(String[] args)
    {
        String str="Ho2w You4 Hlello Ar3e";

        String[] words = str.split(" ");

        String[] result = new String[words.length];

        for(String word : words)
        {
            String text="";
            int position=0;

            for(char ch : word.toCharArray()) // A r 3 e
            {
                if(Character.isDigit(ch))
                {
                    position = ch-'0';
                }
                else
                {
                    text+=ch;
                }
            }
            result[position-1]=text;
        }

        //display the data
        for(String s : result)
        {
            System.out.print(s+" ");
        }
    }
}
```

Q) Write a java program to display given string is anagram or not?

input:

silent
listen

output:

It is a anagram string

```

import java.util.Arrays;
class Test
{
    public static void main(String[] args)
    {
        String str1 = "silent";
        String str2 = "listen";

        char[] carr1 = str1.toCharArray();
        char[] carr2 = str2.toCharArray();

        Arrays.sort(carr1); // e i l n s t
        Arrays.sort(carr2); // e i l n s t

        str1 = new String(carr1); //eilnst
        str2 = new String(carr2); //eilnst

        if(str1.equals(str2))
            System.out.println("It is a Anagram string");
        else
            System.out.println("It is not a Anagram string");
    }
}

```

Q) Write a java program to display most repeating character from given string?

input:
ihubtalentinstitute

output:
t repeating for 5 times

```

class Test
{
    public static void main(String[] args)
    {
        String str="ihubtalentinstitute";

        char character=' ';
        int maxCount = 0;

        for(int i=0;i<str.length();i++)
        {
            int cnt = 0;

            for(int j=0;j<str.length();j++)
            {
                if(str.charAt(i) == str.charAt(j))

```

```

        {
            cnt++;
        }
    }
    if(cnt>maxCount)
    {
        maxCount = cnt;
        character = str.charAt(i);
    }
    System.out.println(character+" repeating for "+maxCount+"
times");
}
}

```

Q) Write a java program to display unique and duplicates characters from given string?

input:

messessippie

output:

Unique characters : mesip

Duplicate characters : sepi

```

class Test
{
    public static void main(String[] args)
    {
        String str="messessippie";

        String duplicates="";
        String unique="";

        for(int i=0;i<str.length();i++)
        {
            String current = Character.toString(str.charAt(i));
            if(unique.contains(current))
            {
                if(!duplicates.contains(current))
                {
                    duplicates+=current;
                    continue;
                }
            }
            else
            {
                continue;
            }
        }
        unique+=str.charAt(i);
    }
}

```



```

        System.out.println("Unique characters :"+unique);
        System.out.println("Duplicate characters :"+duplicates);
    }
}

```

Q) Write a java program to encode the string ?

```

input:
    1106
output:
    AAJF

```

```

class Test
{
    public static void main(String[] args)
    {
        String str="1106";

        for(int i=0;i<str.length();i++)
        {
            int n = Character.getNumericValue(str.charAt(i));

            if(n>0)
            {
                System.out.print((char) ('A'+n-1));
            }
            else
            {
                int k = Integer.parseInt(str.substring(i-1,i+1));
                System.out.print((char) ('A'+k-1));
            }
        }
    }
}

```

Q) Write a java program to decode the string ?

```

input:
    AAJF
output:
    1106

```

```

class Test
{
    public static void main(String[] args)
    {

```

```

String str="AAJF";

for(int i=0;i<str.length();i++)
{
    char ch = str.charAt(i);

    int n = ch - 'A' + 1;

    if(Integer.toString(n).contains("0"))
    {
        System.out.print(n%10);
    }
    else
    {
        System.out.print(n);
    }
}
}

```

Q) Write a java program to insert in word in a given index of a string?

```

input:
    str = JavaStudents
    index = 4
    word = for
output:
    JavaforStudents

```

```

class Test
{
    public static void main(String[] args)
    {
        String str ="JavaStudents";
        int index = 4;
        String word = "for";

        String word1 = str.substring(0,index);
        String word2 = str.substring(index);

        str = word1+word+word2;
        System.out.println(str);
    }
}

```

Q) Write a java program to display the string in a given format?

input:
A1B2C3D4
output:
ABBCCCDDDD

```
class Test
{
    public static void main(String[] args)
    {
        String str ="A1B2C3D4";

        for(int i=0;i<str.length();i++)
        {
            if(Character.isAlphabetic(str.charAt(i)))
            {
                System.out.print(str.charAt(i));
            }
            else
            {
                int n = Character.getNumericValue(str.charAt(i));
                for(int k=1;k<n;k++)
                {
                    System.out.print(str.charAt(i-1));
                }
            }
        }
    }
}
```

Q) Write a java program to display the string in a given format?

input:
ABBCCCDDDD
output:
A1B2C3D4

```
class Test
{
    public static void main(String[] args)
    {
        String str ="ABBCCCDDDD";

        String result="";

        int count =1;

        for(int i=0;i<str.length();i++)
        {
            if(i<str.length()-1 && str.charAt(i)==str.charAt(i+1))
            {
```

```

        count++;
    }
    else
    {
        result+=str.charAt(i);
        result+=count;
        count = 1;
    }
}
System.out.println(result);
}
}

```

Assignment =====

Q) Write a java program to display permutation of a given string?

Input:
ABC

Output:
ABC
ACB
BAC
BCA
CBA
CAB

Q) Write a java program to display permutation of a given string?

input:
ABC

output:
ABC
ACB
BAC
BCA
CBA
CAB

```

class Test
{
    public static void main(String[] args)
    {
        String str="ABC";
        isPermutation(str.toCharArray(),0);
    }
    public static void isPermutation(char[] carr,int fi)
    {
        if(fi==carr.length-1)

```

```

        {
            System.out.println(carr);
            return;
        }
        for(int i=fi;i<carr.length;i++)
        {
            swap(carr,fi,i);
            isPermutation(carr,fi+1);
            swap(carr,fi,i);
        }
    }
    public static void swap(char[] carr,int fi,int i)
    {
        char temp = carr[fi];
        carr[fi] = carr[i];
        carr[i] = temp;
    }
}

```

StringBuffer
=====

If our content change frequently then it is not recommended to use String object because for every change a new object will be created.

To overcome this limitation Sun Micro System introduced in StringBuffer concept.

In StringBuffer all the required changes will be done in a same object.

constructors

1) StringBuffer sb = new StringBuffer()

It creates empty StringBuffer object with default initial capacity of 16.

If capacity reaches to maximum capacity then new capacity will be created with below formula.

ex:
newcapacity = current_capacity+1*2;

ex:

```

class Test
{
    public static void main(String[] args)
    {
        StringBuffer sb = new StringBuffer();
        System.out.println(sb.capacity()); // 16

        sb.append("abcdefghijklmnop");
    }
}

```

```

        System.out.println(sb.capacity()); // 16

        sb.append("qr");
        System.out.println(sb.capacity()); // 16+1*2=34
    }
}

```

2) `StringBuffer sb = new StringBuffer(int initialCapacity)`

It creates `StringBuffer` object with specified initial capacity.

ex:

```

class Test
{
    public static void main(String[] args)
    {
        StringBuffer sb = new StringBuffer(19);
        System.out.println(sb.capacity()); // 19
    }
}

```

3) `StringBuffer sb = new StringBuffer(String s)`

It creates `StringBuffer` object equivalent to `String`.

Here capacity will be created with below formula.

ex:

```
capacity = s.length() + 16;
```

ex:

```

class Test
{
    public static void main(String[] args)
    {
        StringBuffer sb = new StringBuffer("ihub");
        System.out.println(sb.capacity()); // 4+16=20
    }
}

```

Q) Write a java program to display reverse of a string?

input:

hello

output:

olleh

```

class Test
{

```

```

    public static void main(String[] args)
    {
        String str="hello";

        StringBuffer sb = new StringBuffer(str);

        String rev = sb.reverse().toString();

        System.out.println(rev);
    }
}

```

Q) Write a java program to check given string is palindrome or not?

input:
 racar
 output:
 It is a palindrome string

```

class Test
{
    public static void main(String[] args)
    {
        String str="madam";

        StringBuffer sb = new StringBuffer(str);

        String rev = sb.reverse().toString();

        if(str.equals(rev))
            System.out.println("It is a palindrome string");
        else
            System.out.println("It is not a palindrome string");
    }
}

```

Q) Write a java program to multiply two arrays?

input:
 7 3 2
 5 1
 output:
 37332 (732*51)

```

class Test
{
    public static void main(String[] args)

```

```

    {
        int[] arr1 = {7,3,2};
        int[] arr2 = {5,1};

        int a = Integer.parseInt(arrayToString(arr1));
        int b = Integer.parseInt(arrayToString(arr2));
        System.out.println(a*b);
    }

    public static String arrayToString(int[] arr)
    {
        StringBuffer sb = new StringBuffer();
        for(int i : arr)
        {
            sb.append(i);
        }
        return sb.toString();
    }
}

```

Q) Write a java program to display number of 2's present in a given number?

input:

22

output:

6 (2,12,20,21,22)

```

class Test
{
    public static void main(String[] args)
    {
        int n=22;

        StringBuffer sb =new StringBuffer();

        for(int i=1;i<=n;i++)
        {
            sb.append(i);
        }

        int cnt = 0;
        for(int i=0;i<sb.length();i++)
        {
            if(sb.charAt(i) == '2')
            {
                cnt++;
            }
        }
        System.out.println(cnt);
    }
}

```


StringBuilder

=====

StringBuilder exactly same as StringBuffer with following differences.

StringBuffer

Methods are synchronized.

StringBuilder

Methods are not synchronized.

At a time only one thread is allowed to operate

Multiple threads are allowed to operate on StringBuffer object. on StringBuilder object. Hence we can't achieve

Hence we can achieve thread safety. thread safety.

Waiting time of a thread will increase effectively

There is no waiting threads effectively performance is low. performance is high.

It is introduced in 1.0 version. It is introduced in 1.5 version.

Q) Write a java program to display reverse of a given number?

input:

123

output:

321

```
class Test
```

```
{
```

```
    public static void main(String[] args)
```

```
    {
```

```
        int n=123;
```

```
        //convert number to string
```

```
        String str = Integer.toString(n);
```

```
        StringBuilder sb = new StringBuilder(str);
```

```
        str = sb.reverse().toString(); // 321
```

```
        //convert string to number
```

```
        n = Integer.parseInt(str);
```

```
        System.out.println(n);
```

```
    }
```

```
}
```

Note:

If our content not change frequently then it is recommended to use String.

If our content change frequently where thread safety is required then we need to use StringBuffer.

If our content change frequently where thread safety is not required then we need to use StringBuilder.

StringTokenizer

=====

StringTokenizer class present in java.util package.

StringTokenizer class is used to tokenize the string based on delimiters.

We can create StringTokenizer object as follow.

ex:

```
StringTokenizer st = new StringTokenizer(String  
str, RegularExpression regex);
```

StringTokenizer class contains following methods.

ex:

```
public boolean hasMoreTokens()  
public String nextToken()  
public boolean hasMoreElements()  
public Object nextElement()  
public int countTokens()
```

ex:

--

```
import java.util.StringTokenizer;  
class Test  
{  
    public static void main(String[] args)  
    {  
        StringTokenizer st = new StringTokenizer("this is java  
class", " ");  
        System.out.println(st.countTokens());  
    }  
}
```

ex:

```
import java.util.StringTokenizer;  
class Test  
{  
    public static void main(String[] args)  
    {
```

```

        StringTokenizer st = new StringTokenizer("this is java
class", " ");
        while(st.hasMoreTokens())
        {
            String s = st.nextToken();
            System.out.println(s);
        }
    }
}

```

ex:

```

--
import java.util.StringTokenizer;
class Test
{
    public static void main(String[] args)
    {
        StringTokenizer st = new StringTokenizer("this is java
class", " ");
        while(st.hasMoreElements())
        {
            String s = (String)st.nextElement();
            System.out.println(s);
        }
    }
}

```

ex:

```

---
import java.util.StringTokenizer;
class Test
{
    public static void main(String[] args)
    {
        StringTokenizer st = new StringTokenizer("9,99,999",",");
        while(st.hasMoreElements())
        {
            String s = (String)st.nextElement();
            System.out.println(s);
        }
    }
}

```

Exception Handling

=====

Q) What is the difference between Exception and Error?

Exception

Exception is a problem for which we can provide solution programmatically.

Exception will raise due to syntax errors.

ex:

```
ArithmeticException
IllegalArgumentException
FileNotFoundException
```

Error

Error is a problem for which we can't provide solution programmatically.

Error will raise due to lack of system resources.

ex:

```
OutOfMemoryError
StackOverflowError
```

As a part of java application development it is a responsibility of a programmer to provide smooth termination for every java program.

We have two types of terminations.

1) Smooth termination/Graceful termination

During the program execution suppose if we are not getting any interruption in the middle of the program such type of termination is called smooth termination.

ex:

```
class Test
{
    public static void main(String[] args)
    {
        System.out.println("Hello World");
    }
}
```

2) Abnormal termination

During the program execution suppose if we are getting any interruption in the middle of the program such type of termination is called abnormal termination.

ex:

```
class Test
{
    public static void main(String[] args)
    {
        System.out.println(10/0);
    }
}
```

If any exception raised in our program we must and should handle that exception otherwise our program will terminates abnormally.

Here exception will display name of the exception, description of the exception and line number of the exception.

Exception

=====

It is a unwanted, unexpected event which disturbs normal flow of a program.

Exceptions always raise at runtime so they are also known as runtime events.

The main objective of exception handling is to provide graceful termination.

In java, exceptions are divided into two types.

1) Predefined exceptions

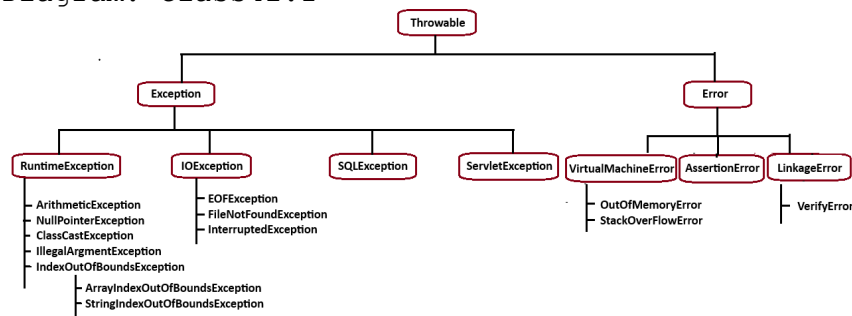
2) Userdefined exceptions

1) Predefined exceptions

Built-In exceptions are called predefined exceptions.

It is classified into two types.

Diagram: class42.1



i) Checked exceptions

Exceptions which are checked by the compiler at the time of compilation is called checked exceptions.

ex:

IOException
FileNotFoundException
EOFException

ii) Unchecked exceptions

Exceptions which are checked by the JVM at the time of runtime is called unchecked exceptions.

ex:

```
ArithmeticException
ClassCastException
NullPointerException
```

If any checked exception raised in our program we must and should handle that exception by using try and catch block.

try block

It is a block which contains risky code.

It associate with catch block and finally block.

If any exception raise in try block then it won't be executed.

It is used to throw the exception to catch block.

catch block

It is a block which contains error handling code.

It always associate with try block.

It is used to catch the exception which is thrown by try block.

If there is no exception in try block then catch block won't be executed.

A catch block will take exception name as a parameter and that name must match with exception class name.

syntax:

```
try
{
    -
    - //risky code
    -
}
catch(ArithmeticException ae)
{
    -
    - // Error Handling Code
    -
}
```

ex:

```

---
class Test
{
    public static void main(String[] args)
    {
        try
        {
            System.out.println("try-block");
        }
        catch (Exception e)
        {
            System.out.println("catch-block");
        }
    }
}
o/p:
    try-block

```

```

ex:
---
class Test
{
    public static void main(String[] args)
    {
        try
        {
            System.out.println(10/0);
        }
        catch (ArithmeticException ae)
        {
            System.out.println("catch-block");
        }
    }
}
o/p:
    catch-block

```

```

ex:
---
class Test
{
    public static void main(String[] args)
    {
        try
        {
            System.out.println("stmt1");
            System.out.println(10/0);
            System.out.println("stmt2");
        }
        catch (ArithmeticException ae)
        {
            System.out.println("catch-block");
        }
    }
}

```

```

        }
    }
}
o/p:
    stmt1
    catch-block

ex:
---
class Test
{
    //static block
    static
    {
        System.out.println("static-block");
    }

    public static void main(String[] args)
    {
        try
        {
            Class.forName("Test");
        }
        catch (ClassNotFoundException cnfe)
        {
            System.out.println("Exception Occured");
        }
    }
}

```

Various ways to display exception details

=====

Throwable class defines following three methods to display exception details.

1) printStackTrace()

It gives name of the exception, description of the exception and line number of the exception.

2) toString()

It gives name of the exception and description of the exception.

3) getMessage()

It gives description of the exception.

ex:

```

class Test
{

```



```

public static void main(String[] args)
{
    try
    {
        System.out.println(10/0);
    }
    catch (ArithmeticException ae)
    {
        ae.printStackTrace();

        System.out.println("=====");

        System.out.println(ae.toString());

        System.out.println("=====");

        System.out.println(ae.getMessage());
    }
}

```

A try with multiple catch blocks

=====

A try block can have multiple catch blocks.

If try block contains multiple catch blocks then order of catch block is very important. It should be from child to parent but not from parent to child.

ex:

```

class Test
{
    public static void main(String[] args)
    {
        try
        {
            //System.out.println(10/0);
            int[] arr = null;
            System.out.println(arr[0]);
        }
        catch (ArithmeticException ae)
        {
            System.out.println("From AE");
        }
        catch (RuntimeException re)
        {
            System.out.println("From RE");
        }
    }
}

```

Q) Can we handle multiple exceptions in a single catch block?

Yes, we can handle multiple exceptions in a single catch block.

ex:

--

```
class Test
{
    public static void main(String[] args)
    {
        try
        {
            //System.out.println(10/0);

            int[] arr = null;
            System.out.println(arr[0]);
        }
        catch (ArithmeticException | NullPointerException e)
        {
            e.printStackTrace();
        }
    }
}
```

finally
=====

It is never recommended to maintain cleanup code in try block because if any exception raised in try block then it won't be executed.

It is never recommended to maintain cleanup code in catch block because if there is no exception in try block then catch block won't be executed.

But we need to place where we can maintain cleanup code and it should execute irrespective of exception raise or not. That block is called finally block.

syntax:

```
try
{
    - //Risky Code
}
catch(Exception e)
{
    - //Error Handling Code
}
finally
```

```

        {
            - //Cleanup Code
        }

ex:
---
class Test
{
    public static void main(String[] args)
    {
        try
        {
            System.out.println("try-block");
        }
        catch (Exception e)
        {
            e.printStackTrace();
        }
        finally
        {
            System.out.println("finally-block");
        }
    }
}
o/p:
    try-block
    finally-block

ex:
--
class Test
{
    public static void main(String[] args)
    {
        try
        {
            System.out.println("stmt1");
            System.out.println(10/0);
            System.out.println("stmt2");
        }
        catch (Exception e)
        {
            e.printStackTrace();
        }
        finally
        {
            System.out.println("finally-block");
        }
    }
}

```

o/p:

```
stmt1
java.lang.ArithmeticException: / by zero
    at Test.main(Test.java:8)
finally-block
```

ex:

```
class Test
{
    public static void main(String[] args)
    {
        int i=1;
        try
        {
            i++;
        }
        catch (Exception e)
        {
            i++;
        }
        finally
        {
            i++;
        }
        System.out.println(i);//3
    }
}
```

A try with finally combination is valid in java.

ex:

```
class Test
{
    public static void main(String[] args)
    {
        try
        {
            System.out.println("try-block");
        }
        finally
        {
            System.out.println("finally-block");
        }
    }
}
```

o/p:

```
try-block
finally-block
```

```

ex:
---
import java.util.Scanner;
class Test
{
    public static void main(String[] args)
    {
        Scanner sc = null;
        try
        {
            sc = new Scanner(System.in);
            System.out.println("Enter the Name :");
            String name = sc.nextLine();
            System.out.println("Welcome :"+name);
        }
        catch(Exception e)
        {
            e.printStackTrace();
        }
        finally
        {
            sc.close();
        }
    }
}

```

Q) What is the difference between final, finally and finalize ?

final

It is a modifier which is applicable for variables, methods and classes.

If we declare any variable as final then reinitialization of that variable is not possible.

If we declare any method as final then overriding of that method is not possible.

If we declare any class as final then creating child class is not possible.

finally

It is a block which contains cleanup code and it executes irrespective of exception raised or not.

finalize

It is a method called by garbage collector just before destroying an object for cleanup activity.

Garbage Collector

=====

It is also known as Daemon thread.

Deamon thread is a leight weight thread which runs in a background to provides services.

It is a programming feature for memory allocation and deallocation.

There are two ways to call garbage collector in java.

1) System.gc()

2) Runtime.getRuntime.gc()

1) System.gc()

```
class Test
{
    //instance variable
    int i=10;

    public static void main(String[] args)
    {
        Test t = new Test();
        System.out.println(t.i); // 10

        t = null;

        System.gc();
    }
    public void finalize()
    {
        System.out.println("finalize method");
    }
}
```

2) Runtime.getRuntime.gc()

```
class Test
{
    //instance variable
    int i=10;

    public static void main(String[] args)
    {
        Test t = new Test();
        System.out.println(t.i); // 10

        t = null;
    }
}
```

```

        Runtime.getRuntime().gc();
    }
    public void finalize()
    {
        System.out.println("finalize method");
    }
}

```

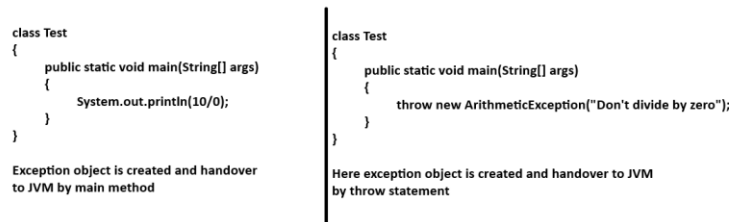
throw statement
=====

Sometimes we will create exception object explicitly and handover to JVM manually by using throw statement.

ex:

```
throw new ArithmeticException("Don't divide by zero");
```

Diagram: class43.1



throws statement
=====

If any checked exception raised in our program we must and should handle that exception by using try and catch block or by using throws statement.

ex:

```

class Test
{
    public static void main(String[] args)
    {
        try
        {
            Thread.sleep(3000);
            System.out.println("Welcome to Java");
        }
        catch (InterruptedException ie)
        {
            ie.printStackTrace();
        }
    }
}

```

```

ex:
---
class Test
{
    public static void main(String[] args) throws InterruptedException
    {
        Thread.sleep(5000);
        System.out.println("Welcome to Java");
    }
}

```

2) Userdefined exceptions

=====

Exceptions which are created by the user based on the application requirements are called user defined exceptions or custom exceptions.

```

ex:
    NoInterestInJavaException
    NoPracticeNoJobException
    LPGGasShortageException
    and etc.

```

```

ex:
---
import java.util.Scanner;

class TooYoungException extends RuntimeException
{
    TooYoungException(String s)
    {
        super(s);
    }
}

class TooOldException extends RuntimeException
{
    TooOldException(String s)
    {
        super(s);
    }
}

class Test
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter the age : ");
        int age = sc.nextInt();

        if(age<18)
            throw new TooYoungException("U r not eligible to
vote");
        else
            throw new TooOldException("U r eligible to vote");
    }
}

```



```
    }  
}
```

```
try-with-resources  
=====
```

A try-with-resources introduced in Java 7.

A try-with-resources is a try block which contains one or more resources.

A try-with-resources ensures that each resource must be closed at the end of the statement.

ex:

```
import java.util.Scanner;
```

```
class Test
```

```
{  
    public static void main(String[] args)  
    {  
        try(Scanner sc = new Scanner(System.in);)  
        {  
            System.out.println("Enter the name :");  
            String name = sc.nextLine();  
            System.out.println("Welcome :"+name);  
        }  
        catch (Exception e)  
        {  
            e.printStackTrace();  
        }  
    }  
}
```

```
java.io package  
=====
```

```
File  
=====
```

```
    File f = new File("abc.txt");
```

File object checks is there any "abc.txt" file already created or not. If it is created then it simply refers to that file. If it is not created then it won't create physical file.

ex:

```
import java.io.*;
```

```
class Test
```

```
{  
    public static void main(String[] args)  
    {
```

```

        File f = new File("abc.txt");
        System.out.println(f.exists()); // false
    }
}

```

File object can be used to create a physical file.

```

ex:
---
import java.io.*;
class Test
{
    public static void main(String[] args) throws IOException
    {
        File f = new File("abc.txt");
        System.out.println(f.exists()); // false

        f.createNewFile();
        System.out.println(f.exists()); // true
    }
}

```

File object can be used to create a directory.

```

ex:
---
import java.io.*;
class Test
{
    public static void main(String[] args)
    {
        File f = new File("ihub");
        System.out.println(f.exists()); // false

        f.mkdir();
        System.out.println(f.exists()); // true
    }
}

```

Q) Write a java program to create cricket folder and inside that folder create abc.txt file?

```

import java.io.*;
class Test
{
    public static void main(String[] args) throws IOException
    {
        File f1 = new File("cricket");
        f1.mkdir();
    }
}

```

```

        File f2 = new File("cricket","abc.txt");
        f2.createNewFile();

        System.out.println("Please check the location");
    }
}

```

FileWriter

=====

It is used to insert character oriented data into a file.

constructor

```
FileWriter fw = new FileWriter(File f)
```

FileWriter object can communicate with file directly.

If file is not available then it will create a physical file.

Methods

1) write(int ch)

It is used to insert single character into a file.

2) write(char[] ch)

It is used to insert array of characters into a file.

3) write(String s)

It is used to insert string into a file.

4) flush()

It gives guarantee that last character of a file is also inserted.

5) close()

It is used to close FileWriter object.

ex:

```

import java.io.*;
class Test
{
    public static void main(String[] args)
    {
        try(FileWriter fw = new FileWriter("aaa.txt");)
        {
            fw.write(98); // b
            fw.write("\n");
        }
    }
}

```

```

        char[] ch={'a','b','c'};
        fw.write(ch);
        fw.write("\n");

        fw.write("ihub\ntalent");

        fw.flush();
        System.out.println("Please check the location");
    }
    catch (IOException ioe)
    {
        ioe.printStackTrace();
    }
}

```

FileReader

It is used to read character oriented data from a file.

constructor

```
FileReader fr = new FileReader(File f);
```

FileReader object can communicate with file directly.

methods

1) read()

It reads next character from a file and returns unicode value.
If next character is not available then it will return -1.

2) read(char[] ch)

It is used to read collection of characters from a file.

3) close()

It is used to close FileReader object.

ex:

--

```

import java.io.*;
class Test
{
    public static void main(String[] args)
    {
        try(FileReader fr = new FileReader("aaa.txt");)
        {

```

```

        int i = fr.read();
        while(i!=-1)
        {
            System.out.print((char)i);
            i = fr.read();
        }
    }
    catch (IOException ioe)
    {
        ioe.printStackTrace();
    }
}

ex:
---
import java.io.*;
class Test
{
    public static void main(String[] args)
    {
        try(FileReader fr = new FileReader("aaa.txt");)
        {
            char[] arr = new char[40];

            fr.read(arr);

            for(char ch : arr)
            {
                System.out.print(ch);
            }
        }
        catch (IOException ioe)
        {
            ioe.printStackTrace();
        }
    }
}

```

Limitations with FileWriter and FileReader

While writing the data using FileWriter object we need to insert line separator(\n) which is very headache for the programmer.

While reading the data using FileReader object we need to read character by character which is not convenient to the programmer.

To overcome above limitations Sun Micro System introduced BufferedWriter and BufferedReader.

BufferedWriter

It is used to insert character oriented data into a file.

constructor

```
BufferedWriter bw = new BufferdWriter(Writer w);
```

BufferedWriter can't communicate with file directly. It will take the support of some Writer object.

If that file is not available then it will create a physical file.

Methods

1) write(int ch)

It is used to insert single character into a file.

2) write(char[] ch)

It is used to insert array of characters into a file.

3) write(String s)

It is used to insert string into a file.

4) flush()

It gives guarantee that last character of a file is also inserted.

5) close()

It is used to close FileWriter object.

6) newLine()

It is used to insert new line in a file.

ex:

```
import java.io.*;
class Test
{
    public static void main(String[] args)
    {
        try(BufferedWriter bw = new BufferedWriter(new
FileWriter("bbb.txt"));)
        {
            bw.write(98); // b
            bw.newLine();

            char[] ch={'a','b','c'};
```

```

        bw.write(ch);
        bw.newLine();

        bw.write("ihub");

        bw.flush();

        System.out.println("Please check the location");
    }
    catch (IOException ioe)
    {
        ioe.printStackTrace();
    }
}

```

BufferedReader

It is an enhanced reader to read character oriented data from a file.

constructor

```
BufferedReader br = new BufferedReader(Reader r);
```

BufferedReader can't communicate with files directly. It will take the support of reader object.

The main objective of BufferedReader over FileReader is we can read line by line instead of character by character.

Methods

1) read()

It reads next character from a file and returns unicode value.
If next character is not available then it will return -1.

2) read(char[] ch)

It is used to read collection of characters from a file.

3) close()

It is used to close FileReader object.

4) readLine()

It is used to read next line from a file. If next line is not available
then it will return null.

```

ex:
---
import java.io.*;
class Test
{
    public static void main(String[] args)
    {
        try(BufferedReader br = new BufferedReader(new
FileReader("bbb.txt"));)
        {
            String line = br.readLine();
            while(line!=null)
            {
                System.out.println(line);
                line = br.readLine();
            }
        }
        catch (IOException ioe)
        {
            ioe.printStackTrace();
        }
    }
}

```

PrintWriter
=====

It is a enhanced writer to write character oriented data into a file.

constructor

```

PrintWriter pw = new PrintWriter(File f);
PrintWriter pw = new PrintWriter(Writer w);

```

PrintWriter object can communicate with file directly and it will take the support of writer object.

If that file is not available then it will create a physical file.

The main advantage of PrintWriter over FileWriter and BufferedWriter is we can insert any type of data. If we have primitive values then it is recommended to use PrintWriter object.

Methods

```

write(int ch)
write(char[] ch)
write(String s)
flush()
close()

```

```

println(int i)

```



```
println(char ch)
println(boolean b)
println(String s)
println(double d);
```

```
print(int i)
print(char ch)
print(boolean b)
print(String s)
print(double d);
```

ex:

```
import java.io.*;
class Test
{
    public static void main(String[] args)
    {
        try(PrintWriter pw = new PrintWriter("ccc.txt");)
        {
            pw.write(98); // b
            pw.println(98); // 98
            pw.println('a'); // a
            pw.println(true); // true
            pw.println(10.5d); // 10.5
            pw.println("Hi"); // Hi
            pw.flush();
            System.out.println("Please check the location");
        }
        catch (IOException ioe)
        {
            ioe.printStackTrace();
        }
    }
}
```

FileOutputStream

It is used to write the data into a file in the form of byte of stream.

FileInputStream

It is used to read the data from a file in the form of byte of stream.

Q) Write a java program to copy the data from one file to another file?

source.txt

a
ab
abc
abcd

```
import java.io.*;
class Test
{
    public static void main(String[] args)
    {
        try(
            FileInputStream fis = new
FileInputStream("source.txt");
            FileOutputStream fos = new
FileOutputStream("destination.txt");
        )
        {
            int byteReads = 0;
            byte[] arr = new byte[100];

            while((byteReads=fis.read(arr))!=-1)
            {
                fos.write(arr,0,byteReads);
            }
            System.out.println("Data Copied!!!");
        }
        catch (IOException ioe)
        {
            ioe.printStackTrace();
        }
    }
}
```

Various ways to provide input values

=====

There are four ways to provide input values.

- 1) Command line argument
- 2) BufferedReader class
- 3) Console class
- 4) Scanner class

- 1) Command line argument

```
class Test
{
```

```

        public static void main(String[] args)
        {
            String name = args[0];
            System.out.println("Welcome :"+name);
        }
    }

```

o/p:

```

javac Test.java
java Test JackMa

```

2) BufferedReader class

```

-----
import java.io.*;
class Test
{
    public static void main(String[] args)throws IOException
    {
        BufferedReader br = new BufferedReader(new
InputStreamReader(System.in));
        System.out.println("Enter the Name :");
        String name = br.readLine();
        System.out.println("Welcome :"+name);
    }
}

```

3) Console class

```

-----
import java.io.*;
class Test
{
    public static void main(String[] args)throws IOException
    {
        Console c = System.console();
        System.out.println("Enter the Name :");
        String name = c.readLine();
        System.out.println("Welcome :"+name);
    }
}

```

4) Scanner class

```

-----
import java.util.*;
class Test
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);

        System.out.println("Enter the student no :");
        int no = sc.nextInt();

        System.out.println("Enter the student name :");
    }
}

```

```

        String name = sc.next();

        System.out.println("Enter the student fee :");
        double fee = sc.nextDouble();

        System.out.println(no+" "+name+" "+fee);
    }
}

```

Serialization

=====

The process of storing object data into a file is called serialization.

Converting object state to file state is called serialization.

To perform serialization we will use ObjectOutputStream and FileOutputStream.

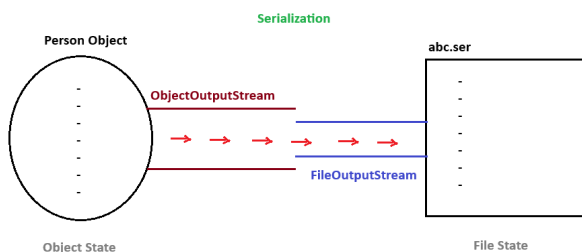
We can perform serialization only for serialized object.

To create serialized object our class must implements Serializable interface.

A Serializable is a marker interface which does not contain methods and constants.

By using Serializable interface we will get some ability by JVM.

Diagram: class44.1



```

import java.io.*;
class Person implements Serializable
{
    private String name;
    private int age;
}

```

```

    public Person(String name,int age)
    {
        this.name = name;
        this.age = age;
    }

    public String toString()
    {
        return name+" "+age;
    }
}
class Test
{
    public static void main(String[] args)
    {
        Person p = new Person("Alan",32);
        try(ObjectOutputStream ostream =
            new ObjectOutputStream(new
FileOutputStream("abc.ser"));)
        {
            ostream.writeObject(p);
            System.out.println("Serialization Done");
        }
        catch(IOException ioe)
        {
            ioe.printStackTrace();
        }
    }
}

```

Deserialization

=====

The process of taking the data from a file and representing an object is called deserialization.

Converting file state to object state is called deserialization.

To perform deserialization we will use `ObjectInputStream` and `FileInputStream`.

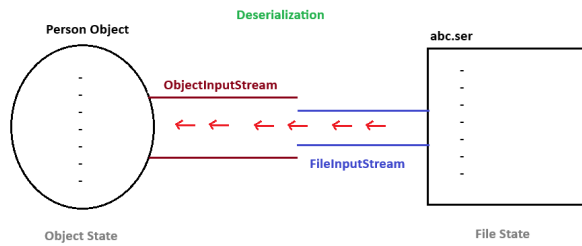
We can perform deserialization only for serialized object.

To create serialized object our class must implements `Serializable` interface.

A `Serializable` is a marker interface which does not contain methods and constants.

By using `Serializable` interface we will get some ability by JVM.

Diagram: class44.2



ex:

```
import java.io.*;
class Person implements Serializable
{
    private String name;
    private int age;

    public Person(String name,int age)
    {
        this.name = name;
        this.age = age;
    }

    public String toString()
    {
        return name+" "+age;
    }
}
class Test
{
    public static void main(String[] args)
    {
        try(ObjectInputStream istream =
            new ObjectInputStream(new
FileInputStream("abc.ser"));)
        {
            Person p =(Person) istream.readObject();
            System.out.println(p);
        }
        catch(IOException | ClassNotFoundException e)
        {
            e.printStackTrace();
        }
    }
}
```

Singleton class

=====

A class which allows us to create only one object is called singleton class.

It is a design pattern which ensures that only one instance of a class will be created.

The main purpose of singleton class is to control object creations, save some memories or resources, consistency across the application and Global access point.

To create a singleton class we required private constructor and static method.

ex:

```
class Singleton
{
    static Singleton single=null;

    private Singleton()
    {
        System.out.println("private-const");
    }

    public static Singleton getInstance()
    {
        if(single==null)
        {
            single = new Singleton();
        }
        return single;
    }
}
class Test
{
    public static void main(String[] args)
    {
        Singleton s1 = Singleton.getInstance();
        System.out.println(s1.hashCode()); //1225197672

        Singleton s2 = Singleton.getInstance();
        System.out.println(s2.hashCode()); //1225197672
    }
}
```

Generics

=====

Arrays are typesafe.

It means we can give guarantee that what type of elements are present in array.

ex:

```
int[] arr = new int[10];
arr[0]=10;
arr[1]=20;
arr[2]=30;
```

If requirement is there to store string values then it is recommended to use String[].

ex:

```
String[] sarr = new String[10];
sarr[0]="Hi";
sarr[1]="Hello";
sarr[2]="bye";
```

At the time of retrieving the data from array we don't need to perform typecasting.

ex:

```
String[] sarr = new String[10];
sarr[0]="Hi";
sarr[1]="Hello";
sarr[2]="bye";
-
-
String val = sarr[0];
```

Collections are not typesafe.

It means we can't provide guarantee that what type of elements are present in Collections.

If requirement is there to store string values then it is recommended to use ArrayList because we won't get any compile time error or runtime error but some times our program get failure.

ex:

```
ArrayList al =new ArrayList();
al.add("Hi");
al.add("Hello");
al.add(10);
```

At the time of retrieving the data compulsory we need to perform typecasting.

ex:

```
ArrayList al =new ArrayList();
al.add("Hi");
al.add("Hello");
al.add(10);
-
-
```



```
String val1 = (String)al.get(0);
```

To overcome above limitations Sun Micro system introduced Generics concept in 1.5 version.

The main objective of generics are

- 1) To make collections as typesafe.
- 2) To avoid typecasting problem.

```
java.util Package  
=====
```

Q) What is the difference between Arrays and Collections?

Arrays	Collections
-----	-----
It is a collection of homogeneous data homogeneous and elements.	It is a collection of heterogeneous data elements.
It is fixed in size.	It is growable in nature.
Performance point of view arrays are Collections are recommanded to use.	Memory point of view recommanded to use.
Arrays not implemented based on data based on structures concept. Hence we can't expect Hence we can any ready made method.	Collections are implemented data structures concept. expect ready made methods.
It supports primitive types and object types.	It supports object types.
It is typesafe.	It is not typesafe.

```
Collection Framework  
=====
```

It defines several utility classes and interfaces to represent group of objects in a single entity.

ex:

Java	C++
-----	-----
Collections	Containers
Collection Framework	STL (Standard Template Library)

Collection

=====

It is an interface which is present in java.util package.

It is a root interface for entire collection framework.

If we want to represent group of individual objects in a single entity then we need to use Collection interface.

It provides some common methods which are available for entire Collection objects.

ex:

```
cmd> javap java.util.Collection
```

```
public abstract int size();
public abstract boolean isEmpty();
public abstract boolean contains(java.lang.Object);
public abstract java.util.Iterator<E> iterator();
public abstract java.lang.Object[] toArray();
public abstract <T> T[] toArray(T[]);
public default <T> T[] toArray(java.util.function.IntFunction<T[]>);
public abstract boolean add(E);
public abstract boolean remove(java.lang.Object);
public abstract boolean containsAll(java.util.Collection<?>);
public abstract boolean addAll(java.util.Collection<? extends E>);
public abstract boolean removeAll(java.util.Collection<?>);
public default boolean removeIf(java.util.function.Predicate<? super
E>);
public abstract boolean retainAll(java.util.Collection<?>);
public abstract void clear();
and etc.
```

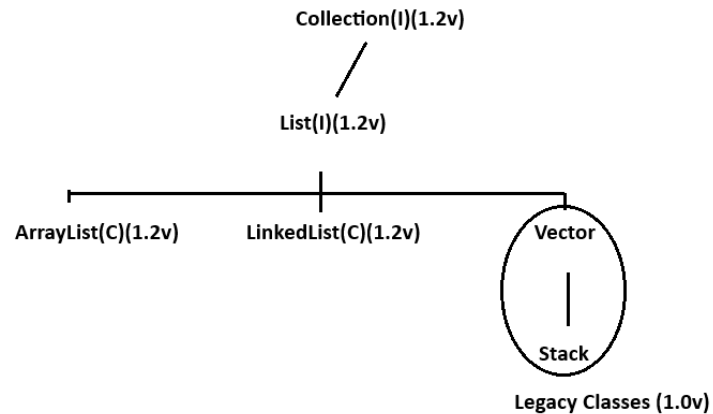
List

====

It is a child interface of Collection interface.

If we want to represent group of individual objects in a single entity where duplicate objects are allowed and order is preserved then we need to use List interface.

Diagram: class45.1



Q) What is the difference between List.of() and Arrays.asList() method?

List.of()

It is introduced in Java 9.
It gives immutable object.
We can't add, remove and replace the elements.
It does not allow null.

ex:

```

import java.util.*;
class Test
{
    public static void main(String[] args)
    {
        List<Integer> list = List.of(7,2,9,1,5,8);
        for(int i : list)
        {
            System.out.print(i+" ");
        }
    }
}

```

ex:

```

import java.util.*;
class Test
{
    public static void main(String[] args)
    {

```

```

        List<Integer> list = List.of(7,2,9,1,5,8);
        list.add(10);
        for(int i : list)
        {
            System.out.print(i+" ");
        }
    }
}

```

o/p:
R.E: UnsupportedOperationException

```

ex:
---
import java.util.*;
class Test
{
    public static void main(String[] args)
    {
        List<Integer> list = List.of(7,2,9,1,5,8);
        list.remove(2);
        for(int i : list)
        {
            System.out.print(i+" ");
        }
    }
}

```

o/p:
R.E: UnsupportedOperationException

```

ex:
---
import java.util.*;
class Test
{
    public static void main(String[] args)
    {
        List<Integer> list = List.of(7,2,9,1,5,8);
        list.set(2,100);
        for(int i : list)
        {
            System.out.print(i+" ");
        }
    }
}

```

o/p:
R.E: UnsupportedOperationException

ex:

```

---
import java.util.*;
class Test
{
    public static void main(String[] args)
    {
        List<Integer> list = List.of(7,2,9,1,5,8,null);
        for(int i : list)
        {
            System.out.print(i+" ");
        }
    }
}

```

o/p:
R.E: NullPointerException

Arrays.asList()

It is introduced in 1.2 version.

It gives mutable object.

We can't add , remove but we can replace the elements.

It allows null.

ex:

```

import java.util.*;
class Test
{
    public static void main(String[] args)
    {
        List<Integer> list = Arrays.asList(7,2,9,1,5,8);
        for(int i : list)
        {
            System.out.print(i+" ");
        }
    }
}

```

ex:

```

import java.util.*;
class Test
{
    public static void main(String[] args)
    {
        List<Integer> list = Arrays.asList(7,2,9,1,5,8);
        list.add(10);
        for(int i : list)
        {
            System.out.print(i+" ");
        }
    }
}

```

```

    }
}

o/p:
R.E : UnsupportedOperationException

```

```

ex:
---
import java.util.*;
class Test
{
    public static void main(String[] args)
    {
        List<Integer> list = Arrays.asList(7,2,9,1,5,8);
        list.remove(2);
        for(int i : list)
        {
            System.out.print(i+" ");
        }
    }
}

```

```

o/p:
R.E : UnsupportedOperationException

```

```

ex:
---
import java.util.*;
class Test
{
    public static void main(String[] args)
    {
        List<Integer> list = Arrays.asList(7,2,9,1,5,8);
        list.set(2,100);
        for(int i : list)
        {
            System.out.print(i+" ");
        }
    }
}

```

```

ex:
---
import java.util.*;
class Test
{
    public static void main(String[] args)
    {
        List<Integer> list = Arrays.asList(7,2,9,1,5,8,null);
        System.out.println(list);
    }
}

```

```
    }  
}
```

Q) Write a java program to sort the list?

input:
ball elephant dog apple cat

output:
apple ball cat dog elephant

```
import java.util.*;  
class Test  
{  
    public static void main(String[] args)  
    {  
        List<String> list =  
Arrays.asList("ball","elephant","dog","apple","cat");  
  
        Collections.sort(list);  
  
        System.out.println(list);  
    }  
}
```

Q) Write a java program to reverse the list?

input:
this is java class

output:
class java is this

```
import java.util.*;  
class Test  
{  
    public static void main(String[] args)  
    {  
        List<String> list =  
Arrays.asList("this","is","java","class");  
  
        Collections.reverse(list);  
  
        System.out.println(list);  
    }  
}
```

Q) How to make a List as synchronized?

```

import java.util.*;
class Test
{
    public static void main(String[] args)
    {
        List<Integer> list = List.of(7,9,1,4,6,2);

        //only one thread is allowed at a time to operate
        List<Integer> synList = Collections.synchronizedList(list);

        System.out.println(synList);
    }
}

```

ArrayList
=====

The underlying data structure is resizable array or growable array.

Duplicate objects are allowed.

Insertion order is preserved.

Heterogeneous objects are allowed.

Null insertion is possible.

It implements List, Serializable, Cloneable and RandomAccess interface.

ex:

```

import java.util.*;
class Test
{
    public static void main(String[] args)
    {
        ArrayList al = new ArrayList();
        al.add("one");
        al.add("two");
        al.add("three");
        System.out.println(al); //[one,two,three]

        al.add("one");
        System.out.println(al); //[one,two,three,one]

        al.add(10);
        System.out.println(al); //[one,two,three,one,10]

        al.add(null);
        System.out.println(al); //[one,two,three,one,10,null]
    }
}

```



```

}

ex:
---
import java.util.*;
class Test
{
    public static void main(String[] args)
    {
        ArrayList<String> al = new ArrayList<>();
        al.add("one");
        al.add("two");
        al.add("three");
        System.out.println(al); //[one,two,three]

        al.add("one");
        System.out.println(al); //[one,two,three,one]

        al.add(null);
        System.out.println(al); //[one,two,three,one,null]
    }
}

```

```

ex:
---
import java.util.*;
class Test
{
    public static void main(String[] args)
    {
        ArrayList<String> al = new ArrayList<>();
        al.add("one");
        al.add("two");
        al.add("three");

        for(String s: al)
        {
            System.out.print(s+" ");
        }
    }
}

```

```

ex:
---
import java.util.*;
class Test
{
    public static void main(String[] args)
    {
        ArrayList<String> al = new ArrayList<>();
        al.add("one");
    }
}

```

```

        al.add("two");
        al.add("three");

        for(int i=0;i<al.size();i++)
        {
            System.out.print(al.get(i)+" ");
        }
    }
}

ex:
---
import java.util.*;
class Test
{
    public static void main(String[] args)
    {
        ArrayList<String> al = new ArrayList<>();
        al.add("one");
        al.add("two");
        al.add("three");

        System.out.println(al.contains("two")); // true

        System.out.println(al.getFirst()); // one

        System.out.println(al.getLast()); // three

        al.add(1,"gogo");
        System.out.println(al); // [one,gogo,two,three]

        al.remove("gogo");
        System.out.println(al); // [one,two,three]

        al.clear();
        System.out.println(al); // []
    }
}

```

```

ex:
---
import java.util.*;
class Test
{
    public static void main(String[] args)
    {
        List<Integer> list = new ArrayList<>();
        list.add(10);
        list.add(20);
        list.add(30);
        System.out.println(list);
    }
}

```

```
    }  
}
```

LinkedList

=====

The underlying data structure is doubly LinkedList.

Duplicate objects are allowed.

Insertion order is preserved.

Hetrogeneous objects are allowed.

Null insertion is possible.

It implements List, Serializable, Cloneable and Deque interface.

ex:

```
import java.util.*;  
class Test  
{  
    public static void main(String[] args)  
    {  
        LinkedList ll = new LinkedList();  
        ll.add("one");  
        ll.add("two");  
        ll.add("three");  
        System.out.println(ll); //[one,two,three]  
  
        ll.add("one");  
        System.out.println(ll); //[one,two,three,one]  
  
        ll.add(10);  
        System.out.println(ll); //[one,two,three,one,10]  
  
        ll.add(null);  
        System.out.println(ll); //[one,two,three,one,10,null]  
    }  
}
```

ex:

--

```
import java.util.*;  
class Test  
{  
    public static void main(String[] args)  
    {  
        LinkedList<String> ll = new LinkedList<>();  
        ll.add("one");  
        ll.add("two");  
    }  
}
```

```

        ll.add("three");
        System.out.println(ll); //[one,two,three]

        ll.add("one");
        System.out.println(ll); //[one,two,three,one]

        ll.add(null);
        System.out.println(ll); //[one,two,three,one,null]
    }
}

ex:
--
import java.util.*;
class Test
{
    public static void main(String[] args)
    {
        LinkedList<String> ll1 = new LinkedList<>();
        ll1.add("one");
        ll1.add("two");
        ll1.add("three");
        System.out.println(ll1); //[one,two,three]

        LinkedList<String> ll2 = new LinkedList<>();
        ll2.add("raja");
        System.out.println(ll2); //[raja]

        ll2.addAll(ll1);
        System.out.println(ll2); //[one,two,three,raja]

        System.out.println(ll2.containsAll(ll1)); // true

        ll2.removeAll(ll1);
        System.out.println(ll2); //[raja]
    }
}

```

Q) What is the difference between ArrayList and LinkedList?

ArrayList	LinkedList
-----	-----
The underlying data structure is resizable array or growable array.	The underlying data structure is doubly LinkedList.
ArrayList is better for storing and manipulating accessing the data.	LinkedList is better for the data.

Memory location for ArrayList elements
LinkedList elements
is contiguous.

Memory location for
is not contiguous.

When ArrayList is initialized a default
capacity.
capacity 10 is assigned to it.

There is no case of default

Vector
=====

The underlying data structure is resizable array or growable array.

Duplicate objects are allowed.

Insertion order is preserved.

Hetrogeneous objects are allowed.

Null insertion is possible.

It implements List, Serializable, Cloneable and RandomAccess
interface.

All methods present in Vector are synchronized. Hence Vector is thread
safe.

Vector class contains following methods.
ex:

```
public synchronized void addElement(E);  
public synchronized E firstElement();  
public synchronized E lastElement();  
public synchronized boolean removeElement(java.lang.Object);  
public synchronized void setElementAt(E, int);  
public synchronized void removeAllElements();  
and etc.
```

ex:

```
import java.util.*;  
class Test  
{  
    public static void main(String[] args)  
    {  
        Vector v = new Vector();  
        System.out.println(v.capacity()); // 10  
  
        for(int i=1;i<=10;i++)  
        {  
            v.addElement(i);  
        }  
        System.out.println(v); //[1,2,3,4,5,6,7,8,9,10]  
    }  
}
```

```

        System.out.println(v.firstElement()); //1

        System.out.println(v.lastElement()); //10

        v.removeElement(9);
        System.out.println(v); // [1,2,3,4,5,6,7,8,10]

        v.removeAllElements();
        System.out.println(v); // []

    }
}

```

Q) What is the difference between ArrayList and Vector?

ArrayList	Vector
-----	-----
It is a non-legacy class.	It is a legacy class.
It is introduced in 1.2 version.	It is introduced in 1.0 version.
Methods are not synchronized.	Methods are synchronized.
At a time multiple threads are allowed to operate on ArrayList object. Hence ArrayList is not thread safe.	At a time only one thread operate on Vector object. Hence Vector is thread safe.
Performance is high because there is no waiting threads.	Performance is low because there is a waiting threads.

Stack
=====

It is a child class of Vector class.

If we depend upon Last In First Out order then we need to use stack.

constructor

```
Stack s = new Stack();
```

methods

1) push(E)

It push the element to stack.

2) pop()

It pop the element from stack

3) peek()

It returns toppest element from stack.

4) isEmpty()

It is used to check stack is empty or not.

5) search(E)

It returns offset value if element is found otherwise it will return -1.

ex:

```
import java.util.*;
class Test
{
    public static void main(String[] args)
    {
        Stack<String> s = new Stack<>();
        s.push("A");
        s.push("B");
        s.push("C");
        System.out.println(s); //[A,B,C]

        s.pop();
        System.out.println(s); //[A,B]

        System.out.println(s.peek()); // B

        System.out.println(s.isEmpty()); // false

        System.out.println(s.search("Z")); // -1

        System.out.println(s.search("A")); // 2
    }
}
```

Q) Write a java program to check given string is balanced or not?

input:

{[()]}

ouptut:

It is a balanced string

```

import java.util.*;
class Test
{
    public static void main(String[] args)
    {
        String str="{[()]}" ;
        if(isBalanced(str.toCharArray()))
            System.out.println("It is a balanced string");
        else
            System.out.println("It is not a balanced string");
    }
    public static boolean isBalanced(char[] carr)
    {
        Stack<Character> stack = new Stack<>();

        for(int i=0;i<carr.length;i++)
        {
            char ch = carr[i];

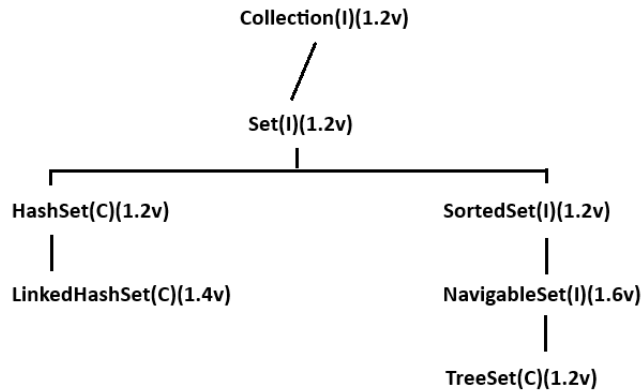
            if(ch == '{' || ch=='[' || ch=='(')
            {
                stack.push(ch);
            }
            else if(ch==')' && !stack.isEmpty() &&
stack.peek()=='(')
            {
                stack.pop();
            }
            else if(ch==']' && !stack.isEmpty() &&
stack.peek()=='[')
            {
                stack.pop();
            }
            else if(ch=='}' && !stack.isEmpty() &&
stack.peek()=='{')
            {
                stack.pop();
            }
            else
            {
                return false;
            }
        }

        return stack.isEmpty();
    }
}
Set
====
It is a child interface of Collection interface.

```


If we want to represent group of individual objects in a single entity where duplicate objects are not allowed and order is not preserved then we need to use Set interface.

Diagram: class46.1



Set.of()

=====

It is introduced in Java 9.
It give Immutable object.
It does not allow duplicates.
It does not allow null.

ex:

```
import java.util.*;
class Test
{
    public static void main(String[] args)
    {
        Set<Integer> set = Set.of(6,9,1,4,2);
        System.out.println(set);
    }
}
```

ex:

```
import java.util.*;
class Test
{
    public static void main(String[] args)
    {
```

```

        Set<Integer> set  = Set.of(6,9,1,4,2);
        set.add(10);
        System.out.println(set);
    }
}

```

o/p:
R.E: UnsupportedOperationException

```

ex:
---
import java.util.*;
class Test
{
    public static void main(String[] args)
    {
        Set<Integer> set  = Set.of(6,9,1,4,2,9);
        System.out.println(set);
    }
}

```

o/p:
R.E : IllegalArgumentException

```

ex:
---
import java.util.*;
class Test
{
    public static void main(String[] args)
    {
        Set<Integer> set  = Set.of(6,9,1,4,2,null);
        System.out.println(set);
    }
}

```

o/p:
R.E : NullPointerException

```

ex:
--
import java.util.*;
class Test
{
    public static void main(String[] args)
    {
        Set<Integer> set  = Set.of(6,9,1,4,2);
        for(int i : set)
        {
            System.out.print(i+" ");
        }
    }
}

```

```
}
```

HashSet

=====

The underlying data structure is Hashtable.

Duplicate objects are not allowed.

Insertion order is not preserved because it takes hashcode of an object.

Hetrogeneous objects are allowed.

Null insertion is possible.

ex:

```
import java.util.*;
class Test
{
    public static void main(String[] args)
    {
        HashSet hs = new HashSet();
        hs.add("one");
        hs.add("nine");
        hs.add("two");
        hs.add("six");
        System.out.println(hs); //[nine, six, one, two]

        hs.add("one");
        System.out.println(hs); //[nine, six, one, two]

        hs.add(10);
        System.out.println(hs); // [nine, six, one, 10, two]

        hs.add(null);
        System.out.println(hs); // [null, nine, six, one, 10, two]
    }
}
```

LinkedHashSet

=====

It is a child class of HashSet class.

LinkedHashSet exactly same as HashSet class with following differences.

HashSet

The underlying data structure is
Hashtable

LinkedHashSet

The underlying data structure is

Hashtable. and LinkedList.

Insertion order is not preserved. Insertion order is preserved.

It is introduced in 1.2 version. It is introduced in 1.4 version.

ex:

```
import java.util.*;
class Test
{
    public static void main(String[] args)
    {
        LinkedHashSet lhs = new LinkedHashSet();
        lhs.add("one");
        lhs.add("nine");
        lhs.add("two");
        lhs.add("six");
        System.out.println(lhs); //[one,nine,two,six]

        lhs.add("one");
        System.out.println(lhs); //[one,nine,two,six]

        lhs.add(10);
        System.out.println(lhs); //[one,nine,two,six,10]

        lhs.add(null);
        System.out.println(lhs); //[one,nine,two,six,10,null]
    }
}
```

TreeSet

=====

The underlying data structure is Balanced Tree.

Duplicate objects are not allowed.

Insertion order is not preserved because it takes sorting order of an object.

Hetrogeneous objects are not allowed otherwise we will get
ClassCastException.

Null insertion is not possible otherwise we will get
NullPointerException.

ex:

```
import java.util.*;
class Test
{
```

```

public static void main(String[] args)
{
    TreeSet ts = new TreeSet();
    ts.add(6);
    ts.add(1);
    ts.add(10);
    ts.add(3);
    System.out.println(ts); // [1,3,6,10]

    ts.add(1);
    System.out.println(ts); // [1,3,6,10]

    //ts.add("Hi");
    //System.out.println(ts); // R.E ClassCastException

    //ts.add(null);
    //System.out.println(ts); // R.E NullPointerException
}
}

```

Q) Write a java program to display distinct elements from given array?

input:

1 2 2 3 3 3 4 4 4 4

output:

1 2 3 4

```

import java.util.*;
class Test
{
    public static void main(String[] args)
    {
        int[] arr = {1,2,2,3,3,3,4,4,4,4};

        Set<Integer> set = new LinkedHashSet<>();

        for(int i : arr)
        {
            set.add(i);
        }

        System.out.println(set);
    }
}

```

Q) Write a java program to find out unique pairs?

input:

```

5
john tom
john ana
john tom
marry ana
marry ana

```

output:

```

1
2
2
3
3

```

```

import java.util.*;
class Test
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt(); // 5

        Set<String> set = new LinkedHashSet<>();
        List<Integer> list = new ArrayList<>();

        //consume a line
        sc.nextLine();

        for(int i=1;i<=n;i++)
        {
            String pair = sc.nextLine().trim();
            set.add(pair);
            list.add(set.size());
        }

        for(int i : list)
        {
            System.out.println(i);
        }
    }
}

```

Map
====

It is not a child interface of Collection interface.

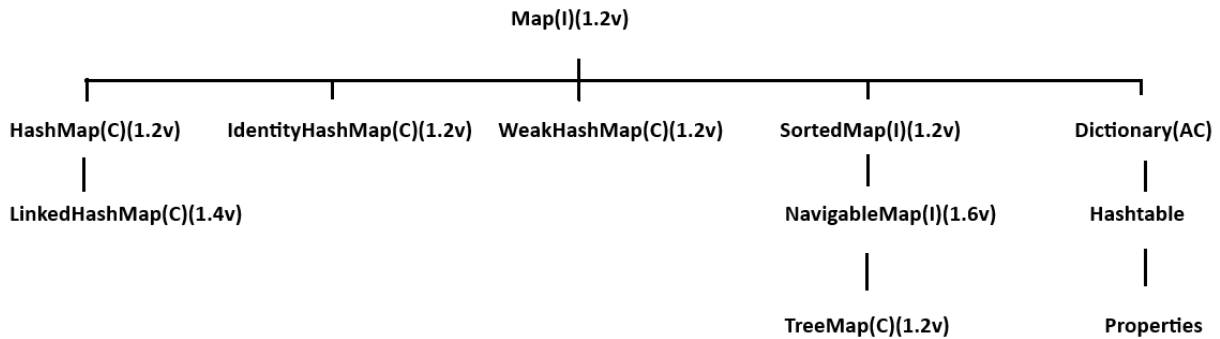
If we want to represent group of individual objects in a key and value pair then we need to use Map interface.

Key and value both must be objects.

key can't be duplicate but value can be duplicate.

Each key and value pair is known as one entry.

Diagram: class46.2



Map.of()

=====

It is introduced in Java 9.

It gives immutable object.

Key and value both can't be null.

ex:

```
import java.util.*;
class Test
{
    public static void main(String[] args)
    {
        Map<Integer,String> map = Map.of(
            1, "one",
            2, "two",
            3, "three");

        System.out.println(map); //{3=three, 2=two, 1=one}
    }
}
```

ex:

```
import java.util.*;
class Test
{
```

```

public static void main(String[] args)
{
    Map<Integer,String> map = Map.of(
        1, "one",
        2, "two",
        3, "three");

    map.put(4,"four");

    System.out.println(map);
}

```

o/p:
R.E : UnsupportedOperationException

```

ex:
---
import java.util.*;
class Test
{
    public static void main(String[] args)
    {
        Map<Integer,String> map = Map.of(
            1, "one",
            2, "two",
            null, "three");

        System.out.println(map);
    }
}

```

o/p:
R.E : NullPointerException

```

ex:
---
import java.util.*;
class Test
{
    public static void main(String[] args)
    {
        Map<Integer,String> map = Map.of(
            1, "one",
            2, "two",
            3, null);

        System.out.println(map);
    }
}

```


o/p:

R.E : NullPointerException

ex:

```
import java.util.*;
class Test
{
    public static void main(String[] args)
    {
        Map<Integer,String> map = Map.of(
            1, "one",
            2, "two",
            3, "three");

        for(Map.Entry<Integer,String> entry : map.entrySet())
        {

            System.out.println(entry.getKey()+"="+entry.getValue());
        }
    }
}
```

HashMap

=====

The underlying data structure is Hashtable.

Key can't be duplicate but value can be duplicate.

Insertion order is not preserved because it takes hashCode of a key.

Hetrogeneous objects are allowed for both key and value.

Null insertion is possible for both key and value.

ex:

```
import java.util.*;
class Test
{
    public static void main(String[] args)
    {
        HashMap hm = new HashMap();
        hm.put(6,"six");
        hm.put(1,"one");
        hm.put(9,"nine");
        System.out.println(hm); // {1=one, 6=six, 9=nine}

        hm.put(1,"gogo");
        System.out.println(hm); // {1=gogo, 6=six, 9=nine}

        hm.put("seven",7);
    }
}
```

```

        System.out.println(hm); // {1=gogo, 6=six, 9=nine, seven=7}

        hm.put(null,null);
        System.out.println(hm); // {null=null, 1=gogo, 6=six,
9=nine, seven=7}
    }
}

```

LinkedHashMap

=====

It is a child class of HashMap class.

LinkedHashMap is exactly same as HashMap class with following differences.

HashMap

The underlying data structure is
Hashtable.

LinkedHashMap

The underlying data structure is
Hashtable and LinkedList.

Insertion order is not preserved.

Insertion order is preserved.

It is introduced in 1.2 version. It is introduced in 1.4 version.

ex:

```

import java.util.*;
class Test
{

```

```

    public static void main(String[] args)
    {

```

```

        LinkedHashMap lhm = new LinkedHashMap();

```

```

        lhm.put(5,"five");

```

```

        lhm.put(1,"one");

```

```

        lhm.put(10,"ten");

```

```

        System.out.println(lhm); // {5=five, 1=one, 10=ten}

```

```

        lhm.put("four",4);

```

```

        System.out.println(lhm); // {5=five, 1=one, 10=ten, four=4}

```

```

        lhm.put(1,"gogo");

```

```

        System.out.println(lhm); // {5=five, 1=gogo, 10=ten,

```

```

four=4}

```

```

        lhm.put(null,null);

```

```

        System.out.println(lhm); // {5=five, 1=gogo, 10=ten,

```

```

four=4, null=null}

```

```

    }

```

```

}

```

TreeMap

=====

The underlying data structure is RED BLACK TREE.

Key can't be duplicate but value can be duplicate.

Insertion order is not preserved because it takes sorting order of the key.

If we depend upon default natural sorting order then keys can be homogeneous or Comparable.

If we depend customized sorting order then keys can be heterogeneous or Non-Comparable.

Key can't be null but value can be null.

```
ex:
---
import java.util.*;
class Test
{
    public static void main(String[] args)
    {
        TreeMap<Integer,String> tm = new TreeMap<>();
        tm.put(5,"five");
        tm.put(1,"one");
        tm.put(10,"ten");
        System.out.println(tm);//{1=one, 5=five, 10=ten}

        tm.put(1,"gogo");
        System.out.println(tm);//{1=gogo, 5=five, 10=ten}

        tm.put(4,null);
        System.out.println(tm); //{1=gogo, 4=null, 5=five, 10=ten}

        //tm.put(null,"six");
        //System.out.println(tm); // R.E NullPointerException
    }
}
```

Hashtable
=====

The underlying data structure is Hashtable.

Key can't be duplicate but value can be duplicate.

Insertion order is not preserved because it takes descending order of the key.

Heterogeneous objects are allowed for both key and value.

Null insertion is not possible for both key and value.

```

ex:
---
import java.util.*;
class Test
{
    public static void main(String[] args)
    {
        Hashtable ht = new Hashtable();
        ht.put(5,"five");
        ht.put(1,"one");
        ht.put(10,"ten");
        System.out.println(ht);//{10=ten, 5=five, 1=one}

        ht.put(1,"gogo");
        System.out.println(ht);//{10=ten, 5=five, 1=gogo}

        ht.put("six",6);
        System.out.println(ht);//{10=ten, six=6, 5=five, 1=gogo}

        //ht.put(4,null);
        //System.out.println(ht);// R.E NullPointerException

        //ht.put(null,"six");
        //System.out.println(ht); // R.E NullPointerException
    }
}

```

Interview Questions

=====

Q) What is difference between Comparable and Comaparator interface?

Comparable

Comparable interface present in java.lang package.

Comparbale interface contains only one method i.e compareTo() method.

If we depend upon default natural sorting order then we need to use Comparable interface.

ex:

```
obj1.compareTo(obj2)
```

It returns -ve if obj1 comes before obj2

It returns +ve if obj1 comes after obj2

It returns 0 if both objects are same

ex:

```

class Test
{
    public static void main(String[] args)
    {
        System.out.println("A".compareTo("Z")); // -25
    }
}

```

```

        System.out.println("Z".compareTo("A")); // +25

        System.out.println("K".compareTo("K")); // 0
    }
}

```

Comparator

Comparator interface present in java.util package.

Comparator interface contains two methods i.e compare() and equals() method.

If we depend upon customized sorting order then we need to use Comparator interface.

While implementing Comparator interface we need to override only compare() method.

ex:

```
public int compare(Object obj1, Object obj2)
```

It returns +ve if obj1 comes before obj2

It returns -ve if obj1 comes after obj2

It returns 0 if both objects are same

Implementation of equals() method is not mandatory because it is present in Object class which is available to the class through inheritance.

ex:

```

import java.util.*;
class Test
{
    public static void main(String[] args)
    {
        TreeSet<Integer> ts = new TreeSet<>(new MyComparator());
        ts.add(1);
        ts.add(10);
        ts.add(5);
        ts.add(3);
        System.out.println(ts); // [10, 5, 3, 1]
    }
}
class MyComparator implements Comparator
{
    public int compare(Object obj1, Object obj2)
    {
        Integer i1 = (Integer)obj1;

```

```

        Integer i2 = (Integer)obj2;
        if(i1<i2)
            return 1;
        else if(i1>i2)
            return -1;
        else
            return 0;
    }
}

ex:
---
import java.util.*;
class Test
{
    public static void main(String[] args)
    {
        TreeSet<Integer> ts = new TreeSet<>(new MyComparator());
        ts.add(1);
        ts.add(10);
        ts.add(5);
        ts.add(3);
        System.out.println(ts); // [1, 3, 5, 10]
    }
}
class MyComparator implements Comparator
{
    public int compare(Object obj1, Object obj2)
    {
        Integer i1 = (Integer)obj1;
        Integer i2 = (Integer)obj2;
        if(i1<i2)
            return -1;
        else if(i1>i2)
            return 1;
        else
            return 0;
    }
}

```

Q) Write a java program to compare two dates?

```

import java.time.*;
class Test
{
    public static void main(String[] args)
    {
        LocalDate date1 = LocalDate.of(2026, 7, 27);
        LocalDate date2 = LocalDate.now();
    }
}

```

```

        if(date1.compareTo(date2)>0)
            System.out.println("date1 is greatest");
        else if(date1.compareTo(date2)<0)
            System.out.println("date2 is greatest");
        else
            System.out.println("Both are same");
    }
}

```

Q) Write a java program to find out number of occurrence of a word in a given string?

input:
this is is java java class

output:
this=1 is=2 java=2 class=1

```

import java.util.*;
class Test
{
    public static void main(String[] args)
    {
        String str="this is is java java class";
        String[] sarr = str.split(" ");

        Map<String,Integer> map = new LinkedHashMap<>();

        for(String s : sarr)
        {
            if(map.containsKey(s))
            {
                map.put(s,map.get(s)+1);
            }
            else
            {
                map.put(s,1);
            }
        }

        for(Map.Entry<String,Integer> entry : map.entrySet())
        {
            System.out.print(entry.getKey()+"="+entry.getValue()+"
");
        }
    }
}

```

Q) Write a java program to find out number of occurrence of a character in a given string?

input:

java

output:

j=1 a=2 v=1

```
import java.util.*;
class Test
{
    public static void main(String[] args)
    {
        String str="java";
        char[] carr = str.toCharArray();

        Map<Character,Integer> map = new LinkedHashMap<>();

        for(char ch : carr)
        {
            if(map.containsKey(ch))
            {
                map.put(ch,map.get(ch)+1);
            }
            else
            {
                map.put(ch,1);
            }
        }

        for(Map.Entry<Character,Integer> entry : map.entrySet())
        {
            System.out.print(entry.getKey()+"="+entry.getValue()+"
");
        }
    }
}
```

Types of cursors in java

=====

Cursor is used to read the objects one by one from Collections.

We have three types of cursors.

- 1) Enumeration
- 2) Iterator
- 3) ListIterator

1) Enumeration

It is used to read the objects one by one from legacy Collection objects.

We can create Enumeration object as follow.

ex:

```
Enumeration e = v.elements();
```

Enumeration interface contains following two methods.

ex:

```
public boolean hasMoreElements()  
public Object nextElement()
```

ex:

```
import java.util.*;  
class Test  
{  
    public static void main(String[] args)  
    {  
        Vector v = new Vector();  
        for(int i=1;i<=10;i++)  
        {  
            v.add(i);  
        }  
  
        Enumeration e = v.elements();  
        while(e.hasMoreElements())  
        {  
            Integer i =(Integer) e.nextElement();  
            System.out.println(i);  
        }  
    }  
}
```

Limitations of Enumeration

Enumeration is used to read objects only from legacy Collection object. Hence it is not a universal cursor.

Using Enumeration we can perform read operation but not remove operation.

To overcome this limitation , Sun Micro System introduced Iterator.

2) Iterator

It is used to read objects one by one from any Collection object. Hence it is a universal cursor.

Using Iterator we can perform read and remove operations.

We can create Iterator object as follow.

ex:

```
Iterator itr = al.iterator();
```

Iterator interface contains following three methods.

ex:

```
public boolean hasNext()  
public Object next()  
public void remove()
```

ex:

--

```
import java.util.*;  
class Test  
{  
    public static void main(String[] args)  
    {  
        ArrayList al = new ArrayList();  
        for(int i=1;i<=10;i++)  
        {  
            al.add(i);  
        }  
  
        Iterator itr = al.iterator();  
        while(itr.hasNext())  
        {  
            Integer i = (Integer) itr.next();  
            if(i%2==0)  
                System.out.println(i);  
            else  
                itr.remove();  
        }  
  
        System.out.println(al); // [2,4,6,8,10]  
    }  
}
```

Limitation with Iterator

Enumeration and Iterator is used to read objects only in forward direction but not backward direction. Hence they are not bi-directional cursors.

Using Iterator we can perform read, remove but we can't perform adding and replacement of new objects.

To overcome this limitation Sun Micro System introduced ListIterator.

3) ListIterator

It is a child interface of Iterator interface.

If we want to read objects one by one from List Collection objects then we need to use ListIterator.

Using ListIterator we can perform read, remove ,adding and replacement of new objects.

We can create ListIterator object as follow.

ex:

```
ListIterator litr = al.listIterator();
```

ListIterator interface contains following 9 methods.

ex:

```
public boolean hasNext()
public Object next()
public int nextIndex()
public boolean hasPrevious()
public Object previous()
public int previousIndex()
public void remove()
public void add(E)
public void set(E)
```

ex:

```
import java.util.*;
class Test
{
    public static void main(String[] args)
    {
        ArrayList al = new ArrayList();
        al.add("chiru");
        al.add("venki");
        al.add("bala");
        al.add("nag");

        ListIterator litr = al.listIterator();
        while(litr.hasNext())
        {
            String s = (String) litr.next();
            System.out.println(s);
        }
    }
}
```

ex:

```
import java.util.*;
class Test
{
    public static void main(String[] args)
```

```

    {
        ArrayList al = new ArrayList();
        al.add("chiru");
        al.add("venki");
        al.add("bala");
        al.add("nag");

        ListIterator litr = al.listIterator(al.size());
        while(litr.hasPrevious())
        {
            String s = (String) litr.previous();
            System.out.println(s);
        }
    }
}

```

ex:

```

import java.util.*;
class Test
{
    public static void main(String[] args)
    {
        ArrayList al = new ArrayList();
        al.add("chiru");
        al.add("venki");
        al.add("bala");
        al.add("nag");

        ListIterator litr = al.listIterator();
        while(litr.hasNext())
        {
            String s = (String) litr.next();
            if(s.equals("bala"))
            {
                litr.remove();
            }
        }

        System.out.println(al); // [chiru, venki, nag]
    }
}

```

ex:

--

```

import java.util.*;
class Test
{
    public static void main(String[] args)
    {
        ArrayList al = new ArrayList();
        al.add("chiru");
    }
}

```

```

        al.add("venki");
        al.add("bala");
        al.add("nag");

        ListIterator litr = al.listIterator();
        while(litr.hasNext())
        {
            String s = (String) litr.next();
            if(s.equals("venki"))
            {
                litr.add("rana");
            }
        }

        System.out.println(al); // [chiru, venki, rana, bala, nag]
    }
}

```

ex:

```

import java.util.*;
class Test
{
    public static void main(String[] args)
    {
        ArrayList al = new ArrayList();
        al.add("chiru");
        al.add("venki");
        al.add("bala");
        al.add("nag");

        ListIterator litr = al.listIterator();
        while(litr.hasNext())
        {
            String s = (String) litr.next();
            if(s.equals("chiru"))
            {
                litr.set("pavan");
            }
        }

        System.out.println(al); // [pavan, venki, bala, nag]
    }
}

```

Multithreading
=====

Q) What is the difference between Thread and Process?

Thread

A thread is a leight weight sub-process.
We can run multiple threads concurently.
One thread can communicate with another thread.

ex:

 showing notifications
 Compose
 Attachment to mail
 searching for mail
 and etc.

Process

A process is a collection of threads.
We can run multiple processes concurently.
One process can't communicate with another process because it is independent
to each other.

ex:

 Gmail

MultiTasking

=====

Executing several tasks simlutenously such concept is called multitasking.

We have two types of multitasking.

1) Thread based multitasking

It is best suitable for programmatic level.
Executing several tasks simlutenously where each task is a same part of a program.

2) Process based multitasking

It is best suitable for OS level.
Executing several tasks simlutenously where eask task is a independent process.

Multithreading

=====

Executing several threads simlutenously such concept is called multithreading.

Multithreading is a feature in Java where multiple threads execute simultaneously to improve performance and responsiveness.

In multithreading only 10% of work should be done by a programmer and 90% of work will be done by JAVA API.

The main important application area of multithreading are

- 1) To implements multi-media graphics
 - 2) To develop video games
 - 3) To develop animations
- and etc.

Ways to create a thread in java

=====

There are two ways to create a thread in java.

- 1) By extending Thread class
- 2) By implementing Runnable interface

- 1) By extending Thread class

```
-----  
class MyThread extends Thread  
{  
    //work of a thread  
    public void run()  
    {  
        for(int i=1;i<=5;i++)  
        {  
            System.out.println("Child-Thread");  
        }  
    }  
}  
class Test  
{  
    public static void main(String[] args)  
    {  
        //instantiate a thread  
        MyThread t = new MyThread();  
  
        //start a thread  
        t.start();  
  
        for(int i=1;i<=5;i++)  
        {  
            System.out.println("Parent-Thread");  
        }  
    }  
}
```

Case1 : Thread Scheduler

If multiple threads are waiting for execution which thread has to be executed will be decided by the thread scheduler.

What algorithm, behaviour or mechanism used by thread scheduler depends upon JVM vendor.

Hence we can't expect any execution order or exact output in multithreading.

case2: What is the difference between `t.start()` and `t.run()` method

If we invoke `t.start()` method a new thread will be created which is responsible to execute `run()` method automatically.

ex:

```
--
class MyThread extends Thread
{
    //work of a thread
    public void run()
    {
        for(int i=1;i<=5;i++)
        {
            System.out.println("Child-Thread");
        }
    }
}
class Test
{
    public static void main(String[] args)
    {
        //instantiate a thread
        MyThread t = new MyThread();

        //start a thread
        t.start();

        for(int i=1;i<=5;i++)
        {
            System.out.println("Parent-Thread");
        }
    }
}
```

If we invoke `t.run()` method, no new thread will be created but `run()` method will execute just like a normal method.

ex:

```
class MyThread extends Thread
{
```



```

        public void run()
        {
            for(int i=1;i<=5;i++)
            {
                System.out.println("Child-Thread");
            }
        }
    }
class Test
{
    public static void main(String[] args)
    {
        //instantiate a thread
        MyThread t = new MyThread();

        //No new thread
        t.run();

        for(int i=1;i<=5;i++)
        {
            System.out.println("Parent-Thread");
        }
    }
}

```

Case3: If we won't override run() method

If we won't override run() method then t.start() method executes Thread class run() method automatically.

Thread class run() method is empty implementation. Hence we won't get any output from child thread.

ex:

--

```

class MyThread extends Thread
{
}
class Test
{
    public static void main(String[] args)
    {
        //instantiate a thread
        MyThread t = new MyThread();

        //start a thread
        t.start();

        for(int i=1;i<=5;i++)

```

```

        {
            System.out.println("Parent-Thread");
        }
    }
}

```

case 4: If we override run() method

If we override run() method then t.start() method always execute run() method with zero parameters only.

ex:

```

class MyThread extends Thread
{
    public void run()
    {
        System.out.println("0-arg method");
    }
    public void run(int i)
    {
        System.out.println("int-arg method");
    }
}
class Test
{
    public static void main(String[] args)
    {
        //instantiate a thread
        MyThread t = new MyThread();

        //start a thread
        t.start();

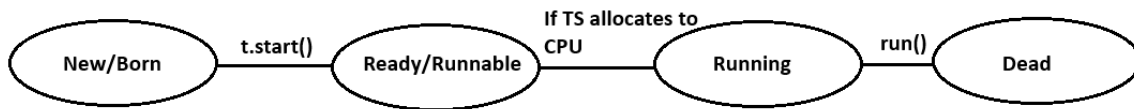
        for(int i=1;i<=5;i++)
        {
            System.out.println("Parent-Thread");
        }
    }
}

```

Case5 : Life cycle of a thread

Diagram: class48.1

```
//instantiate a thread
MyThread t = new MyThread();
```



- 1) Once if we create a thread, our thread will be in new or born state.
- 2) Once if we call t.start() method then our thread will be in ready or runnable state.
- 3) If thread scheduler allocates to CPU then our thread enters to running state.
- 4) Once run() method execution is completed then our thread goes to dead state.

2) By implementing Runnable interface

```
-----
class MyRunnable implements Runnable
{
    public void run()
    {
        for(int i=1;i<=5;i++)
        {
            System.out.println("Child-Thread");
        }
    }
}
class Test
{
    public static void main(String[] args)
    {
        MyRunnable r = new MyRunnable();
    }
}
```

```

        Thread t = new Thread(r); // r is a targetable interface
        t.start();

        for(int i=1;i<=5;i++)
        {
            System.out.println("Parent-Thread");
        }
    }
}

```

Setting and Getting name of a thread

=====

In java, every thread has a name automatically generated by JVM or explicitly provided by the programmer.

We have following methods to set and get name of a thread.

ex:

```

    public final void setName(String name)
    public final String getName()

```

ex:

```

class MyThread extends Thread
{

```

```

}

```

```

class Test
{

```

```

    public static void main(String[] args)
    {

```

```

        System.out.println(Thread.currentThread().getName()); //
main

```

```

        MyThread t = new MyThread();
        System.out.println(t.getName()); // Thread-0

```

```

        Thread.currentThread().setName("Parent-Thread");
        System.out.println(Thread.currentThread().getName()); //
Parent-Thread

```

```

        t.setName("Child-Thread");
        System.out.println(t.getName()); // Child-Thread
    }
}

```

Thread Priority

=====

In java, every thread has a priority automatically generated by JVM or explicitly provided by the programmer.

The valid range of thread priority is 1 to 10. Where 1 is a least priority and 10 is a highest priority.

If we take more than 10 priority then we will get runtime exception called `IllegalArgumentException`.

A `Thread` class defines following standard constants as thread priority.

ex:

```
Thread.MAX_PRIORITY - 10
Thread.MIN_PRIORITY - 1
Thread.NORM_PRIORITY - 5
```

We don't have such constants like `LOW_PRIORITY` and `HIGH_PRIORITY`.

A thread which is having highest priority will be executed first.

If multiple threads having same priority then we can't expect any execution order.

Thread Scheduler uses thread priority while allocating to CPU.

We have following methods to set and get thread priorities.

ex:

```
public final void setPriority(int priority)
public final int getPriority()
```

ex:

```
class MyThread extends Thread
{
}
class Test
{
    public static void main(String[] args)
    {
        System.out.println(Thread.currentThread().getPriority());
// 5

        MyThread t = new MyThread();
        System.out.println(t.getPriority()); // 5

        Thread.currentThread().setPriority(10);
        System.out.println(Thread.currentThread().getPriority());
// 10

        t.setPriority(4);
        System.out.println(t.getPriority()); // 4

        t.setPriority(11); // R.E IllegalArgumentException
```

```
    }  
}
```

Daemon thread
=====

It is a leight weight thread which runs in a background.

It provides services to user threads.

Life of deamon thread is depend upon user threads because when user threads died then daemon thread will die automatically.

There are many daemon threads are running internally.

ex:

```
Garbage Collector  
Finalizer  
Signal Dispatch  
and etc.
```

We can change the behaviour of daemon thread before user threads. IF we change behaviour of daemon thread after user threads then we will get runtime exception called `IllegalThreadStateException`.

To change the behaviour of daemon thread we need to use `setDaemon(true)` method.

To check a thread is daemon or not we need to use `isDaemon()` method.

ex:

```
class MyThread extends Thread  
{  
    public void run()  
    {  
        for(int i=1;i<=5;i++)  
        {  
            System.out.println(Thread.currentThread().isDaemon());  
            System.out.println("Child-Thread");  
        }  
    }  
}  
class Test  
{  
    public static void main(String[] args)  
    {  
        MyThread t = new MyThread();  
        t.setDaemon(true);  
        t.start();  
        for(int i=1;i<=5;i++)  
        {  
            System.out.println("Parent-Thread");  
        }  
    }  
}
```

```
    }  
}
```

Thread Pool

=====

Thread pool is a part of java.util.concurrent package.

Thread pool is a pre-initialized worker threads which used to handle multiple tasks concurrently.

Instead of creating new thread for each task. We can submit the task to thread pool which contains fixed number of threads to handle multiple tasks effeciently.

Advantages:

1) Improves performance

It avoids overhead creation of thread for each task.

2) Memory Resource

It control all concurrent threads to improve system performance.

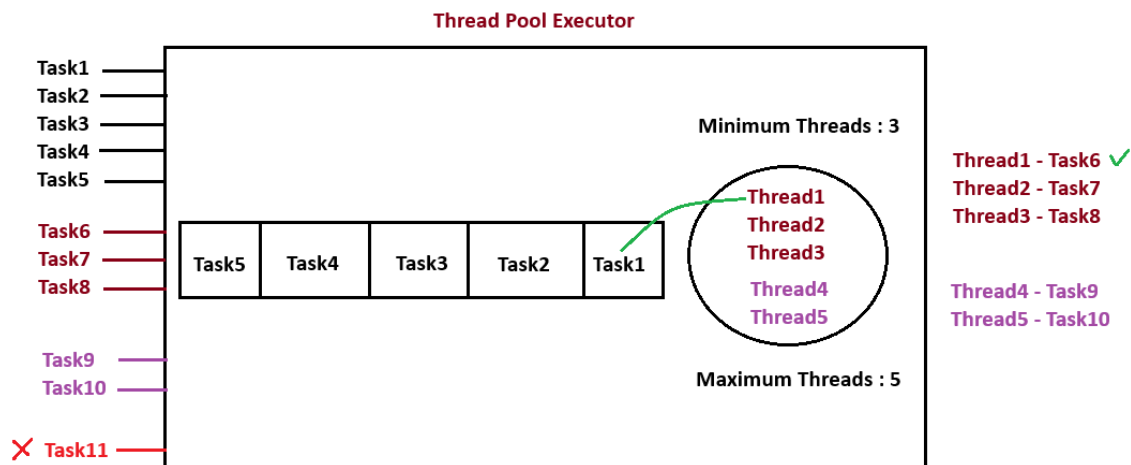
3) Scalability

It handles all short-lived threads.

4) Better Control

It can be queued, processed and cancel.

Diagram: class49.1



```
ex:
---
import java.util.concurrent.*;
import java.io.*;
class Test
{
    public static void main(String[] args)
    {
        ThreadPoolExecutor executor =
            new ThreadPoolExecutor(3,5,10,TimeUnit.MINUTES,
                new ArrayBlockingQueue<>(5),
                Executors.defaultThreadFactory(),
                new ThreadPoolExecutor.DiscardOldestPolicy());

        for(int i=1;i<=10;i++)
        {
            executor.submit(()->
            {
                System.out.println("Task Executed :
"+Thread.currentThread().getName());
            });
        }
        executor.shutdown();
    }
}
```

Problem without synchronization

=====

If there is no synchronization then we will face following problems.

1) Data inconsistency

2) Thread interference

ex:

```
class Table
{
    void printTable(int n)
    {
        for(int i=1;i<=5;i++)
        {
            System.out.println(n*i);
            try
            {
                Thread.sleep(2000);
            }
            catch (InterruptedException ie)
            {
                ie.printStackTrace();
            }
        }
    }
}

class MyThread1 extends Thread
{
    Table t;
    MyThread1(Table t)
    {
        this.t=t;
    }
    public void run()
    {
        t.printTable(5);
    }
}

class MyThread2 extends Thread
{
    Table t;
    MyThread2(Table t)
    {
        this.t=t;
    }
    public void run()
    {
        t.printTable(10);
    }
}

class Test
{
    public static void main(String[] args)
    {
```

```

        Table obj = new Table();
        MyThread1 t1 = new MyThread1(obj);
        MyThread2 t2 = new MyThread2(obj);
        t1.start();
        t2.start();
    }
}

```

Synchronization

=====

A synchronized keyword is applicable for methods and blocks.

A synchronization allows only one thread to execute at a time.

The main advantage of synchronornization is we can achieve thread safety.

The main disadvantage of synchronization is waiting time of a thread will increase which effects the performance.

In reallife synchronization concept used in

ex:

- whatsapp messages
- Ticket booking
- bank transaction
- and etc.

Synchronization deals with lock mechanism.

Whenever a thread wants to access any object, first it has to acquire the lock of it and release it when thread completes it's task.

In synchronized method , threads will get the lock automatically.

When one thread is executing synchronized method then other threads are not allowed to execute other synchronized methods. But they can execute non-synchronized methods concurrently.

If there is a special requirement then only we need to use synchronization concept.

ex:

```

class Table
{
    synchronized void printTable(int n)
    {
        for(int i=1;i<=5;i++)
        {
            System.out.println(n*i);
            try

```

```

        {
            Thread.sleep(2000);
        }
        catch (InterruptedException ie)
        {
            ie.printStackTrace();
        }
    }
}
class MyThread1 extends Thread
{
    Table t;
    MyThread1(Table t)
    {
        this.t=t;
    }
    public void run()
    {
        t.printTable(5);
    }
}
class MyThread2 extends Thread
{
    Table t;
    MyThread2(Table t)
    {
        this.t=t;
    }
    public void run()
    {
        t.printTable(10);
    }
}
class Test
{
    public static void main(String[] args)
    {
        Table obj = new Table();
        MyThread1 t1= new MyThread1(obj);
        MyThread2 t2= new MyThread2(obj);
        t1.start();
        t2.start();
    }
}

```

synchronized block
=====

If we want to perform synchronization on specific resource of a program then we need to use synchronized block.

ex:

If we have 100 line of code. But we want to perform synchronization only for 5 lines of code then we need to use synchronized block.

If we keep all the logics in synchronized block then it acts like a synchronized method.

ex:

```
class Table
{
    void printTable(int n)
    {
        synchronized(this)
        {
            for(int i=1;i<=5;i++)
            {
                System.out.println(n*i);
                try
                {
                    Thread.sleep(2000);
                }
                catch (InterruptedException ie)
                {
                    ie.printStackTrace();
                }
            }
        } //sync
    }
}

class MyThread1 extends Thread
{
    Table t;
    MyThread1(Table t)
    {
        this.t=t;
    }
    public void run()
    {
        t.printTable(5);
    }
}

class MyThread2 extends Thread
{
    Table t;
    MyThread2(Table t)
    {
```

```

        this.t=t;
    }
    public void run()
    {
        t.printTable(10);
    }
}
class Test
{
    public static void main(String[] args)
    {
        Table obj = new Table();
        MyThread1 t1 = new MyThread1(obj);
        MyThread2 t2 = new MyThread2(obj);
        t1.start();
        t2.start();
    }
}

```

static synchronization
=====

If we declare any static method as synchronized then it is called static synchronization.

In static synchronization the lock will be on class but not on object.

ex:

```

class Table
{
    static synchronized void printTable(int n)
    {
        for(int i=1;i<=5;i++)
        {
            System.out.println(n*i);
            try
            {
                Thread.sleep(2000);
            }
            catch (InterruptedException ie)
            {
                ie.printStackTrace();
            }
        }
    }
}

```

```

class MyThread1 extends Thread
{
    public void run()
    {
        Table.printTable(5);
    }
}

```

```

    }
}
class MyThread2 extends Thread
{
    public void run()
    {
        Table.printTable(10);
    }
}
class Test
{
    public static void main(String[] args)
    {
        MyThread1 t1 = new MyThread1();
        MyThread2 t2 = new MyThread2();
        t1.start();
        t2.start();
    }
}

```

DeadLock in Java

=====

Deadlock will occur in a situation where one thread is waiting for object lock which is acquired by another thread and that thread is waiting for object lock which is acquired by first thread. Here both threads are waiting to release the lock but nobody will released such situation is called deadlock.

ex:

```

class Test
{
    public static void main(String[] args)
    {
        final String res1="Hi";
        final String res2="Bye";

        Thread t1 = new Thread()
        {
            public void run()
            {
                synchronized(res1)
                {
                    System.out.println("Thread1 : Locking
Resource1");

                    synchronized(res2)
                    {
                        System.out.println("Thread1 : Locking
Resource2");

                    }
                }
            }
        }
    }
}

```

```

        }
    };
    Thread t2 = new Thread()
    {
        public void run()
        {
            synchronized(res2)
            {
                System.out.println("Thread2 : Locking
Resource2");
            }
            synchronized(res1)
            {
                System.out.println("Thread2 : Locking
Resource1");
            }
        }
    };
    t1.start();
    t2.start();
}
}

```

Various ways to prevent a thread from execution

=====

There are three ways to prevent a thread from execution.

- 1) yield()
- 2) join()
- 3) sleep()

1) yield()

It pause current execution thread and gives the chance to other threads having same priority.

If there is no waiting threads or low priority threads then same thread will continue it's execution.

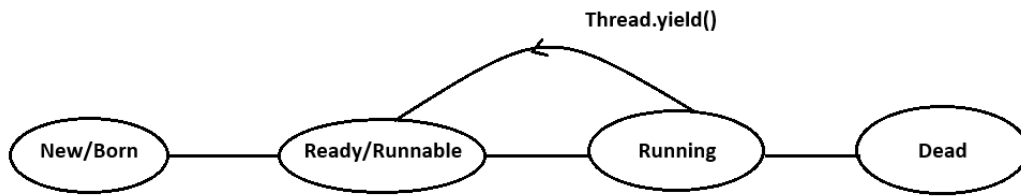
If multiple threads having same priority then we can't expect any execution order.

A thread which is yielded when it get a chance for execution is depend upon mercy of thread scheduler.

ex:

```
public static native void yield()
```

Diagram: class50.1



ex:

```

class MyThread extends Thread
{
    public void run()
    {
        for(int i=1;i<=5;i++)
        {
            Thread.currentThread().yield();
            System.out.println("child-thread");
        }
    }
}
class Test
{
    public static void main(String[] args)
    {
        MyThread t = new MyThread();
        t.start();
        for(int i=1;i<=5;i++)
        {
            System.out.println("parent-thread");
        }
    }
}

```

2) join()

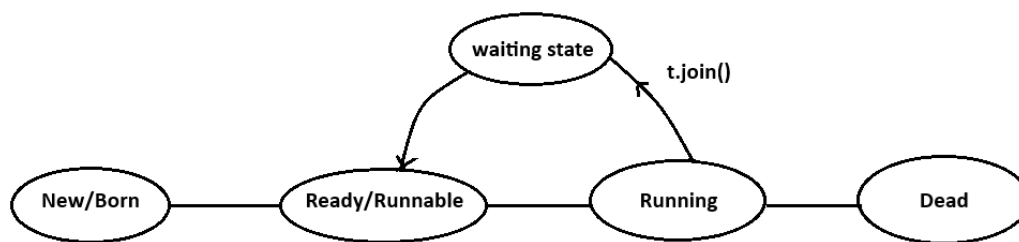
If a thread wants to wait untill the completion of some other threads then we need to use join() method.

A join method will throw one checked exception called InterruptedException so we must and should handle that exception by using try and catch block or by using throws statement.

ex:

```
public final void join()throws InterruptedException
public final void join(long ms)throws InterruptedException
public final void join(long ms,int ns)throws InterruptedException
```

Diagram: class50.2



ex:

```
class MyThread extends Thread
{
    public void run()
    {
        for(int i=1;i<=5;i++)
        {
            System.out.println("child-thread");
        }
    }
}
class Test
{
    public static void main(String[] args)throws InterruptedException
    {
        MyThread t = new MyThread();
        t.start();
        t.join();
        for(int i=1;i<=5;i++)
        {
            System.out.println("parent-thread");
        }
    }
}
```

```

    }
}

```

3) sleep()

If a thread don't want to perform any operation on perticular amount of time then we need to use sleep() method.

A sleep() method will throw one checked exception called InterruptedException so we must and should handle that exception by using try and catch block or by using throws statement.

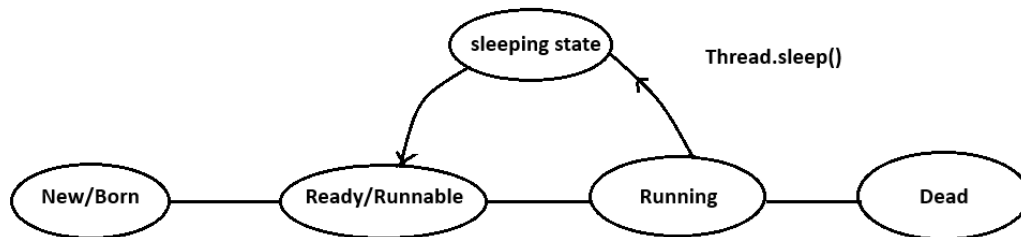
ex:

```

    public static native void sleep()throws InterruptedException
    public static native void sleep(long ms)throws
InterruptedException
    public static native void sleep(long ms,int ns)throws
InterruptedException

```

Diagram: class50.3



```

class MyThread extends Thread
{
    public void run()
    {
        for(int i=1;i<=5;i++)
        {
            System.out.println("child-thread");
            try
            {
                Thread.sleep(2000);
            }
        }
    }
}

```

```

        catch (InterruptedException ie)
        {
            ie.printStackTrace();
        }
    }
}
class Test
{
    public static void main(String[] args)
    {
        MyThread t = new MyThread();
        t.start();
        for(int i=1;i<=5;i++)
        {
            System.out.println("parent-thread");
        }
    }
}

```

Inter-Thread communication

=====

Inter-thread communication possible by using wait(), notify() and notifyAll() method.

A wait(), notify() and notifyAll() method present in Object class.

A thread which is waiting for notification has to call wait() method.

A thread which is giving notification has to call notify() or notifyAll() method.

To call wait(), notify() and notifyAll() method we required synchronized area.

If we call wait() method then thread will release the lock immediately and goes to waiting state.

If we call notify() or notifyAll() method then thread will release the lock but not immediately.

Except wait(), notify() and notifyAll() method , there is no way to release the lock.

ex:

```

class MyThread extends Thread
{
    int total = 0;

```

```

        public void run()
        {
            synchronized(this)
            {
                System.out.println("Child started calculation");

                for(int i=1;i<=10;i++)
                {
                    total+=i;
                }

                System.out.println("Child giving notification");
                this.notify();
            }
        }
    }
}
class Test
{
    public static void main(String[] args) throws InterruptedException
    {
        MyThread t = new MyThread();
        t.start();
        synchronized(t)
        {
            System.out.println("Main method is waiting");
            t.wait();
            System.out.println("Main method got notification");
            System.out.println(t.total);
        }
    }
}

```

Java 8 Features

=====

Functional interface

=====

Functional interface introduced in Java 8.

Interface which contains only one abstract method is called functional interface.

It can have any number of default methods, static methods and private methods.

It is also known as SAM or Single Abstract Method interface.

We have following list of functional interfaces in java.

ex:

Comparable - compareTo()

```
Runnable    - run()
ActionListener - actionPerformed()
and etc.
```

The main objective of functional interface is to achieve functional programming.

ex:

```
a = f1() {}

f1(f2() {} )
{
}
```

@FunctionalInterface annotation is used to declare functional interface which is optional.

ex:

```
@FunctionalInterface
interface ATM
{
    public abstract void deposit();
}
class ATMImpl implements ATM
{
    @Override
    public void deposit()
    {
        System.out.println("Deposit-Method");
    }
}
class Test
{
    public static void main(String[] args)
    {
        ATM atm = new ATMImpl();
        atm.deposit();
    }
}
```

ex:

```
@FunctionalInterface
interface ATM
{
    public abstract void deposit();
}
class Test
{
    public static void main(String[] args)
    {
```

```

        //anonymous inner classs
        ATM atm = new ATM()
        {
            public void deposit()
            {
                System.out.println("Deposit Method");
            }
        };
        atm.deposit();
    }
}

```

Lamda Expression

=====

Lamda expression introduced in Java 8.

We can use lamda expression when we have functional interface.

Lamda expression is used to concise the code.

Lamda expression consider as method.

The main objective of lamda expression is to achieve functional programming.

Lamda expression does not allow name, returntype and modifier.

ex:

```

Java Method
-----
public void m1()
{
    System.out.println("M1-Method");
}

```

```

Lamda Expression
-----
() ->
{
    System.out.println("M1-Method");
};

```

ex:

```

@FunctionalInterface
interface ATM
{
    public abstract void deposit();
}
class Test
{

```

```

    public static void main(String[] args)
    {
        ATM atm = ()->{
            System.out.println("My Deposit Method");
        };

        atm.deposit();
    }
}

ex:
---
@FunctionalInterface
interface ATM
{
    public abstract void deposit(int amount);
}
class Test
{
    public static void main(String[] args)
    {
        ATM atm = (int amount)->{
            System.out.println(amount);
        };

        atm.deposit(10000);
    }
}

ex:
---
@FunctionalInterface
interface ATM
{
    public abstract String deposit(int amount);
}
class Test
{
    public static void main(String[] args)
    {
        ATM atm = (int amount)->{
            return "Balance :"+amount;
        };

        System.out.println(atm.deposit(10000));
    }
}

```

Default methods in interface

=====

Default methods in interface introduced in Java 8.

Java provides facility to declare methods in interface and tagged with default keyword.

It is a non-abstract method.

It can be override.

ex:

```
interface Animal
{
    //abstract method
    public abstract void sound();

    //default method
    default void eat()
    {
        System.out.println("Animal Eat");
    }
}
class Dog implements Animal
{
    @Override
    public void sound()
    {
        System.out.println("Bow Bow");
    }
}
class Test
{
    public static void main(String[] args)
    {
        Animal a = new Dog();
        a.eat();
        a.sound();
    }
}
```

ex:

```
interface Animal
{
    //abstract method
    public abstract void sound();

    //default method
    default void eat()
    {
        System.out.println("Animal Eat");
    }
}
```



```

class Dog implements Animal
{
    @Override
    public void sound()
    {
        System.out.println("Bow Bow");
    }

    @Override
    public void eat()
    {
        System.out.println("Dog Eat");
    }
}
class Test
{
    public static void main(String[] args)
    {
        Animal a = new Dog();
        a.eat();
        a.sound();
    }
}

```

Static methods in interface
=====

Static methods in interface introduced in Java 8.

Java provides facility to declare methods in interface and tagged with static keyword.

It is a non-abstract method.

It can't be override.

ex:

```

interface Animal
{
    //static method
    static void sound()
    {
        System.out.println("Animal Makes Sound");
    }
}
class Test
{
    public static void main(String[] args)
    {
        Animal.sound();
    }
}

```

Stream API =====

Stream API introduced in Java 8.

Stream API present in java.util.stream package.

It allows functional style programming and makes our code simple and easy.

It is used to process the objects/data.

It is used to perform bulk operations on collections.

ex:

```
import java.util.*;
import java.util.stream.*;
class Test
{
    public static void main(String[] args)
    {
        List<Integer> list = List.of(6,9,1,4,3,5,2);

        List<Integer> newList = list.stream()
            .map(i-> i + 10).collect(Collectors.toList());

        System.out.println(newList);
    }
}
```

ex:

--

```
import java.util.*;
import java.util.stream.*;
class Test
{
    public static void main(String[] args)
    {
        List<Integer> list = List.of(6,9,1,4,3,5,2);

        List<Integer> newList = list.stream()
            .filter(i-> i%2==0).collect(Collectors.toList());

        System.out.println(newList);
    }
}
```

ex:

```
import java.util.*;
```

```

import java.util.stream.*;
class Test
{
    public static void main(String[] args)
    {
        List<Integer> list = List.of(6,9,1,4,3,5,2);

        long cnt = list.stream()
            .filter(i-> i%2==0).count();

        System.out.println(cnt);
    }
}

```

ex:

```

import java.util.*;
import java.util.stream.*;
class Test
{
    public static void main(String[] args)
    {
        List<Integer> list = List.of(6,9,1,4,3,5,2);

        List<Integer> newList =
list.stream().sorted().collect(Collectors.toList());

        System.out.println(newList);
    }
}

```

ex:

```

import java.util.*;
import java.util.stream.*;
class Test
{
    public static void main(String[] args)
    {
        List<Integer> list = List.of(6,9,1,4,3,5,2);

        List<Integer> newList = list.stream()

.sorted(Comparator.reverseOrder()).collect(Collectors.toList());

        System.out.println(newList);
    }
}

```

ex:

```

import java.util.*;

```

```

import java.util.stream.*;
class Test
{
    public static void main(String[] args)
    {
        List<Integer> list = List.of(6,9,1,4,3,5,2);

        long min = list.stream().min((i1,i2)-
>i1.compareTo(i2)).get();

        System.out.println(min);
    }
}

```

ex:

```

import java.util.*;
import java.util.stream.*;
class Test
{
    public static void main(String[] args)
    {
        List<Integer> list = List.of(6,9,1,4,3,5,2);

        long max = list.stream().max((i1,i2)-
>i1.compareTo(i2)).get();

        System.out.println(max);
    }
}

```

forEach() method

=====

It is introduced in Java 8.

It is used to iterate objects from Collections.

ex:

```

import java.util.*;
class Test
{
    public static void main(String[] args)
    {
        List<Integer> list = List.of(6,9,1,4,3,5,2);

        list.forEach(ele -> System.out.print(ele+" "));
    }
}

```

ex:

```

import java.util.*;
class Test
{
    public static void main(String[] args)
    {
        Set<Integer> set = Set.of(6,9,1,4,3,5,2);

        set.forEach(ele -> System.out.print(ele+" "));
    }
}

ex:
---
import java.util.*;
class Test
{
    public static void main(String[] args)
    {
        Map<Integer,String> map =
Map.of(1,"one",2,"two",3,"three");

        map.forEach((k,v) -> System.out.println(k+"="+v));
    }
}

```

Method Reference (::)
=====

Method reference introduced in Java 8.

It is a special type of lamda expression.

It is a concise way to call existing methods.

```

ex:
---
import java.util.*;
class Test
{
    public static void main(String[] args)
    {
        List<Integer> list = List.of(7,3,6,1,9,2);

        list.forEach(System.out::println);
    }
}

ex:
--
import java.util.*;
class Test
{

```

```

        public static void main(String[] args)
        {
            Map<Integer,String> map =
Map.of(1,"one",2,"two",3,"three");
            map.forEach(Test::entry);

        }
        public static void entry(Integer key,String value)
        {
            System.out.println(key+"="+value);
        }
    }

```

Optional
=====

Optional class introduced in Java 8.

Optional class is used to perform null checks gracefully.

There are three ways to create object for Optional class.

ex:

```

Optional<Object> optional = Optional.empty();
Optional<Object> optional = Optional.of();
Optional<Object> optional = Optional.ofNullable();

```

ex:

```

import java.util.*;
class Test
{
    public static void main(String[] args)
    {
        Optional optional = Optional.empty();
        System.out.println(optional);

    }
}

```

ex:

```

import java.util.*;
class Test
{
    public static void main(String[] args)
    {
        String str="Hi";
        Optional optional = Optional.of(str);
        System.out.println(optional.get());

    }
}

```

```

ex:
---
import java.util.*;
class Test
{
    public static void main(String[] args)
    {
        String str=null;
        Optional optional = Optional.of(str);
        System.out.println(optional.get());

    }
}

```

o/p: R.E NullPointerException

```

ex:
---
import java.util.*;
class Test
{
    public static void main(String[] args)
    {
        String str=null;
        Optional<String> optional = Optional.ofNullable(str);
        System.out.println(optional.orElse("Value Not Found"));

    }
}

```

```

ex:
---
import java.util.*;
class Test
{
    public static void main(String[] args)
    {
        String str="Hi";
        Optional<String> optional = Optional.ofNullable(str);
        System.out.println(optional.orElse("Value Not Found"));

    }
}

```

```

ex:
---
import java.util.*;
class Test
{
    //instance variable
    int i = 10;
}

```

```

public static void main(String[] args)
{
    Test t = null;

    Optional optional = Optional.ofNullable(t);

    if(!optional.isPresent())
    {
        t = new Test();
    }

    System.out.println(t.i);
}
}

```

Access Modifiers:

- An access modifiers specifies who can access them.
- In java, Access modifiers help to restrict the scope of a class constructor , variables , method or a member.

There are four access modifiers used in java.

- Default
- Public
- Private
- Protected

Access modifiers are restricted to two levels.

Level 1 : Class level access modifiers.

Level 2 : Member level access modifiers.

Class level access modifiers:

Only two access modifiers are allowed

- Public
- Default

If a class is public then it can be accessed from anywhere.

If a class is (no modifier) default then it can only be accessed from same package.

Member level access modifiers:

Applied to methods , constructors and fields.

- The private access modifier is specified using the keyword private. The methods or data members declared as private are accessible only within the class.
- The protected access modifier is specified using the keyword private. The methods or data members declared as protected are accessible only within the class and sub classes.

- How to exit from the program?
System.exit(0);

- How to exit from the loops?
Break;
- How to exit from the method?
return;
- Which is destroy object creation?
Garbage collector internally finalizer.
- Natural sorting?
Comparable
- Custom sorting?
Comparator
- Local variables stored in?
Java stack.
- Instance variables stored in?
Heap memory.
- Global variables stored in?
Method area.
- Which application it is Jyo.java and JyoApp.java?
Jyo.java - java and JyoApp.java - JDBC application.
- When we use for loop?
Number of iterations are known.
- When we use while and do while loop?
While - Don't know the exact number of iterations.
Do while - Need to run code exactly once.
- What is main method?
Public static void main(String []args)
Public - Access modifier
Static - class
Void - return type doesn't return any value
Main - method name (Start execution)
String []args - Command line argument

- When abstraction becomes 100% abstraction?
An interface contains:
 - Only abstract methods(no method body).
 - No implementation only method declaration.
- Which application we are developed in java and JDBC?
Java - One-tier (Standalone applications).
JDBC - Two-tier applications.
- When Null pointer exception will raise?
A NULL pointer exception in java occurs when you try to use a method or field on a null reference.
(It occurs when accessing an object with a null reference.)
Eg:
 - 1.calling a method on a null reference
String s = null;
s.length(); // Null pointer exception
 - 2.Accessing a field (variable)of a null object
Student s = null;

```

        System.out.println(s.id); //Null pointer exception
3.String s = null;
        System.out.println(s); // null

```

- What is software?
Software is a collection of instructions, data or programs used to operate computers and execute specific tasks.
- What is compilation?
Compilation is the process of converting high-level source code into bytecode or machine code by a compiler before execution. (converting source code into byte code).
Runtime compiler:
A JIT(just in time) compiler inside the JVM that converts bytecode into native machine code during program execution to improve performance.
- What is the default value of instance variables?
Default values depends upon datatype(0,false,null).
- Can we override static methods?
No, Static methods are hidden , not overridden.
- Can we overload the main method?
Yes, but JVM calls only main method.
- What is byte code?
Byte code is the compiled code executed by the JVM.
- What is constructor?
A special method used to initialize objects.
- What is default constructor?
Constructor with no parameters.
- What is access modifier?
Defines visibility of classes and members.

- What is transient keyword?
Prevents serialization of variables.
- What is volatile keyword?
Ensures visibility of changes across threads.
- What is varargs?
Variable number of arguments.
- What is System.out.println()?
System - A class in java.lang package
Out - A public static object of Print stream(standard output)
Println - A method of Printstream that prints and adds a new line
- What is class loader?
Loads classes into JVM.
- What are types of classloaders?
Bootstrap, Extension, Application.
- What is optional?
Container to avoid null checks.
- What is method reference?
Reference to a method using ::operator.

JAVA FUNDAMENTALS

1. Why java?
2. History of java?
3. Types of modules?
4. Naming conventions?
5. Comments in java?
6. Identifiers?
7. Reserved words?
8. First java application development?
9. Internal architecture of JVM?
10. Data types?
11. Literals?
12. Types of variables?
13. Var-arg method?
14. Main method?
15. Command line argument?
16. System.out.println()?
17. Fully Qualified name?
18. Import statements?
19. Basic java programs?
20. Type casting?
21. Types of Blocks?
22. Java source file structure?
23. Math class?
24. Random class?
25. Access modifiers?
26. Operators?

OPERATORS :

Assignment operators
Ternary operators
Logical operators
Bitwise operators
Arithmetic operators
Relational operators
Shift operators
Unary operators

CONTROL STATEMENTS:

1. Decision making statements
 - i. If
 - ii. If else
 - iii. If else if ladder
 - iv. Nested if
 - v. Switch case
2. Iteration statements
 - i. Do while loop
 - ii. While loop

- iii. For loop
- iv. Loop patterns
- v. For each loop
- 3. Jump statements
 - i. Break
 - ii. Continue
 - iii. Return

JAVA METHODS:

- ❖ No return type with no argument method
- ❖ NO return type with argument method
- ❖ With return type with no argument method
- ❖ With return type with argument method

RECURSION:

- ❖ What is recursion?
- ❖ Programs on recursion.

ARRAYS:

- I. Single dimensional array
- II. Double dimensional array
- III. Multi dimensional array
- IV. Anonymous array
- V. Jagged array

OOPS:

- 1.Oops
- 2.Class
- 3.Object
- 4.Data hiding
- 5.Abstraction
- 6.Encapsulation
- 7.Is-A relationship
- 8.Inheritance
- 9.Has-A relationship
- 10. Composition vs aggreagation
- 11. Method overloading
- 12. Method overriding
- 13. Method hiding
- 14. Polymorphism

CONSTRUCTORS:

- What is constructor?
- Types of constructor?
- Constructor overloading?

THIS KEYWORD:

- What is this keyword?
- This keyword utilization?

SUPER KEYWORD:

- What is super keyword?
- Super keyword utilization?

INTERFACES:

- What is interface?
- How to use interface in java?
- Marker interface?

ABSTRACT CLASSES:

- What is abstract class?
- How to use abstract class in java?

PACKAGES:

- What is package?
- Types of package?

ENUM:

1. What is enum?
2. Internal implementation of enum?
3. Declaration and usage of enum?
4. Enum vs interfaces?
5. Java.lang.Enum class?
6. Speciality of java enum?

WRAPPER CLASSES:

1. Need of wrapper classes?
2. Utility methods?
3. Autoboxing?
4. Autounboxing?

INNER CLASSES:

- What is inner class?
- Types of inner classes?

TYPES OF OBJECTS:

- Mutable objects
- Immutable objects

STRINGS:

- What is string
- String methods
- == and .equals()
- Cloning
- Shallow vs deep cloning
- Programs

STRING BUFFER:

- What is string buffer
- Method of stringbuffer
- Programs

STRING BUILDER:

- What is string builder
- String buffer vs string builder
- Method chaining

STRING TOKENIZER:

- What is string tokenizer
- Methods of string tokenizer

EXCEPTION HANDLING:

1. Exception vs error
2. Smooth termination vs abnormal termination
3. Runtime stack mechanism
4. Default exception handling
5. What is exception
6. Types of exception
7. Try block
8. Catch block
9. Methods to display exception details
10. Finally block
11. Final vs finally vs finalize
12. Try with resources
13. Throw statement
14. Throws statement
15. User defined exceptions

GARBAGE COLLECTOR:

- What is garbage collector
- How to call garbage collector in java

SINGLETON CLASS:

- What is singleton class
- How to create singleton class in java

SERIALIZATION AND DESERIALIZATION:

- Serialization
- Deserialization
- Transient keyword
- Static vs transient keyword
- Transient vs final keyword
- Object graph in serialization
- Custom serialization

JAVA.IO PACKAGE:

- File
- File writer
- File reader
- Buffered writer
- Buffered reader
- Print writer
- Programs

GENERICs:

- Need to generics

COLLECTIONS:

- Arrays vs collections
- Collection
- List
- Set
- Map
- Queue interface
- Types of cursors

MULTITHREADING:

- Thread vs process
- Multitasking
- Ways to create a thread in java
- Thread names
- Thread priorities
- Daemon thread
- Various ways to present a thread from execution
- Problems without synchronization
- Problems with synchronization
- Synchronization
- Inter thread communication
- Dead lock
- Disadvantages of multitasking

JAVA 8 FEATURES:

- Functional interface
- Lamda expression
- Default methods in interface
- Static methods in interface
- About null pointer exception
- Optional class
- Stream API
- For each() method
- Method reference
- programs